

Rapport de Projet - Mini-Projet CICS-VSAM

Identification

Candidat	Josué ROCHA
Formation	POEI Développeur Mainframe COBOL
Organisme	M2i Formation, Strasbourg
Période	19 Décembre 2025 - 22 Janvier 2026

Thème

Développement d'un mini-projet COBOL-CICS sous z/OS pour l'alimentation du Data Set CLIENT d'une institution financière.

Introduction

Ce projet a été réalisé dans le cadre de la formation POEI Développeur Mainframe COBOL. L'objectif est de mettre en pratique les compétences acquises en programmation COBOL-CICS et en gestion de fichiers VSAM en mode transactionnel.

Le projet consiste à développer un système de gestion de clientèle pour une institution financière, permettant :

- La consultation des informations client
 - L'ajout de nouveaux clients
 - La modification des données existantes
 - La suppression de clients
 - La navigation et les statistiques par région
-

Objectifs pédagogiques

Compétences visées

Domaine	Objectif
VSAM	Définir et manipuler des fichiers VSAM (KSDS, ESDS, RRDS), créer des index alternatifs (AIX/PATH)
CICS	Programmer des transactions avec gestion des commandes (SEND, RECEIVE, READ, WRITE, REWRITE, DELETE, STARTBR, READNEXT)
BMS	Concevoir des écrans (MAPs) avec attributs dynamiques, validation de saisie et messages d'erreur

Domaine	Objectif
JCL	Maîtriser les jobs de définition VSAM (IDCAMS), chargement de données, assemblage BMS et compilation COBOL-CICS
Administration CICS	Définir et gérer des ressources via CEDA (DEFINE, INSTALL), CEMT (SET, INQ) et déboguer avec CEDF

Progression pédagogique

Partie 0	Partie 1	Partie 2a-2c	Partie 3
Préparation (Libraries)	READ (Lecture)	WRITE, REWRITE DELETE (CRUD)	STARTBR/READNEXT (Navigation, AIX)
Environnement → Fondamentaux → Opérations CRUD → Techniques avancées			

Livrables du projet

Livrable	Quantité	Description
Programmes COBOL-CICS	7	Gestion complète du cycle de vie client
MAPs BMS	7	Écrans de saisie et d'affichage
Transactions CICS	7	AFFI, AJOU, MAJO, SUPP, DELG, LGEN, STAT
Fichiers VSAM	2	FCLIENT (KSDS), PCLIENT (PATH via AIX)
JCL	17	Définition, assemblage, compilation
Captures d'écran	160+	Documentation de chaque étape

Environnement de travail

Système et interface

Élément	Description
Système d'exploitation	z/OS sous émulateur Hercules (TK4-)
Interface utilisateur	TSO/ISPF pour le développement, CICS pour l'exécution
Gestionnaire transactionnel	CICS Transaction Server
Type de fichier	VSAM KSDS (Key-Sequenced Data Set)

Langages et technologies

Langage/Technologie	Usage
COBOL	Programmation des traitements métier
CICS (commandes)	Gestion transactionnelle (SEND, RECEIVE, READ, WRITE, etc.)
BMS (Assembleur)	Définition des écrans (MAPs)
JCL	Compilation des programmes et assemblage des MAPs
VSAM	Stockage et accès aux données

Libraries utilisées

Library	Contenu
ROCHA.CICS.SOURCE	Sources COBOL, MAPs BMS, JCL de compilation
ROCHA.CICS.LINK	Copybooks générés (DSECT des MAPs)
ROCHA.CICS.LOAD	Modules exécutables (programmes et MAPs)

Organisation du rapport

Note : Le document fourni présente les exercices de manière linéaire (exercices 1 à 19). J'ai choisi d'organiser ce rapport en parties thématiques pour une meilleure lisibilité et compréhension de la progression pédagogique.

Le projet est organisé en **4 parties** regroupant **19 exercices** :

Partie 0 : Préparation	
└─	Exercice 0 : Création des libraries CICS
Partie 1 : Affichage (READ)	
└─	Exercice 1 : Définition VSAM et intégration CICS
└─	Exercice 2 : MAP BMS d'affichage
└─	Exercice 3 : Programme COBOL-CICS (READ)
└─	Exercice 4 : Définition de la transaction
└─	Exercice 5 : Tests et débogage (CEDF)
Partie 2 : Opérations CRUD	
└─	2a – Ajout (WRITE)
└─	└─ Exercice 6 : MAP BMS d'ajout
└─	└─ Exercice 7 : Programme COBOL-CICS (WRITE)
└─	└─ Exercice 8 : Transaction d'ajout
└─	2b – Mise à jour (REWRITE)
└─	└─ Exercice 9 : MAP BMS de mise à jour
└─	└─ Exercice 10 : Programme COBOL-CICS (REWRITE)
└─	└─ Exercice 11 : Transaction de mise à jour
└─	2c – Suppression (DELETE)
└─	└─ Exercice 12 : MAP BMS de suppression

- └─ Exercice 13 : Programme COBOL-CICS (DELETE)
- └─ Exercice 14 : Transaction de suppression
- └─ Exercice 15 : Variante avec lecture préalable

Partie 3 : Opérations avancées

- └─ Exercice 16 : Création de clients génériques
- └─ Exercice 17 : Suppression par code générique (STARTBR, READNEXT, DELETE)
- └─ Exercice 18 : Liste générique paginée (10 clients/page)
- └─ Exercice 19 : Statistiques par région

Justification de ce découpage :

- **Partie 0** : Prérequis techniques avant de commencer
- **Partie 1** : Introduction aux concepts CICS avec une opération simple (lecture)
- **Partie 2** : Complète le CRUD avec les opérations d'écriture
- **Partie 3** : Techniques avancées de navigation et agrégation

Sommaire

Partie	Contenu
Partie 0 - Préparation	Exercice 0 : Création des Libraries
Partie 1 - Affichage	Exercices 1-5 : VSAM, MAP, READ
Partie 2a - Ajout	Exercices 6-8 : WRITE
Partie 2b - Mise à jour	Exercices 9-11 : REWRITE
Partie 2c - Suppression	Exercices 12-15 : DELETE
Partie 3 - Avancées	Exercices 16-19 : STARTBR, READNEXT
Conclusion	Bilan, annexes, références

Partie 0 : Préparation de l'environnement

Exercice 0 : Création des Libraries

Énoncé

Ce travail nécessite la création de trois Library pour stocker les membres à créer au cours de sa réalisation. Les Library à définir doivent porter le nom sous la forme suivante :

- **ROCHA.CICS.SOURCE** : Programmes COBOL et JCL
- **ROCHA.CICS.LINK** : Programmes objets (après compilation)
- **ROCHA.CICS.LOAD** : Programmes exécutables (après link-edit)

Mon travail

Avant de commencer le développement des programmes CICS, j'ai créé les trois libraries nécessaires via ISPF option 3.2 (Data Set Utility). Ces libraries sont des PDS (Partitioned Data Sets) qui contiendront tous les membres du projet.

Choix des caractéristiques :

- **Format d'enregistrement** : FB (Fixed Block) avec LRECL=80 pour les sources
- **Taille** : 10 tracks primaires, 5 secondaires (suffisant pour le projet)
- **Directory blocks** : 10 blocs pour l'index des membres

Résolution

Via ISPF 3.2 (Data Set Utility) :

```
Option ==> 3.2

DATA SET UTILITY

A - Allocate new data set

Data Set Name: ROCHA.CICS.SOURCE

Allocation Parameters:
  Management class . .
  Storage class . . . .
  Volume serial . . . .
  Device type . . . . .
  Data class . . . . .
  Space units . . . . . TRACK
  Primary quantity . . 10
  Secondary quantity . 5
  Directory blocks . . 10
  Record format . . . . FB
```

```
Record length . . . . 80
Block size . . . . . 27920
Data set name type . PDS
```

Répéter l'opération pour `ROCHA.CICS.LINK` et `ROCHA.CICS.LOAD`.

Captures d'écran

Liste des Data Sets créés

La commande DSLIST (option 3.4) permet de vérifier que les trois libraries ont été correctement créées sur le volume FDDBAS :

```
DSLIST - Data Sets Matching ROCHA.CICS                               Row 1 of 3
Command ==> _____ Scroll ==> CSR

Command - Enter "/" to select action                                Message                                Volume
-----
-          ROCHA.CICS.LINK                                           Info - I                                FDDBAS
          ROCHA.CICS.LOAD                                           FDDBAS
          ROCHA.CICS.SOURCE                                           FDDBAS
***** End of Data Set list *****
```

Cette capture montre les trois PDS créés : `ROCHA.CICS.LINK`, `ROCHA.CICS.LOAD` et `ROCHA.CICS.SOURCE`. La commande `"I"` (Info) permet d'afficher les caractéristiques détaillées de chaque Data Set.

Caractéristiques de ROCHA.CICS.LINK

```
Command ==> _____

Data Set Name . . . : ROCHA.CICS.LINK

General Data                                           Current Allocation
Volume serial . . . : FDDBAS                          Allocated tracks . : 10
Device type . . . . : 3390                            Allocated extents . : 1
Organization . . . . : PO                             Maximum dir. blocks : 10
Record format . . . . : FB
Record length . . . . : 80
Block size . . . . . : 27920
1st extent tracks . : 10
Secondary tracks . . : 5

                                           Current Utilization
                                           Used tracks . . . . : 1
                                           Used extents . . . . : 1
                                           Used dir. blocks . . : 1
                                           Number of members . : 0

                                           Dates
                                           Creation date . . . : 2026/01/12
                                           Referenced date . . : 2026/01/12
                                           Expiration date . . : ***None***
```

La library `LINK` contient les modules objets après compilation. Elle utilise le format `FB` (Fixed Block) avec une longueur d'enregistrement de 80 octets, le standard pour les fichiers source `z/OS`. La date de

création (2026/01/12) confirme l'allocation récente.

Caractéristiques de ROCHA.CICS.LOAD

```

Command ==> _____

Data Set Name . . . : ROCHA.CICS.LOAD

General Data
Volume serial . . . : FddbAS
Device type . . . . : 3390
Organization . . . . : PO
Record format . . . . : U
Record length . . . . : 0
Block size . . . . . : 27920
1st extent tracks . : 10
Secondary tracks . . : 5

Current Allocation
Allocated tracks . . : 10
Allocated extents . . : 1
Maximum dir. blocks : 10

Current Utilization
Used tracks . . . . . : 1
Used extents . . . . . : 1
Used dir. blocks . . . : 1
Number of members . . : 0

Dates
Creation date . . . . : 2026/01/12
Referenced date . . . : ***None***
Expiration date . . . : ***None***

```

La library LOAD contient les programmes exécutables (load modules). Elle utilise le format U (Undefined) car les modules exécutables n'ont pas de longueur d'enregistrement fixe. C'est le format standard pour les load modules z/OS.

Caractéristiques de ROCHA.CICS.SOURCE

```

Command ==> _____

Data Set Name . . . : ROCHA.CICS.SOURCE

General Data
Volume serial . . . : FddbAS
Device type . . . . : 3390
Organization . . . . : PO
Record format . . . . : FB
Record length . . . . : 80
Block size . . . . . : 27920
1st extent tracks . : 10
Secondary tracks . . : 5

Current Allocation
Allocated tracks . . : 10
Allocated extents . . : 1
Maximum dir. blocks : 10

Current Utilization
Used tracks . . . . . : 1
Used extents . . . . . : 1
Used dir. blocks . . . : 1
Number of members . . : 0

Dates
Creation date . . . . : 2026/01/12
Referenced date . . . : ***None***
Expiration date . . . : ***None***

```

La library SOURCE contiendra les programmes COBOL, les définitions BMS et les JCL. Comme LINK, elle utilise le format FB/80 pour la compatibilité avec les éditeurs ISPF et les compilateurs.

Structure des libraries

Library	Contenu	RECFM	LRECL
ROCHA.CICS.SOURCE	Programmes COBOL (.cbl), MAPs BMS (.bms), JCL, Copybooks	FB	80
ROCHA.CICS.LINK	Modules objets après compilation	FB	80
ROCHA.CICS.LOAD	Modules exécutables (load modules)	U	-

Membres créés dans SOURCE

Au cours du projet, les membres suivants ont été créés dans **ROCHA.CICS.SOURCE** :

Programmes COBOL :

Membre	Transaction	Description
PRGCLIA	AFFI	Affichage client (READ)
PRGAJT	AJOU	Ajout client (WRITE)
PRGMAJ	MAJO	Mise à jour client (REWRITE)
PRGSUP	SUPP	Suppression client (DELETE)
PRGDELG	DELG	Suppression générique (STARTBR/READNEXT/DELETE)
PRGLGEN	LGEN	Liste générique (STARTBR/READNEXT)
PRGSTAT	STAT	Statistiques par région

MAPs BMS :

Membre	Programme	Description
CLIAFF	PRGCLIA	Écran affichage client
CLIAJT	PRGAJT	Écran ajout client
CLIMAJ	PRGMAJ	Écran mise à jour client
CLISUP	PRGSUP	Écran suppression client
CLIDEL	PRGDELG	Écran suppression générique
CLILIST	PRGLGEN	Écran liste générique paginée
CLISTAT	PRGSTAT	Écran statistiques

JCL :

Membre	Usage	Exercice
DEFVSAM	Définition cluster VSAM	Ex 1

Membre	Usage	Exercice
LOADVSAM	Chargement données initiales	Ex 1
ASMCLAF	Assemblage MAP CLIAFF	Ex 2
CMPCLAF	Compilation PRGCLIA	Ex 3
ASMAJT	Assemblage MAP CLIAJT	Ex 6
CMPAJT	Compilation PRGAJT	Ex 7
ASMMAJ	Assemblage MAP CLIMAJ	Ex 9
CMPMAJ	Compilation PRGMAJ	Ex 10
ASMSUP	Assemblage MAP CLISUP	Ex 12
CMPSUP	Compilation PRGSUP	Ex 13
ASMDEL	Assemblage MAP CLIDEL	Ex 17
CMPDELG	Compilation PRGDELG	Ex 17
ASMLIST	Assemblage MAP CLILIST	Ex 18
CMPLGEN	Compilation PRGLGEN	Ex 18
DEFPATH	Définition AIX et PATH	Ex 19
ASMSTAT	Assemblage MAP CLISTAT	Ex 19
CMPSTAT	Compilation PRGSTAT	Ex 19

Note sur les copybooks : Les copybooks pour les MAPs BMS sont générés automatiquement lors de l'assemblage avec l'option **TYPE=DSECT**. Ils contiennent les structures de données avec les suffixes :

- **I** : Zone input (données reçues de l'écran)
- **O** : Zone output (données à envoyer)
- **L** : Longueur du champ saisi
- **A** : Attribut du champ (couleur, intensité, etc.)

Partie 1 : Création du Data Set et Affichage

Exercice 1 : Définition du Data Set CLIENT dans CICS

Énoncé

Définir le Data Set CLIENT dans la procédure de démarrage de CICS et comme ressource VSAM à utiliser par les programmes. Les opérations de lecture, écriture et suppression seront autorisées sur ce Data Set.

Mon travail

Cet exercice comporte deux étapes principales :

1. **Création du fichier VSAM** : J'ai utilisé IDCAMS pour définir un cluster KSDS avec une clé de 6 octets (numéro de compte) en position 0.
2. **Intégration dans CICS** : J'ai déclaré le fichier dans CICS via CEDA pour permettre les opérations READ, WRITE, REWRITE, DELETE et BROWSE.

Choix des paramètres VSAM :

- **KEYS(6 0)** : Clé de 6 caractères en début d'enregistrement (numéro compte)
- **RECORDSIZE(80 80)** : Enregistrements de taille fixe (80 octets) - compatible avec LRECL=80 par défaut du JCL
- **FREESPACE(20 10)** : Réserve de l'espace pour les insertions futures
- **SHAREOPTIONS(2 3)** : Permet le partage entre régions CICS

Note technique : Les enregistrements font 80 octets (64 données + 16 filler) pour être compatibles avec le LRECL=80 par défaut des DD * en JCL. Les programmes COBOL utiliseront un FILLER de 16 caractères en fin d'enregistrement.

Résolution

Étape 1 : JCL de définition VSAM (IDCAMS)

```
//ROCHA01 JOB (ACCT), 'DEF VSAM CLIENT', CLASS=A, MSGCLASS=X,
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID
//*****
//* DEFINITION DU DATA SET CLIENT (VSAM KSDS)
//*****
//*
//* ETAPE 1 : SUPPRESSION DU CLUSTER EXISTANT (SI EXISTE)
//*
//STEP1     EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN     DD *
DELETE ROCHA.CICS.CLIENT CLUSTER
SET MAXCC = 0
```

```
/*
/**
/** ETAPE 2 : DEFINITION DU CLUSTER VSAM KSDS
/**
//STEP2      EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN      DD *
    DEFINE CLUSTER (
        NAME(ROCHA.CICS.CLIENT)
        INDEXED
        VOLUMES(FDDBAS)
        KEYS(6 0)
        RECORDSIZE(80 80)
        TRACKS(5 5)
        FREESPACE(20 10)
        SHAREOPTIONS(2 3)
    )
    DATA (NAME(ROCHA.CICS.CLIENT.DATA))
    INDEX (NAME(ROCHA.CICS.CLIENT.INDEX))

/*
/**
/** ETAPE 3 : VERIFICATION DE LA CREATION
/**
//STEP3      EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN      DD *
    LISTCAT ENTRIES(ROCHA.CICS.CLIENT) ALL
/*
```

Étape 2 : Intégration dans CICS via CEDA

Sur l'émulateur TK4-, j'utilise CEDA en mode interactif pour définir le fichier. La commande **CEDA** **DEFINE** ouvre un écran de saisie où je renseigne les paramètres :

```
CEDA DEFINE FILE(FCLIENT) GROUP(CLIGROUP)
```

Cela affiche un écran de définition. Je renseigne les paramètres suivants :

Paramètre	Valeur	Description
DSName	ROCHA.CICS.CLIENT	Nom physique du dataset VSAM
Add	Yes	Autoriser WRITE (ajout)
Browse	Yes	Autoriser STARTBR/READNEXT
Delete	Yes	Autoriser DELETE
Read	Yes	Autoriser READ
Update	Yes	Autoriser REWRITE

Paramètre	Valeur	Description
RECORDFormat	Fixed	Format d'enregistrement fixe
RECORDSize	80	Taille de l'enregistrement
Keylength	6	Longueur de la clé

Après validation (ENTER), j'installe la ressource :

```
CEDA INSTALL FILE(FCLIENT) GROUP(CLIGROUP)
```

Note TK4- : Sur l'émulateur, certains paramètres comme STATUS et OPENTIME peuvent avoir des valeurs par défaut. Le fichier s'ouvre automatiquement lors du premier accès.

Note TK4- : Sur l'émulateur Hercules TK4-, une étape supplémentaire est nécessaire pour que CICS reconnaisse le fichier VSAM. Il faut ajouter une entrée dans le membre CICSTS51 de la bibliothèque ADCD.Z113F.PROCLIB (table de configuration CICS). Cette manipulation est spécifique à l'environnement d'émulation et ne serait pas requise sur un z/OS de production où la configuration CICS est gérée via CSD (CICS System Definition).

Étape 3 : Vérification avec CEMT

```
CEMT INQUIRE FILE(FCLIENT)
```

Résultat attendu :

```
FILE(FCLIENT)  Dsn(ROCHA.CICS.CLIENT)
                 Ena Ope Rea Upd Add Bro Del
                 Vsam Ksds
```

Commandes CEMT utiles pour la gestion du fichier :

Commande	Usage
CEMT SET FILE(FCLIENT) OPEN	Ouvrir le fichier
CEMT SET FILE(FCLIENT) ENABLED	Activer le fichier
CEMT SET FILE(FCLIENT) CLOSED	Fermer le fichier (pour maintenance VSAM)
CEMT INQ FILE(FCLIENT)	Vérifier l'état du fichier

Étape 4 : Chargement des données initiales

Le chargement utilise directement IDCAMS REPRO avec des enregistrements de 80 octets (64 données + 16 espaces en filler). Le DD * lit par défaut en LRECL=80, ce qui est maintenant compatible avec notre

définition VSAM.

```
//ROCHA02 JOB (ACCT),'LOAD VSAM CLIENT',CLASS=A,MSGCLASS=X,
//          MSGLEVEL=(1,1),NOTIFY=&SYSUID
//*****
//* CHARGEMENT DES DONNEES INITIALES DANS LE FICHIER CLIENT
//*
//* Structure enregistrement (80 octets) :
//*   Pos 01-06 : Numero compte (cle)
//*   Pos 07-08 : Code region
//*   Pos 09-10 : Nature compte
//*   Pos 11-20 : Nom client (10 car)
//*   Pos 21-30 : Prenom client (10 car)
//*   Pos 31-38 : Date naissance (AAAAMMJJ)
//*   Pos 39    : Sexe (M/F)
//*   Pos 40-41 : Activite professionnelle
//*   Pos 42    : Situation sociale (C/M/D/V)
//*   Pos 43-52 : Adresse (10 car)
//*   Pos 53-62 : Solde (10 car)
//*   Pos 63-64 : Position (DB/CR)
//*   Pos 65-80 : Filler (16 espaces)
//*****
//*
//STEP1    EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//INFILE   DD *
0000010120DUPONT      JEAN      19850315M10CPARIS      0000150000CR
0000020125MARTIN      MARIE     19900622F15MPARIS      0000080000DB
0000030220BERNARD     PIERRE    19780410M20CMARSEILLE 0000250000CR
0000040225PETIT       SOPHIE    19880912F05MMARSEILLE 0000045000DB
0000050330ROBERT      ALAIN     19750520M10CLYON      0000320000CR
0000060335RICHARD     CLAIRE    19920805F25VLYON      0000012000DB
0000070420DURAND      PAUL      19820718M30DLILLE      0000180000CR
0000080425MOREAU      ANNE      19950303F15CLILLE      0000095000DB
0000090130LAURENT     MARC      19800125M20MPARIS      0000420000CR
0000100235SIMON       JULIE     19870930F10CMARSEILLE 0000067000DB
2220010120LEROY       MICHEL    19830214M05CPARIS      0000145000CR
2220020125ROUX        NATHALIE 19910607F15MMARSEILLE 0000032000DB
2220030230DAVID       FRANCOIS 19760819M20CLYON      0000278000CR
2220040335BERTRAND    ISABELLE 19890423F25VLILLE      0000089000DB
2220050420MOREL       PHILIPPE 19840111M30DPARIS      0000156000CR
/*
//SYSIN     DD *
      REPRO INFILE(INFILE) -
            OUTDATASET(ROCHA.CICS.CLIENT)
/*
/*
//STEP2     EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN     DD *
      PRINT INDATASET(ROCHA.CICS.CLIENT) -
            CHARACTER
/*
```

```

/*
//STEP3      EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN      DD *
  LISTCAT ENTRIES(ROCHA.CICS.CLIENT) ALL
/*

```

Note technique : Les données font 64 caractères et sont automatiquement complétées à 80 caractères (padding avec des espaces) par le JCL. Le VSAM étant défini avec RECORDSIZE(80 80), les enregistrements sont compatibles.

Données chargées :

Type	Numéros	Quantité	Usage
Clients de base	000001-000010	10	Tests CRUD (Ex 3-15)
Clients 222xxx	222001-222005	5	Test READNEXT (Ex 18)

Répartition par région (pour Ex 19 - Statistiques) :

Région	Débiteurs	Créditeurs	Total
01 Paris	1	4	5
02 Marseille	2	2	4
03 Lyon	1	2	3
04 Lille	2	1	3

Note : Les clients 111xxx, 444xxx et 777xxx seront créés manuellement via la transaction AJOU dans l'exercice 16.

Captures d'écran

Création du cluster VSAM

Après exécution du JCL IDCAMS DEFINE, le cluster ROCHA.CICS.CLIENT est créé. La commande LISTCAT affiche les composants du cluster :

```

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
IDCAMS  SYSTEM SERVICES                                TIME: 11:59:55

  DEFINE CLUSTER (                                     -      00360009
    NAME(ROCHA.CICS.CLIENT)                           -      00370012
    INDEXED                                             -      00380009
    VOLUME(FDDBAS)                                     -      00381011
    KEYS(6 0)                                          -      00390009
    RECORDSIZE(64 64)                                -      00400009
    TRACKS(5 5)                                       -      00410009
    FREESPACE(20 10)                                  -      00420009
    SHAREOPTIONS(2 3)                                 -      00430009
    )                                                  -      00440009
    DATA (NAME(ROCHA.CICS.CLIENT.DATA))              -      00450012
    INDEX (NAME(ROCHA.CICS.CLIENT.INDEX))              -      00460012
IDC0508I DATA ALLOCATION STATUS FOR VOLUME FDDBAS IS 0
IDC0509I INDEX ALLOCATION STATUS FOR VOLUME FDDBAS IS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

```

Sortie IDCAMS montrant la création du cluster VSAM avec ses composants DATA et INDEX. Les paramètres KEYS(6 0) définissent la clé primaire (numéro de compte) en position 0 sur 6 octets.

Vérification des Data Sets créés

La commande DSLIST montre le cluster VSAM avec ses composants (DATA et INDEX) ainsi que les libraries du projet :

```

DSLIST - Data Sets Matching ROCHA.CICS                Row 1 of 6
Command ==> _____ Scroll ==> CSR

Command - Enter "/" to select action                Message                Volume
-----
      ROCHA.CICS.CLIENT                             *VSAM*
      ROCHA.CICS.CLIENT.DATA                         FDDBAS
      ROCHA.CICS.CLIENT.INDEX                       FDDBAS
      ROCHA.CICS.LINK                               FDDBAS
      ROCHA.CICS.LOAD                               FDDBAS
      ROCHA.CICS.SOURCE                             FDDBAS
***** End of Data Set list *****

```

Le cluster VSAM apparaît avec ses deux composants. Noter que VSAM gère automatiquement les composants DATA et INDEX - l'application accède uniquement au cluster via son nom (ROCHA.CICS.CLIENT).

Contenu du fichier après chargement

Le JCL de chargement utilise IDCAMS REPRO pour insérer les 15 enregistrements. La commande PRINT affiche le contenu :

```

PRINT INDATASET(ROCHA.CICS.CLIENT) - 0063000
CHARACTER 0064000
1IDCAMS SYSTEM SERVICES TIME: 12:42:3
- LISTING OF DATA SET -ROCHA.CICS.CLIENT
0KEY OF RECORD - 000001
0000010120DUPONT JEAN 19850315M10CPARIS 0000150000CR 0031000
0KEY OF RECORD - 000002
0000020125MARTIN MARIE 19900622F15MPARIS 0000080000DB 0032000
0KEY OF RECORD - 000003
0000030220BERNARD PIERRE 19780410M20CMARSEILLE 0000250000CR 0033000
0KEY OF RECORD - 000004
0000040225PETIT SOPHIE 19880912F05MMARSEILLE 0000045000DB 0034000
0KEY OF RECORD - 000005
0000050330ROBERT ALAIN 19750520M10CLYON 0000320000CR 0035000
0KEY OF RECORD - 000006
0000060335RICHARD CLAIRE 19920805F25VLYON 0000012000DB 0036000
0KEY OF RECORD - 000007
0000070420DURAND PAUL 19820718M30DLILLE 0000180000CR 0037000
0KEY OF RECORD - 000008

```

Premiers enregistrements chargés (000001 à 000008). La structure est visible : numéro compte (6), code région (2), nature compte (2), nom (10), prénom (10), etc.

```

0000080425MOREAU ANNE 19950303F15CLILLE 0000095000DB 0038000
0KEY OF RECORD - 000009
0000090130LAURENT MARC 19800125M20MPARIS 0000420000CR 0039000
0KEY OF RECORD - 000010
0000100235SIMON JULIE 19870930F10CMARSEILLE 0000067000DB 0040000
0KEY OF RECORD - 222001
2220010120LEROY MICHEL 19830214M05CPARIS 0000145000CR 0041000
0KEY OF RECORD - 222002
2220020125ROUX NATHALIE 19910607F15MMARSEILLE 0000032000DB 0042000
0KEY OF RECORD - 222003
2220030230DAVID FRANCOIS 19760819M20CLYON 0000278000CR 0043000
0KEY OF RECORD - 222004
2220040335BERTRAND ISABELLE 19890423F25VLILLE 0000089000DB 0044000
0KEY OF RECORD - 222005
2220050420MOREL PHILIPPE 19840111M30DPARIS 0000156000CR 0045000
0IDC0005I NUMBER OF RECORDS PROCESSED WAS 15
0IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
1IDCAMS SYSTEM SERVICES TIME: 12:42:3
0

```

Suite et fin des enregistrements (000009 à 222005). Les 15 clients sont correctement chargés, incluant les 5 clients 222xxx pour les tests de browse.

Visualisation VSAM avec DITTO

L'utilitaire DITTO/ESA permet de naviguer dans le fichier VSAM et visualiser les enregistrements :

DITTO/ESA for MVS VB - VSAM Browse

CATALOG

RBA 0 Key 000001 Col 1 Format CHAR

VOLSER FDDBAS Type KSDS DSNAME ROCHA.CICS.CLIENT

RBA	Len	<===5>	..10	5	...20	5	...30	5	...40	5	...50	5	...60
0	80	000001	0120	DUPONT	JEAN	1985	0315	M10C	PARIS	0000	1500		
80	80	000002	0125	MARTIN	MARIE	1990	0622	F15M	PARIS	0000	0800		
160	80	000003	0220	BERNARD	PIERRE	1978	0410	M20C	MARSEILLE	0000	2500		
240	80	000004	0225	PETIT	SOPHIE	1988	0912	F05M	MARSEILLE	0000	0450		
320	80	000005	0330	ROBERT	ALAIN	1975	0520	M10C	LYON	0000	3200		
400	80	000006	0335	RICHARD	CLAIRE	1992	0805	F25V	LYON	0000	0120		
480	80	000007	0420	DURAND	PAUL	1982	0718	M30D	LILLE	0000	1800		
560	80	000008	0425	MOREAU	ANNE	1995	0303	F15C	LILLE	0000	0950		
640	80	000009	0130	LAURENT	MARC	1980	0125	M20M	PARIS	0000	4200		
720	80	000010	0235	SIMON	JULIE	1987	0930	F10C	MARSEILLE	0000	0670		
800	80	222001	0120	LEROY	MICHEL	1983	0214	M05C	PARIS	0000	1450		

DITTO montre les enregistrements avec leur RBA (Relative Byte Address) et leur contenu. On voit clairement les champs : DUPONT JEAN, PARIS, 0000150000CR pour le premier client.

DITTO/ESA for MVS VB - VSAM Browse

CATALOG

RBA 880 Key 222002 Col 1 Format CHAR

VOLSER FDDBAS Type KSDS DSNAME ROCHA.CICS.CLIENT

RBA	Len	<===5>	..10	5	...20	5	...30	5	...40	5	...50	5	...60
880	80	222002	0125	ROUX	NATHALIE	1991	0607	F15M	MARSEILLE	0000	0320		
960	80	222003	0230	DAVID	FRANCOIS	1976	0819	M20C	LYON	0000	2780		
1040	80	222004	0335	BERTRAND	ISABELLE	1989	0423	F25V	LILLE	0000	0890		
1120	80	222005	0420	MOREL	PHILIPPE	1984	0111	M30D	PARIS	0000	1560		
**** End of data ****													

Fin du fichier avec "End of data" confirmant les 15 enregistrements chargés. Le dernier client est MOREL PHILIPPE (222005).

Définition du fichier dans CICS

La commande CEDA DEFINE FILE crée la ressource FCLIENT dans le groupe CLIGROUP :

```

OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA  DEFine File( FCLIENT  )
  File      : FCLIENT
  Group     : CLIGROUP
  DEScription ==>
VSAM PARAMETERS
  DSName     ==> ROCHA.CICS.CLIENT
  Password   ==>                                PASSWORD NOT SPECIFIED
  Rlsaccess  ==> No                               Yes ! No
  LSRPOOLId  : 1                                1-8 ! None
  LSRPOOLNum ==> 001                             1-255 ! None
  READInteg  ==> Uncommitted                     Uncommitted ! Consistent ! Repeatable
  DSNSharing ==> Allreqs                         Allreqs ! Modifyreqs
  STRings    ==> 001                             1-255
  Nsrgroup   ==>
REMOTE ATTRIBUTES
  REMOTESystem ==>
  REMOTENAME  ==>
+ REMOTE AND CFDATATABLE PARAMETERS

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                        TIME: 12.58.32  DATE: 01/12/26

```

Écran de définition du fichier CICS. Les paramètres importants : DSName=ROCHA.CICS.CLIENT, Rlsaccess=No (pas de Record Level Sharing), DSNSharing=Allreqs (partage entre régions autorisé).

Installation du fichier

Après définition, la ressource doit être installée pour être active :

```

INSTALL FILE(FCLIENT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA  Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  DOctemplate ==>
  Enqmodel    ==>
  File        ==> FCLIENT
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+ MApset      ==>

                                           SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL                        TIME: 12.59.36  DATE: 01/12/26

```

Message "INSTALL SUCCESSFUL" confirmant que le fichier est maintenant actif dans CICS. La date/heure de l'installation est enregistrée.

Vérification avec CEMT

La commande CEMT INQUIRE FILE permet de vérifier l'état du fichier :

```
INQUIRE FILE(FCLIENT)
STATUS: RESULTS - OVERTYPE TO MODIFY
Fil(FCLIENT ) Vsa Clo Ena Rea Upd Add Bro Del Sha
Dsn( ROCHA.CICS.CLIENT )
```

Le fichier est actif avec tous les droits : Vsa (VSAM), Clo (Closed au démarrage), Ena (Enabled), Rea (Read), Upd (Update), Add (Add), Bro (Browse), Del (Delete). Le statut "Sha" indique le partage activé.

Exercice 2 : Création de la MAP BMS pour affichage

Énoncé

Créer la MAP conformément à la structure du Data Set CLIENT permettant l'affichage des nouvelles données. Prévoir dans ce cadre le contrôle des données redondantes et une zone de message de 40 caractères pour afficher les informations nécessaires en cas d'erreur ou de saisie correcte.

Mon travail

J'ai créé une MAP BMS avec tous les champs du fichier CLIENT. La MAP comprend :

- Un titre en haut de l'écran
- Une zone de saisie pour le numéro de compte (clé de recherche)
- Les 12 champs d'affichage avec leurs libellés
- Des zones libellés pour afficher les descriptions (région, sexe, situation, position)
- Une zone de message de 60 caractères en bas
- Les touches fonction en bas de l'écran

Choix de conception :

- **CTRL=(FREEKB,FRSET)** : Clavier débloqué et MDT remis à zéro
- **TIOAPFX=YES** : Réserve 12 octets pour le préfixe TIOA (requis pour CICS)
- Seul le champ NUMCPT est saisissable (UNPROT), les autres sont en affichage (ASKIP)

Comprendre les concepts BMS :

Option	Signification	Rôle
FREEKB	Free Keyboard	Débloque le clavier après l'envoi de la MAP, permettant à l'utilisateur de saisir
FRSET	Flag Reset	Remet le MDT à zéro pour tous les champs
MDT	Modified Data Tag	Bit qui indique si un champ a été modifié par l'utilisateur
TIOAPFX	TIOA Prefix	Réserve 12 octets au début de la zone MAP pour le préfixe CICS
UNPROT	Unprotected	Champ saisissable par l'utilisateur

Option	Signification	Rôle
ASKIP	Auto-skip	Champ en affichage seul, le curseur le saute

Le MDT (Modified Data Tag) : Chaque champ de l'écran possède un bit MDT. Quand l'utilisateur modifie un champ, le MDT passe à 1. Lors du RECEIVE MAP, seuls les champs avec MDT=1 sont transmis au programme. L'option FRSET remet tous les MDT à 0 au SEND MAP, permettant de détecter les nouvelles modifications.

Résolution

MAP BMS : CLIAFF.bms

Le code source est stocké dans **ROCHA.CICS.SOURCE(CLIAFF)**. Voici le code complet :

```
*****
* MAPSET : CLIAFF - Affichage Client
* Transaction : AFFI
* Fil Rouge CICS - Exercice 2
*****
CLIAFF  DFHMSD TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,                                X
        STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
*****
MAPAFF  DFHMDI SIZE=(24,80),LINE=1,COLUMN=1
*-----
* TITRE
*-----
        DFHMDF POS=(1,25),LENGTH=30,ATTRB=(ASKIP,BRT),                                X
        INITIAL='*** AFFICHAGE CLIENT ***'
        DFHMDF POS=(2,1),LENGTH=78,ATTRB=ASKIP,                                    X
        INITIAL='-----'X
        -----'
*-----
* ZONE DE SAISIE - NUMERO DE COMPTE (CLE)
*-----
        DFHMDF POS=(4,2),LENGTH=16,ATTRB=ASKIP,                                    X
        INITIAL='NUMERO COMPTE :'
NUMCPT  DFHMDF POS=(4,19),LENGTH=6,ATTRB=(UNPROT,NUM,IC)
        DFHMDF POS=(4,26),LENGTH=1,ATTRB=ASKIP
*-----
* ZONES D'AFFICHAGE - DONNEES CLIENT
*-----
        DFHMDF POS=(6,2),LENGTH=16,ATTRB=ASKIP,                                    X
        INITIAL='CODE REGION  :'
CODREG  DFHMDF POS=(6,19),LENGTH=2,ATTRB=(ASKIP,BRT)
        DFHMDF POS=(6,25),LENGTH=20,ATTRB=ASKIP
LIBREG  DFHMDF POS=(6,46),LENGTH=15,ATTRB=(ASKIP,BRT)
*
        DFHMDF POS=(7,2),LENGTH=16,ATTRB=ASKIP,                                    X
        INITIAL='NATURE COMPTE :'
NATCPT  DFHMDF POS=(7,19),LENGTH=2,ATTRB=(ASKIP,BRT)
        DFHMDF POS=(7,25),LENGTH=20,ATTRB=ASKIP
```

```

LIBNAT  DFHMDf POS=(7,46),LENGTH=15,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(8,2),LENGTH=16,ATTRB=ASKIP,                                X
          INITIAL='NOM                      : '
NOM      DFHMDf POS=(8,19),LENGTH=10,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(9,2),LENGTH=16,ATTRB=ASKIP,                                X
          INITIAL='PRENOM                   : '
PRENOM   DFHMDf POS=(9,19),LENGTH=10,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(10,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='DATE  NAISSANCE: '
DATNA    DFHMDf POS=(10,19),LENGTH=10,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(11,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='SEXE                     : '
SEXE     DFHMDf POS=(11,19),LENGTH=1,ATTRB=(ASKIP,BRT)
          DFHMDf POS=(11,24),LENGTH=10,ATTRB=ASKIP
LIBSEX   DFHMDf POS=(11,35),LENGTH=8,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(12,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='ACTIVITE PRO  : '
ACTPRO   DFHMDf POS=(12,19),LENGTH=2,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(13,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='SITUATION SOC : '
SITSO    DFHMDf POS=(13,19),LENGTH=1,ATTRB=(ASKIP,BRT)
          DFHMDf POS=(13,24),LENGTH=10,ATTRB=ASKIP
LIBSIT   DFHMDf POS=(13,35),LENGTH=12,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(14,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='ADRESSE                 : '
ADRESSE  DFHMDf POS=(14,19),LENGTH=10,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(15,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='SOLDE                   : '
SOLDE    DFHMDf POS=(15,19),LENGTH=12,ATTRB=(ASKIP,BRT)
*
          DFHMDf POS=(16,2),LENGTH=16,ATTRB=ASKIP,                               X
          INITIAL='POSITION                 : '
POSIT    DFHMDf POS=(16,19),LENGTH=2,ATTRB=(ASKIP,BRT)
          DFHMDf POS=(16,25),LENGTH=10,ATTRB=ASKIP
LIBPOS   DFHMDf POS=(16,36),LENGTH=10,ATTRB=(ASKIP,BRT)
*-----
*  ZONE MESSAGE
*-----
          DFHMDf POS=(19,1),LENGTH=78,ATTRB=ASKIP,                                X
          INITIAL='-----X
          -----'
          DFHMDf POS=(20,2),LENGTH=10,ATTRB=ASKIP,INITIAL='MESSAGE : '
MSG      DFHMDf POS=(20,13),LENGTH=60,ATTRB=(ASKIP,BRT)
*-----
*  TOUCHES FONCTION
*-----

```

```

DFHMDF POS=(23,2),LENGTH=70,ATTRB=ASKIP,
INITIAL='ENTER=Rechercher PF3=Quitter CLEAR=Effacer'
*****
DFHMSD TYPE=FINAL
END

```

X

Zones de la MAP :

Zone	Longueur	Attribut	Description
NUMCPT	6	UNPROT,NUM,IC	Numéro compte (saisie)
CODREG	2	ASKIP,BRT	Code région
LIBREG	15	ASKIP,BRT	Libellé région
NATCPT	2	ASKIP,BRT	Nature compte
LIBNAT	15	ASKIP,BRT	Libellé nature
NOM	10	ASKIP,BRT	Nom client
PRENOM	10	ASKIP,BRT	Prénom client
DATNA	10	ASKIP,BRT	Date naissance
SEXE	1	ASKIP,BRT	Sexe
LIBSEX	8	ASKIP,BRT	Libellé sexe
ACTPRO	2	ASKIP,BRT	Activité professionnelle
SITSO	1	ASKIP,BRT	Situation sociale
LIBSIT	12	ASKIP,BRT	Libellé situation
ADRESSE	10	ASKIP,BRT	Adresse
SOLDE	12	ASKIP,BRT	Solde
POSIT	2	ASKIP,BRT	Position (DB/CR)
LIBPOS	10	ASKIP,BRT	Libellé position
MSG	60	ASKIP,BRT	Zone message

JCL d'assemblage : ASMCLAF.jcl

Choix de conception : J'ai créé un JCL d'assemblage par MAP BMS (ASMCLAF, ASMAJT, ASMMAJ, ASMSUP) plutôt qu'un JCL générique paramétrable. Ce choix permet :

- Une meilleure traçabilité dans SDSF (nom de job explicite)
- Une modification indépendante si une MAP nécessite des options spécifiques
- Une simplicité d'utilisation (pas de substitution de variables)

```
//ROCHA03 JOB (ACCT),'ASSEMBL BMS CLIAFF',CLASS=A,MSGCLASS=X,
//                MSGLEVEL=(1,1),NOTIFY=&SYSUID
//*****
//* ASSEMBLAGE DE LA MAP BMS CLIAFF (AFFICHAGE CLIENT)
//*
//* Ce JCL assemble le source BMS et génère :
//*   - Le module MAP physique dans ROCHA.CICS.LOAD
//*   - Le copybook DSECT dans ROCHA.CICS.LINK
//*****
//PROC MAN JCLLIB ORDER=(DFH510.CICS.SDFHPROC,ROCHA.CICS.SOURCE,
//                ROCHA.CICS.LINK,ROCHA.CICS.LOAD)
//*
//ASSEM      EXEC DFHMAPS,INDEX='DFH510.CICS',
//                MAPLIB='ROCHA.CICS.LOAD',
//                DSCTLIB='ROCHA.CICS.LINK',
//                MAPNAME='CLIAFF',RMODE=24
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD DSN=ROCHA.CICS.SOURCE(CLIAFF),DISP=SHR
/*
```

Note : La procédure DFHMAPS génère automatiquement le module physique (MAP) et le copybook COBOL (DSECT). Le copybook sera stocké dans ROCHA.CICS.LINK avec le nom du mapset. Attention à ne pas utiliser la même library pour le source et le DSECT, sinon le source sera écrasé !

Structure du copybook généré (DSECT) :

Pour chaque champ nommé dans la MAP BMS (ex: NUMCPT), le copybook généré contient plusieurs variables avec des suffixes :

Suffixe	Type	Description	Exemple
L	S9(04) COMP	Longueur des données reçues	NUMCPTL
F	X(01)	Flag (usage interne)	NUMCPTF
A	X(01)	Attribut dynamique	NUMCPTA
I	X(nn)	Zone input (données reçues)	NUMCPTI
O	X(nn)	Zone output (données à envoyer)	NUMCPTO

Important : Avec **STORAGE=AUTO**, les zones I et O partagent la même mémoire. Après un **RECEIVE MAP**, sauvegarder les valeurs importantes avant de faire **MOVE LOW-VALUES**.

Captures d'écran

Assemblage du source BMS

Le JCL d'assemblage utilise la procédure DFHMAPS pour générer le module physique et le copybook DSECT :

```

SDSF OUTPUT DISPLAY ROCHA03 JOB00076 DSID 104 LINE 322 COLUMNS 02- 81
COMMAND INPUT ==> _ SCROLL ==> CSR

      No Statements Flagged in this Assembly
HIGH LEVEL ASSEMBLER, 5696-234, RELEASE 6.0, PTF UK92594
SYSTEM: z/OS 01.13.00 JOBNAME: ROCHA03 STEPNAME: ASSEM PRO
Data Sets Allocated for this Assembly
Con DDname Data Set Name Volume Member
P1 SYSIN SYS26012.T145428.RA000.ROCHA03.TEMPM.H01
L1 SYSLIB DFH510.CICS.SDFHMAC FDC511
L2 SYS1.MACLIB FDRES1
SYSPRINT IBMUSER.ROCHA03.JOB00076.D0000104.?
SYSPUNCH ROCHA.CICS.LINK FDDBAS CLIAFF

31964K allocated to Buffer Pool Storage required 1072K
96 Primary Input Records Read 6922 Library Records Read
0 ASMAOPT Records Read 339 Primary Print Records Written
130 Object Records Written 0 ADATA Records Written
Assembly Start Time: 14.54.29 Stop Time: 14.54.29 Processor Time: 00.00.00.2863
Return Code 000
***** BOTTOM OF DATA *****

```

Sortie de l'assembleur High Level Assembler. Le "Return Code 000" confirme l'assemblage réussi. On voit les fichiers utilisés : SYSLIB (macros CICS), SYSPUNCH (sortie vers ROCHA.CICS.LINK membre CLIAFF).

Copybook généré dans ROCHA.CICS.LINK

L'assemblage génère automatiquement le copybook COBOL (DSECT) dans la library LINK :

```

Menu Functions Confirm Utilities Help
EDIT ROCHA.CICS.LINK Row 00001 of 00001
Command ==> Scroll ==> CSR
Name Prompt Size Created Changed ID
CLIAFF
**End**

```

Le membre CLIAFF est créé dans ROCHA.CICS.LINK. Ce copybook contient les structures de données avec les suffixes L, F, A, I, O pour chaque champ de la MAP.

Définition du MAPSET dans CICS

La commande CEDA DEFINE MAPSET déclare l'écran BMS comme ressource CICS :


```

DEFINE MAPSET(CLIAFF) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFine MApset( CLIAFF  )
  MApset      : CLIAFF
  Group       : CLIGROUP
  DEScription ==> _
  REsident    ==> No                No ! Yes
  USAge       ==> Normal            Normal ! Transient
  USElpacopy  ==> No                No ! Yes
  Status      ==> Enabled            Enabled ! Disabled
  RSl         : 00                  0-24 ! Public
DEFINITION SIGNATURE
  DEFInetime  : 01/12/26 17:59:42
  CHANGETime  : 01/12/26 17:59:42
  CHANGEUsrid : CICSUSER
  CHANGEAGEnt : CSDApi              CSDApi ! CSDBatch
  CHANGEAGRel : 0680

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                           TIME: 17.59.42  DATE: 01/12/26

```

Définition du mapset CLIAFF dans le groupe CLIGROUP. USAge=Normal signifie que le mapset sera chargé à la première utilisation et déchargé après un certain temps d'inactivité.

Vérification de la définition

La commande CEDA VIEW permet de vérifier les caractéristiques du mapset :

```

VIEW MAPSET(CLIAFF) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View MApset( CLIAFF  )
  MApset      : CLIAFF
  Group       : CLIGROUP
  DEScription :
  REsident    : No                No ! Yes
  USAge       : Normal            Normal ! Transient
  USElpacopy  : No                No ! Yes
  Status      : Enabled            Enabled ! Disabled
  RSl         : 00                  0-24 ! Public
DEFINITION SIGNATURE
  DEFInetime  : 01/12/26 17:59:42
  CHANGETime  : 01/12/26 17:59:42
  CHANGEUsrid : CICSUSER
  CHANGEAGEnt : CSDApi              CSDApi ! CSDBatch
  CHANGEAGRel : 0680

```

Caractéristiques du mapset : Status=Enabled, DEFInetime indique la date/heure de création, CHANGEUsrid montre l'utilisateur qui a créé la définition (CICSUSER).

Exercice 3 : Programme COBOL-CICS d'affichage

Énoncé

Créer le PROGRAMME nécessaire pour l'affichage des données pour un code CLIENT saisi. Il doit permettre une saisie multiple de code CLIENT jusqu'à fin de saisie d'affichage de la part de l'utilisateur. De même, il faut accompagner chaque anomalie ou action par un message d'information ou d'avertissement.

Mon travail

J'ai développé un programme COBOL-CICS en mode **pseudo-conversationnel**.

Pourquoi le mode pseudo-conversationnel ?

J'ai choisi ce mode pour plusieurs raisons :

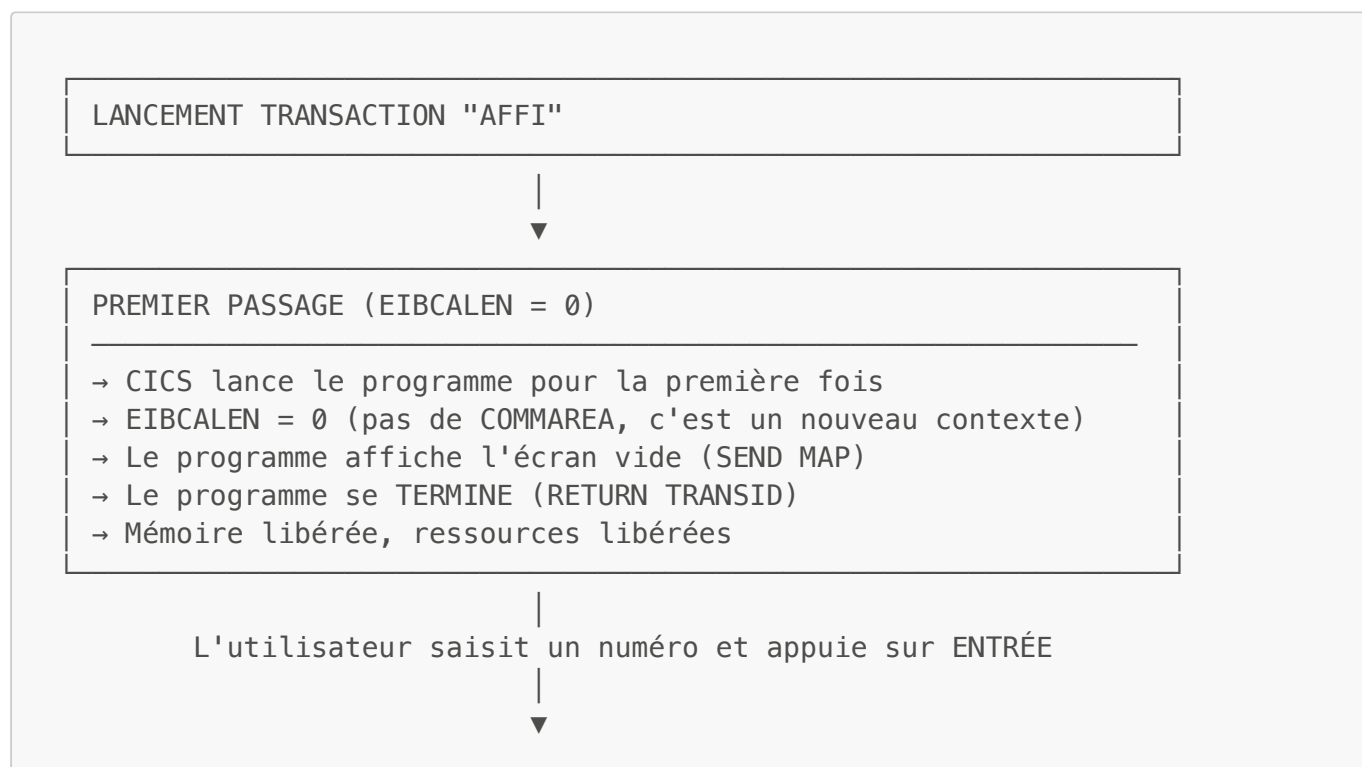
- **Optimisation des ressources** : Le programme libère la mémoire entre chaque interaction utilisateur
- **Exigence de l'énoncé** : "Saisie multiple jusqu'à fin de saisie" implique plusieurs allers-retours écran
- **Bonne pratique CICS** : C'est le mode standard pour les transactions interactives en production
- **Scalabilité** : Permet de supporter de nombreux utilisateurs simultanés

Comprendre le mode pseudo-conversationnel

En CICS, un programme ne reste pas en mémoire pendant que l'utilisateur réfléchit. Au lieu de cela :

1. **Le programme s'exécute** : traite les données, affiche un écran
2. **Le programme se TERMINE** : libère la mémoire et les ressources
3. **L'utilisateur saisit** : pendant ce temps, le programme n'existe plus
4. **CICS relance le programme** : quand l'utilisateur appuie sur une touche

C'est le mode **pseudo-conversationnel** : l'utilisateur a l'impression d'une conversation continue, mais en réalité le programme est relancé à chaque interaction.



PASSAGES SUIVANTS (EIBCALEN > 0)

- CICS relance le programme (nouveau processus)
- EIBCALEN > 0 (la COMMAREA indique un contexte existant)
- Le programme reçoit la saisie (RECEIVE MAP)
- Le programme lit le fichier (READ FILE)
- Le programme affiche le résultat (SEND MAP)
- Le programme se TERMINE à nouveau (RETURN TRANSID)

Variables clés du EIB (Exec Interface Block)

CICS fournit un bloc de données appelé EIB contenant des informations sur le contexte :

Variable	Description	Valeurs typiques
EIBCALEN	Longueur de la COMMAREA	0 = premier passage, >0 = passage suivant
EIBAID	Touche appuyée	DFHENTER, DFHPPF3, DFHCLEAR
EIBTRNID	Code transaction	'AFFI'
EIBRESP	Code réponse dernière commande	0=OK, 13=NOTFND

Logique du programme

Le programme utilise un **EVALUATE** pour aiguiller selon le contexte :

Condition	Action	Paragraphe
EIBCALEN = 0	Premier passage, afficher écran vide	1000-PREMIER-PASSAGE
EIBAID = DFHPPF3	Touche PF3, quitter	9000-FIN-PROGRAMME
EIBAID = DFHCLEAR	Touche CLEAR, réinitialiser	1000-PREMIER-PASSAGE
Autre (ENTER)	Traiter la saisie	2000-TRAITEMENT

Points techniques importants

- **DFHAID** : Copybook contenant les constantes des touches (DFHPPF3, DFHCLEAR, DFHENTER)
- **SEND TEXT** : Nécessite une variable, pas une constante littérale (**FROM(W S-MSG)** pas **FROM('texte')**)
- **Copybook BMS** : Généré par l'assemblage, contient MAPAFFI (input) et MAPAFFO (output)

Résolution

Programme : PRGCLIA.cbl

Le code source est stocké dans **ROCHA.CICS.SOURCE(PRGCLIA)**. Voici le code complet :

IDENTIFICATION DIVISION.

PROGRAM-ID. PRGCLIA.

* PROGRAMME : PRGCLIA

* FONCTION : Affichage d'un client par numéro de compte

* TRANSACTION : AFFI

* FICHER : FCLIENT (VSAM KSDS)

* MAP : MAPAFF (MAPSET CLIAFF)

*

* MODE PSEUDO-CONVERSATIONNEL :

* - Premier passage : Affiche écran vide

* - Passages suivants : Lit et affiche le client

* - PF3 : Quitter la transaction

*

* FIL ROUGE CICS - EXERCICE 3

ENVIRONNEMENT DIVISION.

CONFIGURATION SECTION.

DATA DIVISION.

WORKING-STORAGE SECTION.

*-----

* ZONE DE COMMUNICATION (COMMAREA)

*-----

01 WS-COMMAREA.

05 WS-FLAG-INIT PIC X(01) VALUE 'N'.

88 PREMIER-PASSAGE VALUE 'N'.

88 PASSAGE-SUIVANT VALUE '0'.

*-----

* COPYBOOKS CICS

*-----

COPY DFHAID.

*-----

* COPYBOOK GENERE PAR ASSEMBLAGE BMS (DSECT)

* Stocké dans ROCHA.CICS.LINK(CLIAFF)

*-----

COPY CLIAFF.

*-----

* STRUCTURE ENREGISTREMENT CLIENT (80 OCTETS)

*-----

01 ENR-CLIENT.

05 CLI-NUMCPT PIC X(06).

05 CLI-CODREG PIC X(02).

05 CLI-NATCPT PIC X(02).

05 CLI-NOM PIC X(10).

05 CLI-PRENOM PIC X(10).

05 CLI-DATNAISS PIC X(08).

05 CLI-SEXE PIC X(01).

05 CLI-ACTPRO PIC X(02).

05 CLI-SITSO PIC X(01).

```

05 CLI-ADRESSE          PIC X(10).
05 CLI-SOLDE            PIC X(10).
05 CLI-POSITION         PIC X(02).
05 FILLER               PIC X(16).

```

```

*-----
* VARIABLES DE TRAVAIL

```

```

*-----
01 WS-RESP              PIC S9(08) COMP VALUE 0.
01 WS-NUMCPT            PIC X(06) VALUE SPACES.
01 WS-MSG-FIN           PIC X(40)
    VALUE 'TRANSACTION AFFI TERMINEE - AU REVOIR'.

```

```

*****
PROCEDURE DIVISION.
*****

```

```

*-----
0000-PRINCIPAL.

```

```

*-----
* Point d'entrée du programme
*-----

```

```

    EVALUATE TRUE
        WHEN EIBCALEN = 0
            PERFORM 1000-PREMIER-PASSAGE
        WHEN EIBAID = DFHPF3
            PERFORM 9000-FIN-PROGRAMME
        WHEN EIBAID = DFHCLEAR
            PERFORM 1000-PREMIER-PASSAGE
        WHEN OTHER
            PERFORM 2000-TRAITEMENT
    END-EVALUATE

    EXEC CICS RETURN
        TRANSID('AFFI')
        COMMAREA(WS-COMMAREA)
        LENGTH(LENGTH OF WS-COMMAREA)
    END-EXEC.

```

```

*-----
1000-PREMIER-PASSAGE.

```

```

*-----
* Affichage de l'écran vide avec message de saisie
*-----

```

```

    MOVE LOW-VALUES TO MAPAFF0
    MOVE 'SAISIR LE NUMERO DE COMPTE ET APPUYER SUR ENTREE'
        TO MSG0
    MOVE '0' TO WS-FLAG-INIT

    EXEC CICS SEND MAP('MAPAFF')
        MAPSET('CLIAFF')
        ERASE
    END-EXEC.

```

```
*-----
2000-TRAITEMENT.
*-----
* Réception des données et recherche du client
*-----
      EXEC CICS RECEIVE MAP('MAPAFF')
          MAPSET('CLIAFF')
          RESP(WS-RESP)
      END-EXEC

* Gestion MAPFAIL (aucune donnée transmise)
      IF WS-RESP = DFHRESP(MAPFAIL)
          MOVE LOW-VALUES TO MAPAFFO
          MOVE 'ERREUR RECEPTION - RESSAISIR' TO MSGO
          EXEC CICS SEND MAP('MAPAFF')
              MAPSET('CLIAFF')
              ERASE
          END-EXEC
          GO TO 2000-FIN
      END-IF

* Vérifier que le numéro de compte est saisi
      IF NUMCPTL = 0 OR NUMCPTI = SPACES
          MOVE LOW-VALUES TO MAPAFFO
          MOVE 'VEUILLEZ SAISIR UN NUMERO DE COMPTE' TO MSGO
          EXEC CICS SEND MAP('MAPAFF')
              MAPSET('CLIAFF')
          END-EXEC
          GO TO 2000-FIN
      END-IF

* Préparer la clé de recherche
      MOVE NUMCPTI TO WS-NUMCPT

* Lecture du fichier VSAM
      EXEC CICS READ
          FILE('FCLIENT')
          INTO(ENR-CLIENT)
          RIDFLD(WS-NUMCPT)
          RESP(WS-RESP)
      END-EXEC

* Traitement du résultat
      EVALUATE WS-RESP
          WHEN DFHRESP(NORMAL)
              PERFORM 3000-AFFICHER-CLIENT
          WHEN DFHRESP(NOTFND)
              MOVE LOW-VALUES TO MAPAFFO
              MOVE WS-NUMCPT TO NUMCPTO
              MOVE 'CLIENT INEXISTANT - VERIFIEZ LE NUMERO'
                  TO MSGO
          WHEN OTHER
              MOVE LOW-VALUES TO MAPAFFO
              MOVE 'ERREUR LECTURE FICHIER - CONTACTEZ SUPPORT'
```

```
                TO MSGO
END-EVALUATE

EXEC CICS SEND MAP('MAPAFF')
      MAPSET('CLIAFF')
END-EXEC.

2000-FIN.
EXIT.

*-----
3000-AFFICHER-CLIENT.
*-----
* Transfert des données du fichier vers la MAP
*-----
      MOVE LOW-VALUES TO MAPAFFO

* Données directes
      MOVE CLI-NUMCPT    TO NUMCPTO
      MOVE CLI-CODREG    TO CODREGO
      MOVE CLI-NATCPT    TO NATCPTO
      MOVE CLI-NOM       TO NOMO
      MOVE CLI-PRENOM    TO PRENOMO
      MOVE CLI-DATNAISS  TO DATNAO
      MOVE CLI-SEXE      TO SEXEO
      MOVE CLI-ACTPRO    TO ACTPROO
      MOVE CLI-SITSO     TO SITSOO
      MOVE CLI-ADRESSE   TO ADRESSEO
      MOVE CLI-SOLDE     TO SOLDEO
      MOVE CLI-POSITION  TO POSITO

* Libellé région
      EVALUATE CLI-CODREG
        WHEN '01' MOVE '01 - PARIS' TO LIBREGO
        WHEN '02' MOVE '02 - MARSEILLE' TO LIBREGO
        WHEN '03' MOVE '03 - LYON' TO LIBREGO
        WHEN '04' MOVE '04 - LILLE' TO LIBREGO
        WHEN OTHER MOVE 'REGION INCONNUE' TO LIBREGO
      END-EVALUATE

* Libellé sexe
      EVALUATE CLI-SEXE
        WHEN 'M' MOVE 'MASCULIN' TO LIBSEXO
        WHEN 'F' MOVE 'FEMININ' TO LIBSEXO
        WHEN OTHER MOVE 'INCONNU' TO LIBSEXO
      END-EVALUATE

* Libellé situation sociale
      EVALUATE CLI-SITSO
        WHEN 'C' MOVE 'CELIBATAIRE' TO LIBSITO
        WHEN 'M' MOVE 'MARIE(E)' TO LIBSITO
        WHEN 'D' MOVE 'DIVORCE(E)' TO LIBSITO
        WHEN 'V' MOVE 'VEUF(VE)' TO LIBSITO
        WHEN OTHER MOVE 'INCONNU' TO LIBSITO
```

```

END-EVALUATE

* Libellé position
  EVALUATE CLI-POSITION
    WHEN 'CR' MOVE 'CREDITEUR' TO LIBPOS0
    WHEN 'DB' MOVE 'DEBITEUR' TO LIBPOS0
    WHEN OTHER MOVE 'INCONNU' TO LIBPOS0
  END-EVALUATE

  MOVE 'CLIENT TROUVE - PF3=QUITTER OU NOUVELLE RECHERCHE'
    TO MSGO.

*-----
  9000-FIN-PROGRAMME.
*-----
* Fin de la transaction
*-----

  EXEC CICS SEND TEXT
    FROM(WS-MSG-FIN)
    LENGTH(40)
    ERASE
  END-EXEC

  EXEC CICS RETURN
  END-EXEC.

```

JCL de compilation : CMPCLAF.jcl

```

//ROCHA04 JOB (ACCT),'COMPILE PRGCLIA',CLASS=A,MSGCLASS=X,
//          MSGLEVEL=(1,1),NOTIFY=&SYSUID
//*****
//* COMPILATION DU PROGRAMME COBOL-CICS PRGCLIA
//*****
//PROC MAN  JCLLIB ORDER=(DFH510.CICS.SDFHPROC,ROCHA.CICS.SOURCE,
//          ROCHA.CICS.LINK,ROCHA.CICS.LOAD)
//*
//COMPIL    EXEC PROC=DFHYITVL,
//          INDEX='DFH510.CICS',
//          PROGLIB='ROCHA.CICS.LOAD',
//          AD370HLQ='IGY420',
//          DSCTLIB='ROCHA.CICS.LINK',
//          LE370HLQ='CEE'
//TRN.SYSIN DD DSN=ROCHA.CICS.SOURCE(PRGCLIA),DISP=SHR
//LKED.SYSIN DD *
//          INCLUDE SYSLIB(DFHELII)
//          NAME PRGCLIA(R)
//*

```

Commandes CICS utilisées :

Commande	Usage
SEND MAP	Envoyer l'écran
RECEIVE MAP	Recevoir la saisie
READ FILE	Lire VSAM par clé
RETURN TRANSID	Retour pseudo-conversationnel
SEND TEXT	Message de fin

Structure du programme :

Paragraphe	Fonction
0000-PRINCIPAL	Point d'entrée, aiguillage selon EIBCALEN et EIBAUD
1000-PREMIER-PASSAGE	Affichage de l'écran vide
2000-TRAITEMENT	Réception saisie, lecture VSAM, affichage résultat
3000-AFFICHER-CLIENT	Transfert données vers MAP avec conversion libellés
9000-FIN-PROGRAMME	Message de fin et RETURN sans TRANSID

Captures d'écran

Modules compilés dans ROCHA.CICS.LOAD

Après compilation du programme COBOL et assemblage de la MAP, les modules exécutables sont stockés dans ROCHA.CICS.LOAD :

EDIT ROCHA.CICS.LOAD Row 00001 of 00002								
Command ==> Scroll ==> CSR								
	Name	Prompt	Alias-of	Size	TTR	AC	AM	RM
	CLIAFF			00000690	000021	00	31	24
	PRGCLIA			00001B90	000028	00	31	ANY
	End							

La library LOAD contient les deux modules : CLIAFF (module physique de la MAP BMS) et PRGCLIA (programme COBOL compilé). La taille et le TTR (Track Table Record) confirment que les modules sont bien générés.

Compilation COBOL réussie

Le JCL de compilation utilise la procédure DFHYITVL qui effectue la traduction CICS, la compilation COBOL et le link-edit :

```

SDSF OUTPUT DISPLAY ROCHA04  JOB00085  DSID   103 LINE 1,418  COLUMNS 02- 81
COMMAND INPUT ===>          SCROLL ===> CSR
0003EC  (LIT+672)   40D4C1D7 C1C6C640 D7D9C7C3 D3C9C1C4 C2000000 00000001 2C000
00040C  (LIT+704)   30000000 03000000 00000000 00000000 00000000 00000000 000000
00042C  (LIT+736)   00000000 00000000 00800000 00400000 00000000 00000000 00400
00044C  (LIT+768)   00000000 00000000 00400000 000025C0 0001C000 07080000 2C02C
00046C  (LIT+800)   40C00001 40000708 00002C02 C402C000 07080000 2C02C4

TGT      WILL BE ALLOCATED FOR 00000170 BYTES
SPEC-REG WILL BE ALLOCATED FOR 0000007E BYTES
WRK-STOR WILL BE ALLOCATED FOR 00000B4D BYTES
DSA      WILL BE ALLOCATED FOR 00000160 BYTES
CONSTANT GLOBAL TABLE FOR DYNAMIC STORAGE INITIALIZATION AT LOCATION 000D50
INITD CODE FOR DYNAMIC STORAGE INITIALIZATION BEGINS AT LOCATION 000EB0 FOR LENG
* Statistics for COBOL program PRGCLIA:
*   Source records = 622
*   Data Division statements = 261
*   Procedure Division statements = 80
End of compilation 1, program PRGCLIA, no statements flagged.
Return code 0
***** BOTTOM OF DATA *****

```

Statistiques de compilation : 622 source records, 261 Data Division statements, 80 Procedure Division statements. Le "Return code 0" confirme une compilation sans erreur.

Définition du programme dans CICS

La commande CEDA DEFINE PROGRAM déclare le programme compilé :

```

DEFINE PROGRAM(PRGCLIA) GROUP(CLIGROUP) LANGUAGE(COBOL)
OVERTYPE TO MODIFY                                     CICS RELEASE = 0680
CEDA DEFine PROGram( PRGCLIA )
  PROGram      : PRGCLIA
  Group        : CLIGROUP
  DEScription  ==> _
  Language     ==> CObol          CObol ! Assembler ! Le370 ! C ! PlI
  REload       ==> No             No ! Yes
  RESident     ==> No             No ! Yes
  USAge        ==> Normal         Normal ! Transient
  USElpacopy   ==> No             No ! Yes
  Status       ==> Enabled        Enabled ! Disabled
  RSl          : 00              0-24 ! Public
  CEdf         ==> Yes            Yes ! No
  DATalocation ==> Below          Below ! Any
  EXECKey      ==> User           User ! Cics
  COncurrency  ==> Quasirent      Quasirent ! Threadsafe ! Required
  Api          ==> Cicsapi        Cicsapi ! Openapi
  REMOTE ATTRIBUTES
+  DYNamic     ==> No             No ! Yes

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 17.06.28 DATE: 01/12/26

```

Définition du programme PRGCLIA avec Language=CObol. Les paramètres CEdf=Yes permettent le debug avec CEDF, DATalocation=Below indique que les données seront allouées sous la barre des 16 Mo (compatible 24-bit).

Vérification du programme

La commande CEMT INQUIRE PROGRAM vérifie l'état du programme :

```
INQ PROGRAM(PRGCLIA)
STATUS: RESULTS - OVERTYPE TO MODIFY
Prog(PRGCLIA ) Leng(0000000000) Cob Pro Ena Pri      Ced
Res(0000) Use(0000000000) Bel Uex Ful Qua Cic
```

Le programme PRGCLIA est actif : Cob (COBOL), Pro (Protected), Ena (Enabled), Pri (Private). Leng indique la taille du module en mémoire.

Exercice 4 : Création de la transaction via CEDA

Énoncé

Créer la transaction correspondante à l'opération d'affichage des données de CLIENT avec l'interface CICS en utilisant la commande CEDA. Mettre éventuellement le GROUP et la LIST à jour en cas de besoin.

Mon travail

Pour qu'une transaction CICS fonctionne, plusieurs ressources doivent être définies et liées :

1. **FILE** : Le fichier VSAM (déjà défini dans l'exercice 1)
2. **MAPSET** : Le module BMS compilé (écran physique)
3. **PROGRAM** : Le programme COBOL-CICS compilé
4. **TRANSACTION** : Le code de 4 caractères qui lance le programme

Ces ressources sont regroupées dans un GROUP (ici CLIGROUP) qui permet de les gérer ensemble. L'ordre de définition est important car la transaction référence le programme.

Résolution

Étape 1 : Définition des nouvelles ressources

Le fichier FCLIENT étant déjà défini et installé (exercice 1), je définis uniquement les nouvelles ressources :

```
CEDA DEFINE MAPSET(CLIAFF) GROUP(CLIGROUP)

CEDA DEFINE PROGRAM(PRGCLIA) GROUP(CLIGROUP)
LANGUAGE(COBOL)

CEDA DEFINE TRANSACTION(AFFI) GROUP(CLIGROUP)
PROGRAM(PRGCLIA)
```

Étape 2 : Installation des ressources

Option A : Installation individuelle (recommandée)

Cette méthode évite les erreurs si certaines ressources sont déjà installées :

```
CEDA INSTALL MAPSET(CLIAFF) GROUP(CLIGROUP)
CEDA INSTALL PROGRAM(PRGCLIA) GROUP(CLIGROUP)
CEDA INSTALL TRANSACTION(AFFI) GROUP(CLIGROUP)
```

Option B : Installation du groupe complet

```
CEDA INSTALL GROUP(CLIGROUP)
```

Note : Si FCLIENT est déjà installé (exercice 1), cette commande affichera une erreur "ALREADY INSTALLED" pour le fichier. C'est normal et les autres ressources seront quand même installées.

Tableau récapitulatif des ressources du groupe CLIGROUP :

Ressource	Nom	Défini dans	Description
FILE	FCLIENT	Exercice 1	Fichier VSAM CLIENT
MAPSET	CLIAFF	Exercice 4	Écran BMS d'affichage
PROGRAM	PRGCLIA	Exercice 4	Programme COBOL-CICS
TRANSACTION	AFFI	Exercice 4	Code transaction (4 car)

Étape 3 : Vérification avec CEMT

```
CEMT INQ FILE(FCLIENT)
```

Résultat attendu : **Fi**l(FCLIENT) **Dsn**(ROCHA.CICS.CLIENT) **Ena** **Ope** **Rea** **Upd** **Add** **Bro** **Del**
Vsam **Ksds**

```
CEDA VIEW MAPSET(CLIAFF) GROUP(CLIGROUP)
```

Résultat attendu : Affichage de la définition du mapset (DEFINITION SIGNATURE, RESIDENT, etc.)

```
CEMT INQ PROG(PRGCLIA)
```

Résultat attendu : **Pro**(PRGCLIA) **Len**(...) **Cob** **Ena** **Pri**

```
CEMT INQ TRAN(AFFI)
```

Résultat attendu : Tra(AFFI) Pro(PRGCLIA) Ena

Captures d'écran

Définition de la transaction

La commande CEDA DEFINE TRANSACTION crée le lien entre le code AFFI et le programme PRGCLIA :

```

DEFINE TRANSACTION(AFFI) GROUP(CLIGROUP) PROGRAM(PRGCLIA)
OVERTYPE TO MODIFY                                     CICS RELEASE = 0680
CEDA DEFine TRANsAction( AFFI )
  TRANsAction      : AFFI
  Group            : CLIGROUP
  DEScription      ==> _
  PROGram          ==> PRGCLIA
  TWAsize          ==> 00000          0-32767
  PROFile          ==> DFHCICST
  PArtitionset     ==>
  STatus           ==> Enabled        Enabled ! Disabled
  PRIMedsize       : 00000          0-65520
  TASKDATALoc      ==> Below         Below ! Any
  TASKDATAKey      ==> User          User ! Cics
  STOrageclear     ==> No            No ! Yes
  RUNaway          ==> System        System ! 0 ! 500-2700000
  SHUTDOWN         ==> Disabled      Disabled ! Enabled
  ISolate          ==> Yes           Yes ! No
  Brexit           ==>
+ REMOTE ATTRIBUTES

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                         TIME: 17.09.12 DATE: 01/12/26

```

Définition de la transaction AFFI. Les paramètres importants : PROGram=PRGCLIA (programme à exécuter), PROFile=DFHCICST (profil par défaut), STatus=Enabled (transaction active).

Installation du groupe complet

L'installation du groupe CLIGROUP tente d'installer toutes les ressources définies :

```

INSTALL GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  All
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Engmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
+ LSRpool     ==>
MESSAGES: 1 SEVERE 1 WARNING

                                SYSID=S670 APPLID=CICSTS51
                                TIME: 17.14.36 DATE: 01/12/26

INSTALL UNSUCCESSFUL

```

L'installation échoue partiellement avec "1 SEVERE 1 WARNING". Ceci est normal car certaines ressources (comme FCLIENT) sont déjà installées depuis l'exercice 1.

```

INSTALL GROUP(CLIGROUP)
MESSAGES - USE ENTER TO RETURN
  S Install failed because an existing definition for file FCLIENT could
    not be deleted._
  W GROUP CLIGROUP has been partially installed.

```

Le message explique l'échec : "Install failed because an existing definition for file FCLIENT could not be deleted." C'est un comportement attendu - FCLIENT était déjà installé. Le groupe est "partially installed", les autres ressources sont actives.

Vérification de la transaction

Malgré l'erreur partielle, la transaction AFFI est bien installée et opérationnelle :

```

INQ TRANSACTION(AFFI)
STATUS: RESULTS - OVERTYPE TO MODIFY
Tra(AFFI) Pri( 001 ) Pro(PRGCLIA ) Tcl( DFHTCL00 ) Ena Sta
          Prf(DFHCICST) Uda Bel Iso          Bac Wai

```

La transaction AFFI est active : Pri(001) = priorité 1, Pro(PRGCLIA) = programme associé, Tcl(DFHTCL00) = classe de transaction, Ena Sta = Enabled Status. La transaction est prête à être utilisée.

Exercice 5 : Test avec debugger CEDF

Énoncé

Activer la transaction en mode debugger avec la commande CEDF et par suite sans debugger.

Mon travail

J'ai testé la transaction AFFI en mode debug avec CEDF pour vérifier le bon enchaînement des commandes CICS et observer les valeurs des variables EIB (voir Exercice 3 pour les explications sur le mode pseudo-conversationnel et les variables EIB).

Résolution

Étape 1 : Activation du debugger et lancement de la transaction

CEDF

L'écran se vide et le curseur se positionne en haut. Le mode EDF est activé mais aucun message ne s'affiche. Il faut maintenant lancer la transaction à déboguer :

AFFI

CEDF intercepte alors la transaction et affiche le premier point d'arrêt.

Note : Sur TK4-, CEDF n'affiche pas de message de confirmation. Le debugger est actif dès que la commande est saisie.

Étape 2 : Navigation dans CEDF

Touche	Action
ENTER	Passer à l'étape suivante
PF5	Afficher la WORKING-STORAGE
PF4	Afficher l'EIB (Exec Interface Block)
PF3	Terminer le debug et continuer l'exécution

Étape 3 : Points d'arrêt observés

Étape	Commande CICS	RESP attendu
1	SEND MAP	NORMAL
2	RETURN TRANSID	-
3	TASK TERMINATION	-
4	RECEIVE MAP	NORMAL
5	READ FILE	NORMAL ou NOTFND
6	SEND MAP	NORMAL

Étape	Commande CICS	RESP attendu
7	RETURN TRANSID	-

Étape 4 : Test sans debugger

Pour tester la transaction sans le debugger CEDF, il suffit de lancer directement la transaction depuis un écran CICS vierge (sans avoir activé CEDF au préalable) :

AFFI

La transaction s'exécute normalement sans interruption, affichant directement l'écran de saisie.

Désactiver CEDF : Pour sortir du mode debug, appuyer sur PF3 pendant un point d'arrêt, ou simplement lancer une nouvelle transaction sans avoir tapé CEDF.

Captures d'écran

Premier passage - Écran vide

Lors du lancement de la transaction AFFI, le programme affiche l'écran de saisie vide :

```

*** AFFICHAGE CLIENT ***
-----
NUMERO COMPTE :  _
CODE REGION   :
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:
SEXE          :
ACTIVITE PRO  :
SITUATION SOC :
ADRESSE       :
SOLDE         :
POSITION      :
-----
MESSAGE :  SAISIR LE NUMERO DE COMPTE ET APPUYER SUR ENTREE

ENTER=RECHERCHER  PF3=QUITTER  CLEAR=EFFACER

```

L'écran initial invite l'utilisateur à saisir un numéro de compte. Tous les champs sont vides et le curseur est positionné sur NUMERO COMPTE (attribut IC = Initial Cursor dans la MAP BMS).

Debug CEDF - SEND MAP

Le debugger CEDF intercepte la commande SEND MAP et affiche les détails :


```

TRANSACTION: AFFI PROGRAM: PRGCLIA TASK: 0000127 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS SEND MAP
  MAP ('MAPAFF ')
  FROM ('.....2.....')
  LENGTH (254)
  MAPSET ('CLIAFF ')
  TERMINAL
  ERASE

OFFSET:X'000896'          LINE:00129          EIBFN=X'1804'
RESPONSE: NORMAL          EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED           PF2 : SWITCH HEX/CHAR       PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS   PF5 : WORKING STORAGE       PF6 : USER DISPLAY
PF7 : SCROLL BACK         PF8 : SCROLL FORWARD        PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY    PF11: EIB DISPLAY           PF12: ABEND USER TASK

```

Point d'arrêt après SEND MAP : MAP='MAPAFF', MAPSET='CLIAFF', RESPONSE: NORMAL (EIBRESP=0).
La zone FROM contient les données de la MAP envoyées à l'écran (254 octets).

Terminaison du premier passage

Après le SEND MAP, le programme exécute RETURN TRANSID et se termine :

```

TRANSACTION: AFFI          TASK: 0000127 APPLID: CICSTS51 DISPLAY: 00
STATUS: TASK TERMINATION

CONTINUE EDF? (ENTER YES OR NO)          REPLY: YES
ENTER: CONTINUE
PF1 : UNDEFINED           PF2 : SWITCH HEX/CHAR       PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS   PF5 : WORKING STORAGE       PF6 : USER DISPLAY
PF7 : SCROLL BACK         PF8 : SCROLL FORWARD        PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY    PF11: EIB DISPLAY           PF12: UNDEFINED

```

Fin de la tâche (TASK TERMINATION). Le prompt "CONTINUE EDF? (ENTER YES OR NO)" permet de continuer le debug lors du prochain passage ou de désactiver CEDF.

Saisie d'un numéro de compte

L'utilisateur saisit le numéro 000001 pour rechercher un client :

```
*** AFFICHAGE CLIENT ***
-----
NUMERO COMPTE :  000001
CODE REGION   :
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:
SEXE          :
ACTIVITE PRO  :
SITUATION SOC :
ADRESSE       :
SOLDE         :
POSITION      :
-----
MESSAGE :  SAISIR LE NUMERO DE COMPTE ET APPUYER SUR ENTREE

ENTER=RECHERCHER  PF3=QUITTER  CLEAR=EFFACER
```

Le numéro de compte 000001 est saisi. Après ENTER, CICS relance le programme qui va recevoir cette saisie via RECEIVE MAP.

Debug CEDF - RECEIVE MAP

Le debugger montre la réception des données saisies par l'utilisateur :

```

TRANSACTION: AFFI PROGRAM: PRGCLIA TASK: 0000134 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS RECEIVE MAP
  MAP ('MAPAFF ')
  INTO ('.....000001.....')
  MAPSET ('CLIAFF ')
  TERMINAL
  NOHANDLE

OFFSET:X'0008FE'      LINE:00139      EIBFN=X'1802'
RESPONSE: NORMAL      EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED      PF2 : SWITCH HEX/CHAR      PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS  PF5 : WORKING STORAGE      PF6 : USER DISPLAY
PF7 : SCROLL BACK      PF8 : SCROLL FORWARD      PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY  PF11: EIB DISPLAY      PF12: ABEND USER TASK

```

Point d'arrêt après RECEIVE MAP : la zone INTO contient "000001" (le numéro saisi). RESPONSE: NORMAL confirme que la saisie a été correctement transmise au programme.

Debug CEDF - READ FILE

Le programme effectue ensuite une lecture VSAM avec la clé saisie :

```

TRANSACTION: AFFI PROGRAM: PRGCLIA TASK: 0000134 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS READ
  FILE ('FCLIENT ')
  INTO ('0000010120DUPONT      JEAN      19850315M10CPARIS      0000150000CR'...)
  LENGTH (80)
  RIDFLD ('000001')
  EQUAL
  NOHANDLE

OFFSET:X'000A7E'      LINE:00169      EIBFN=X'0602'
RESPONSE: NORMAL      EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED      PF2 : SWITCH HEX/CHAR      PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS  PF5 : WORKING STORAGE      PF6 : USER DISPLAY
PF7 : SCROLL BACK      PF8 : SCROLL FORWARD      PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY  PF11: EIB DISPLAY      PF12: ABEND USER TASK

```

Point d'arrêt après READ FILE : la zone INTO contient l'enregistrement complet du client DUPONT Jean.
On voit les données : 0000010120DUPONT JEAN 19850315M10CPARIS 0000150000CR.

Résultat - Client trouvé

Après le READ réussi, le programme affiche les données du client :

```
*** AFFICHAGE CLIENT ***
-----
NUMERO COMPTE : 000001
CODE REGION   : 01                      01 - PARIS
NATURE COMPTE : 20
NOM           : DUPONT
PRENOM        : JEAN
DATE NAISSANCE : 19850315
SEXE          : M                      MASCULIN
ACTIVITE PRO  : 10
SITUATION SOC : C                      CELIBATAIRE
ADRESSE       : PARIS
SOLDE         : 0000150000
POSITION      : CR                      CREDITEUR
-----
MESSAGE : CLIENT TROUVE - PF3=QUITTER OU NOUVELLE RECHERCHE

ENTER=RECHERCHER  PF3=QUITTER  CLEAR=EFFACER
```

Le client Jean DUPONT est affiché avec toutes ses informations : région 01 (Paris), nature compte 20, date de naissance 19850315, sexe M (MASCULIN), situation C (CELIBATAIRE), solde 0000150000, position CR (CREDITEUR). Le message confirme "CLIENT TROUVE".

Fin du debug

L'utilisateur peut arrêter le mode debug en répondant "no" au prompt :

```
TRANSACTION: AFFI                                TASK: 0000156 APPLID: CICSTS51 DISPLAY: 00
```

```
STATUS: TASK TERMINATION
```

```
CONTINUE EDF? (ENTER YES OR NO)
```

```
REPLY: no
```

```
ENTER: CONTINUE
```

```
PF1 : UNDEFINED
```

```
PF2 : SWITCH HEX/CHAR
```

```
PF3 : END EDF SESSION
```

```
PF4 : SUPPRESS DISPLAYS
```

```
PF5 : WORKING STORAGE
```

```
PF6 : USER DISPLAY
```

```
PF7 : SCROLL BACK
```

```
PF8 : SCROLL FORWARD
```

```
PF9 : STOP CONDITIONS
```

```
PF10: PREVIOUS DISPLAY
```

```
PF11: EIB DISPLAY
```

```
PF12: UNDEFINED
```

En tapant "no", CEDF se désactive et la transaction continue sans interruption. Pour les prochains tests, il suffit de lancer directement AFFI sans passer par CEDF.

Test d'erreur - Client inexistant

Le programme gère correctement les erreurs, par exemple un numéro de compte inexistant :

```
*** AFFICHAGE CLIENT ***
```

```
NUMERO COMPTE : 222222
```

```
CODE REGION :
```

```
NATURE COMPTE :
```

```
NOM :
```

```
PRENOM :
```

```
DATE NAISSANCE:
```

```
SEXE :
```

```
ACTIVITE PRO :
```

```
SITUATION SOC :
```

```
ADRESSE :
```

```
SOLDE :
```

```
POSITION :
```

```
MESSAGE : CLIENT INEXISTANT - VERIFIEZ LE NUMERO
```

```
ENTER=RECHERCHER PF3=QUITTER CLEAR=EFFACER
```

Le numéro 222222 n'existe pas dans le fichier. Le programme affiche le message "CLIENT INEXISTANT - VERIFIEZ LE NUMERO" et conserve le numéro saisi pour correction.

Partie 2a : Opérations d'Ajout (WRITE)

Cette section couvre les exercices 6 à 8 : création de la MAP d'ajout, programme d'ajout avec la commande WRITE, et définition de la transaction AJOU.

READ vs WRITE : Deux opérations opposées

Après avoir maîtrisé la lecture (READ) dans la Partie 1, cette section introduit l'écriture (WRITE). Ces deux commandes sont complémentaires :

Aspect	READ (Partie 1)	WRITE (Partie 2a)
Action	Lire un enregistrement existant	Créer un nouvel enregistrement
Prérequis	Le client DOIT exister	Le client ne doit PAS exister
Erreur typique	NOTFND (client inexistant)	DUPREC (doublon)
Données	Fichier → Programme	Programme → Fichier
Clé (RIDFLD)	Recherche	Insertion

Exercice 6 : MAP pour ajout de client

Énoncé

Créer ou adapter la MAP précédente pour une opération d'ajout de CLIENT dans le Data Set CLIENT.

Mon travail

J'ai adapté la MAP d'affichage (CLIAFF) pour créer une nouvelle MAP de saisie (CLIAJT). La structure BMS est similaire mais le comportement des champs change fondamentalement.

Pourquoi une MAP différente pour l'ajout ?

En affichage, l'utilisateur ne saisit que le numéro de compte (clé de recherche) et les autres champs sont en lecture seule. En ajout, **tous les champs** doivent être saisissables car l'utilisateur crée un nouveau client de toutes pièces.

Différences entre CLIAFF et CLIAJT :

Aspect	CLIAFF (Affichage)	CLIAJT (Ajout)
NUMCPT	UNPROT (saisie clé)	UNPROT (saisie)
Autres champs	ASKIP (affichage)	UNPROT (saisie)
Libellés (région, sexe...)	Affichés (LIBREG, LIBSEX...)	Non présents
Titre	"AFFICHAGE CLIENT"	"AJOUT CLIENT"

Aspect	CLIAFF (Affichage)	CLIAJT (Ajout)
Touches	ENTER=Rechercher	ENTER=Valider

Pourquoi pas de libellés dans la MAP d'ajout ?

Dans CLIAFF, des zones supplémentaires (LIBREG, LIBSEX, LIBSIT, LIBPOS) affichent les libellés correspondant aux codes (ex: "01" → "PARIS"). En ajout, l'utilisateur saisit directement les codes, donc ces zones seraient vides et inutiles. On les remplace par des indications statiques à côté des champs (ex: "(01=Paris,02=Mars...)").

Résolution

MAP BMS : CLIAJT.bms

Le code source est stocké dans **ROCHA.CICS.SOURCE (CLIAJT)**. La structure reprend les mêmes concepts BMS que CLIAFF (voir Partie 1, Exercice 2 pour les explications sur DFHMSD, DFHMDI, DFHMDF et les attributs).

Extrait du code BMS - Déclaration du MAPSET :

```
*****
* MAPSET : CLIAJT - Ajout Client
* Transaction : AJOU
*****
CLIAJT  DFHMSD TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,                X
        STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
*****
MAPAJT  DFHMDI SIZE=(24,80),LINE=1,COLUMN=1
```

Extrait - Champs de saisie (tous en UNPROT) :

```
*-----
* ZONES DE SAISIE - TOUS LES CHAMPS EN UNPROT
*-----
        DFHMDF POS=(4,2),LENGTH=16,ATTRB=ASKIP,                    X
        INITIAL='NUMERO COMPTE : '
NUMCPT  DFHMDF POS=(4,19),LENGTH=6,ATTRB=(UNPROT,NUM,IC)
        DFHMDF POS=(4,26),LENGTH=1,ATTRB=ASKIP
*
        DFHMDF POS=(5,2),LENGTH=16,ATTRB=ASKIP,                    X
        INITIAL='CODE REGION   : '
CODREG  DFHMDF POS=(5,19),LENGTH=2,ATTRB=(UNPROT,NUM)
        DFHMDF POS=(5,22),LENGTH=25,ATTRB=ASKIP,                    X
        INITIAL='(01=PAR,02=MAR,03=LYO,04=LIL) '
*
        DFHMDF POS=(7,2),LENGTH=16,ATTRB=ASKIP,                    X
        INITIAL='NOM           : '
```



```
NOM      DFHMDF POS=(7,19),LENGTH=10,ATTRB=UNPROT
          DFHMDF POS=(7,30),LENGTH=1,ATTRB=ASKIP
```

Différence clé avec CLIAFF : Tous les champs de données sont en **UNPROT** (saisissables) au lieu de **ASKIP** (affichage seul). Des indications statiques comme (**01=PAR,02=MAR,03=LY0,04=LIL**) remplacent les zones de libellés dynamiques.

Zones de saisie :

Zone	Longueur	Attribut	Description
NUMCPT	6	UNPROT,NUM,IC	Numéro de compte (curseur initial)
CODREG	2	UNPROT,NUM	Code région (01/02/03/04)
NATCPT	2	UNPROT,NUM	Nature du compte
NOM	10	UNPROT	Nom du client
PRENOM	10	UNPROT	Prénom du client
DATNA	8	UNPROT,NUM	Date de naissance (AAAAMMJJ)
SEXE	1	UNPROT	Sexe (M/F)
ACTPRO	2	UNPROT,NUM	Code activité professionnelle
SITSO	1	UNPROT	Situation sociale (C/M/D/V)
ADRESSE	10	UNPROT	Adresse
SOLDE	10	UNPROT,NUM	Solde du compte
POSIT	2	UNPROT	Position (DB/CR)
MSG	60	ASKIP,BRT	Zone message (affichage seul)

Note : L'attribut **IC** (Initial Cursor) sur NUMCPT positionne automatiquement le curseur sur ce champ au premier affichage. L'attribut **NUM** (Numeric) force la saisie en mode numérique sur certains terminaux.

JCL d'assemblage : ASMAJT.jcl

Le JCL d'assemblage suit la même structure que ASMCLAF.jcl (voir Partie 1, Exercice 2). Seuls le nom du job (ROCHA05) et le membre source (CLIAJT) changent.

Définition CICS

La définition et l'installation du mapset suivent le même processus que pour CLIAFF (voir Partie 1, Exercice 4 pour les explications sur CEDA) :

```
CEDA DEFINE MAPSET(CLIAJT) GROUP(CLIGROUP)
CEDA INSTALL MAPSET(CLIAJT) GROUP(CLIGROUP)
```

Vérification

```
CEDA VIEW MAPSET(CLIAJT) GROUP(CLIGROUP)
```

Captures d'écran

Résultat de l'assemblage BMS

Après soumission du JCL d'assemblage ASMAJT, le job ROCHA05 s'exécute avec succès.

```
No Statements Flagged in this Assembly
HIGH LEVEL ASSEMBLER, 5696-234, RELEASE 6.0, PTF UK92594
SYSTEM: z/OS 01.13.00          JOBNAME: ROCHA05      STEPNAME: ASSEM      PRO
Data Sets Allocated for this Assembly
Con DDname      Data Set Name                      Volume  Member
P1 SYSIN        SYS26013.T124519.RA000.ROCHA05.TEMPM.H01
L1 SYSLIB       DFH510.CICS.SDFHMAC                  FDC511
L2              SYS1.MACLIB                          FDRES1
               SYSPRINT IBMUSER.ROCHA05.JOB00098.D0000104.?
               SYSPUNCH ROCHA.CICS.LINK                FDDBAS  CLIAJT

31964K allocated to Buffer Pool      Storage required  1072K
 100 Primary Input Records Read      6922 Library Records Read
   0 ASMAOPT Records Read            305 Primary Print Records Written
  95 Object Records Written          0 ADATA Records Written
Assembly Start Time: 12.45.20 Stop Time: 12.45.20 Processor Time: 00.00.00.2857
Return Code 000
***** BOTTOM OF DATA *****
```

Le job d'assemblage retourne RC=0000, confirmant que la MAP CLIAJT a été correctement compilée. Le copybook est généré dans ROCHA.CICS.LINK(CLIAJT) via SYSPUNCH.

Définition du mapset CLIAJT dans CICS

Après l'assemblage BMS réussi, on définit le mapset dans CICS avec CEDA.

```

DEFINE MAPSET(CLIAJT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA  DEfIne MApset( CLIAJT  )
  MApset      : CLIAJT
  Group       : CLIGROUP
  DEscription ==> _
  REsident    ==> No                No ! Yes
  USAge       ==> Normal            Normal ! Transient
  USElpacopy  ==> No                No ! Yes
  Status      ==> Enabled            Enabled ! Disabled
  RSl         : 00                  0-24 ! Public
DEFINITION SIGNATURE
DEfInetime   : 01/19/26 17:28:08
CHANGETime   : 01/19/26 17:28:08
CHANGEUsrid  : CICSUSER
CHANGEAGEnt  : CSDApi              CSDApi ! CSDBatch
CHANGEAGRel  : 0680

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                          TIME: 17.28.08 DATE: 01/19/26

```

La commande `CEDA DEFINE MAPSET(CLIAJT) GROUP(CLIGROUP)` crée la définition du mapset d'ajout. Le message "DEFINE SUCCESSFUL" confirme la création. On note le statut *Enabled* par défaut.

Installation du mapset CLIAJT

```

INSTALL MAPSET(CLIAJT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA  Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Engmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+  MApset     ==> CLIAJT

                                           SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL                          TIME: 17.30.12 DATE: 01/19/26
PF 1 HELP          3 END          6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA INSTALL MAPSET(CLIAJT)` charge le mapset en mémoire CICS. Le message "INSTALL SUCCESSFUL" indique que le mapset est prêt à être utilisé.

Vérification de la définition

```

VIEW      MAPSET(CLIAJT) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View Mapset( CLIAJT )
  Mapset      : CLIAJT
  Group       : CLIGROUP
  Description  :
  REsident    : No                      No ! Yes
  USAge       : Normal                  Normal ! Transient
  USElpacopy   : No                     No ! Yes
  Status      : Enabled                 Enabled ! Disabled
  RSl         : 00                      0-24 ! Public
DEFINITION SIGNATURE
  DEFinetime   : 01/19/26 17:28:08
  CHANGETime   : 01/19/26 17:28:08
  CHANGEUsrid  : CICSUSER
  CHANGEAGEnt  : CSDApi                 CSDApi ! CSDBatch
  CHANGEAGRel  : 0680

```

CEDA VIEW permet de consulter tous les paramètres du mapset : nom, groupe, résidence (Normal), et statut (Enabled).

Exercice 7 : Programme d'ajout (WRITE)

Énoncé

Créer le PROGRAMME pour une opération d'ajout d'un nouveau CLIENT dans le Data Set CLIENT. Un contrôle de conformité de donnée et de doublure doit être effectué.

Mon travail

J'ai développé le programme PRGAJT qui gère l'ajout de nouveaux clients. Ce programme est plus complexe que PRGCLIA car il doit valider les données avant l'écriture et gérer plusieurs types d'erreurs.

Pourquoi un mode pseudo-conversationnel à 2 phases ?

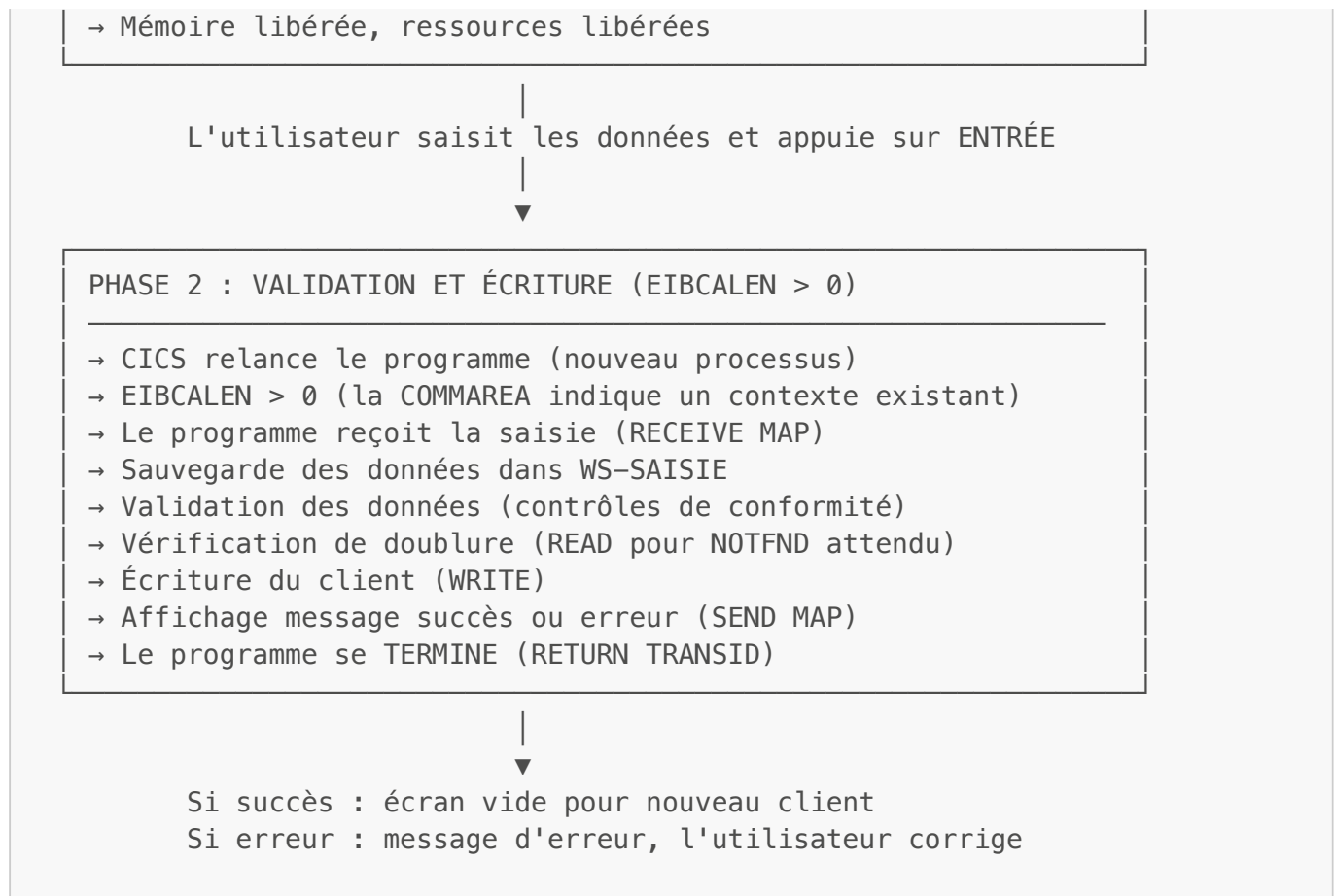
Contrairement à la mise à jour (3 phases), l'ajout ne nécessite que 2 phases car il n'y a pas de recherche préalable :

LANCEMENT TRANSACTION "AJOU"



PHASE 1 : SAISIE (EIBCALEN = 0)

- CICS lance le programme pour la première fois
- EIBCALEN = 0 (pas de COMMAREA, c'est un nouveau contexte)
- Le programme affiche l'écran vide (SEND MAP avec ERASE)
- Le programme se TERMINE (RETURN TRANSID)



Voir Partie 1, Exercice 3 pour les explications détaillées sur le mode pseudo-conversationnel et les variables EIB.

Pourquoi sauvegarder les données dans WS-SAISIE ?

C'est un point technique crucial. Avec **MODE=INOUT** et **STORAGE=AUTO** dans la définition BMS, les zones input (suffixe I) et output (suffixe O) **partagent la même zone mémoire** (voir Partie 1, Exercice 2 pour les explications sur la structure DSECT).

Problème : Après le **RECEIVE MAP**, les données saisies sont dans **NUMCPTI**, **CODREGL**, etc. Mais dès qu'on fait **MOVE LOW-VALUES TO MAPAJTO** pour préparer l'affichage, ces données sont écrasées !

Solution : Sauvegarder immédiatement les données dans des variables Working-Storage (préfixe WS-) :

```

* SAUVEGARDE DES DONNEES AVANT ECRASEMENT PAR LOW-VALUES
MOVE NUMCPTI    TO WS-NUMCPT
MOVE NUMCPTL    TO WS-NUMCPTL
MOVE CODREGI    TO WS-CODREG
MOVE CODREGL    TO WS-CODREGL
...
  
```

Pourquoi vérifier la doublure avant l'écriture ?

La commande **WRITE** avec une clé existante retourne l'erreur **DUPREC** (Duplicate Record). Cependant, il est préférable de vérifier **avant** l'écriture avec un **READ** :

1. **Meilleur message** : On peut afficher "CE CLIENT EXISTE DÉJÀ" au lieu de l'erreur technique DUPREC
2. **Cohérence** : On valide toutes les données avant toute tentative d'écriture
3. **Performance** : Le READ est moins coûteux qu'un WRITE qui échoue

Logique inversée : Dans ce READ, on **espère** un **NOTFND** ! Si le READ retourne **NORMAL**, c'est que le client existe déjà → erreur de doublure.

Résolution

Programme : PRGAJT.cbl

Le code source est stocké dans **ROCHA.CICS.SOURCE(PRGAJT)**. Voici les sections clés du programme.

Structure de la COMMAREA (WORKING-STORAGE) :

```
*-----
* ZONE DE COMMUNICATION (COMMAREA)
*-----
01 WS-COMMAREA.
   05 WS-FLAG-INIT          PIC X(01) VALUE 'N'.
   88 PREMIER-PASSAGE      VALUE 'N'.
   88 PASSAGE-SUIVANT      VALUE '0'.
```

La COMMAREA de l'ajout est simple : un seul indicateur pour distinguer le premier passage des suivants.

Zone de sauvegarde des données saisies :

```
*-----
* SAUVEGARDE DES DONNEES SAISIES (EVITE ECRASEMENT PAR LOW-VALUES)
*-----
01 WS-SAISIE.
   05 WS-NUMCPT              PIC X(06).
   05 WS-NUMCPTL             PIC S9(04) COMP.
   05 WS-CODREG              PIC X(02).
   05 WS-CODREGL             PIC S9(04) COMP.
   05 WS-NATCPT              PIC X(02).
   05 WS-NOM                 PIC X(10).
   05 WS-NOML                PIC S9(04) COMP.
   05 WS-PRENOM              PIC X(10).
   05 WS-DATNAISS            PIC X(08).
   05 WS-SEXE                PIC X(01).
   05 WS-SEXEL               PIC S9(04) COMP.
   05 WS-ACTPRO              PIC X(02).
   05 WS-SITSO               PIC X(01).
   05 WS-SITSOL              PIC S9(04) COMP.
   05 WS-ADRESSE             PIC X(10).
   05 WS-SOLDE               PIC X(10).
```

05 WS-POSITION	PIC X(02).
05 WS-POSITL	PIC S9(04) COMP.

Note : On sauvegarde aussi les longueurs (suffixe L) car elles indiquent si l'utilisateur a saisi quelque chose dans le champ. Une longueur = 0 signifie que le champ est vide.

Point d'entrée avec gestion des touches :

```

0000-PRINCIPAL.
  EVALUATE TRUE
    WHEN EIBCALEN = 0
      PERFORM 1000-PREMIER-PASSAGE
    WHEN EIBAID = DFHPF3
      PERFORM 9000-FIN-PROGRAMME
    WHEN EIBAID = DFHCLEAR
      PERFORM 1000-PREMIER-PASSAGE
    WHEN OTHER
      PERFORM 2000-TRAITEMENT THRU 2000-FIN
  END-EVALUATE

  EXEC CICS RETURN
    TRANSID('AJOU')
    COMMAREA(WS-COMMAREA)
    LENGTH(LENGTH OF WS-COMMAREA)
  END-EXEC.

```

Paragraphe de traitement avec PERFORM THRU :

```

2000-TRAITEMENT.
  MOVE 'N' TO WS-ERREUR

  EXEC CICS RECEIVE MAP('MAPAJT')
    MAPSET('CLIAJT')
    RESP(WS-RESP)
  END-EXEC

  * Gestion MAPFAIL (aucune donnée transmise)
  IF WS-RESP = DFHRESP(MAPFAIL)
    MOVE LOW-VALUES TO MAPAJTO
    MOVE 'AUCUNE DONNEE SAISIE - VEUILLEZ REMPLIR' TO MSGO
    EXEC CICS SEND MAP('MAPAJT')
      MAPSET('CLIAJT')
      ERASE
    END-EXEC
    GO TO 2000-FIN
  END-IF

  * SAUVEGARDE DES DONNEES AVANT ECRASEMENT PAR LOW-VALUES
  MOVE NUMCPTI TO WS-NUMCPT

```

```

        MOVE NUMCPTL    TO WS-NUMCPTL
        MOVE CODREGI    TO WS-CODREG
        MOVE CODREGL    TO WS-CODREGL
        . . .

* Validation des données
    PERFORM 2100-VALIDER-DONNEES THRU 2100-FIN

    IF ERREUR-DETECTEE
        EXEC CICS SEND MAP('MAPAJT')
            MAPSET('CLIAJT')
            ERASE
        END-EXEC
        GO TO 2000-FIN
    END-IF

* Vérification doublure (client existe déjà ?)
    PERFORM 2200-VERIFIER-DOUBLURE THRU 2200-FIN
    IF ERREUR-DETECTEE
        EXEC CICS SEND MAP('MAPAJT')
            MAPSET('CLIAJT')
            ERASE
        END-EXEC
        GO TO 2000-FIN
    END-IF

* Préparation et écriture de l'enregistrement
    PERFORM 2300-PREPARER-ENREGISTREMENT
    PERFORM 2400-ECRIRE-CLIENT

    EXEC CICS SEND MAP('MAPAJT')
        MAPSET('CLIAJT')
    END-EXEC.

2000-FIN.
EXIT.

```

Paragraphe de validation des données :

```

    2100-VALIDER-DONNEES.
*-----
* Contrôles de conformité des données saisies
* Utilise les variables WS- sauvegardées (pas MAPAJTI)
*-----
        MOVE LOW-VALUES TO MAPAJTO

* Contrôle numéro de compte (obligatoire et numérique)
    IF WS-NUMCPTL = 0 OR WS-NUMCPT = SPACES
        MOVE 'NUMERO DE COMPTE OBLIGATOIRE' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF

```



```

    IF WS-NUMCPT NOT NUMERIC
        MOVE 'NUMERO DE COMPTE DOIT ETRE NUMERIQUE' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF

* Contrôle code région (01, 02, 03 ou 04)
    IF WS-CODREGL = 0 OR WS-CODREG = SPACES
        MOVE 'CODE REGION OBLIGATOIRE' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF

    IF WS-CODREG NOT = '01' AND WS-CODREG NOT = '02'
        AND WS-CODREG NOT = '03' AND WS-CODREG NOT = '04'
        MOVE 'CODE REGION INVALIDE (01/02/03/04)' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF

* Contrôle nom (obligatoire)
    IF WS-NOML = 0 OR WS-NOM = SPACES
        MOVE 'NOM OBLIGATOIRE' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF

* Contrôle sexe (M ou F)
    IF WS-SEXEL = 0 OR WS-SEXE = SPACES
        MOVE 'SEXE OBLIGATOIRE' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF

    IF WS-SEXE NOT = 'M' AND WS-SEXE NOT = 'F'
        MOVE 'SEXE INVALIDE (M OU F)' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 2100-FIN
    END-IF
    ...

2100-FIN.
EXIT.

```

Paragraphe de vérification de doublure :

```

2200-VERIFIER-DOUBLURE.
*-----
* Vérification que le client n'existe pas déjà
* Note: NOTFND est attendu (client nouveau), NORMAL = doublure
*-----

```

```

MOVE WS-NUMCPT TO CLI-NUMCPT

EXEC CICS READ
  FILE('FCLIENT')
  INTO(ENR-CLIENT)
  RIDFLD(CLI-NUMCPT)
  RESP(WS-RESP)
END-EXEC

IF WS-RESP = DFHRESP(NORMAL)
  MOVE 'ENREGISTREMENT EN DOUBLE - CE CLIENT EXISTE DEJA'
  TO MSGO
  MOVE '0' TO WS-ERREUR
END-IF.

2200-FIN.
EXIT.

```

Paragraphe d'écriture du client :

```

2400-ECRIRE-CLIENT.
*-----
* Écriture du nouvel enregistrement dans le fichier VSAM
*-----
EXEC CICS WRITE
  FILE('FCLIENT')
  FROM(ENR-CLIENT)
  RIDFLD(CLI-NUMCPT)
  RESP(WS-RESP)
END-EXEC

EVALUATE WS-RESP
  WHEN DFHRESP(NORMAL)
    MOVE LOW-VALUES TO MAPAJTO
    MOVE 'CLIENT AJOUTE AVEC SUCCES - NOUVEAU OU PF3'
    TO MSGO
  WHEN DFHRESP(DUPREC)
    MOVE 'ENREGISTREMENT EN DOUBLE' TO MSGO
    MOVE '0' TO WS-ERREUR
  WHEN OTHER
    MOVE 'ERREUR ECRITURE FICHIER - CONTACTEZ SUPPORT'
    TO MSGO
    MOVE '0' TO WS-ERREUR
END-EVALUATE.

```

JCL de compilation : CMPAJT.jcl

Le JCL de compilation suit la même structure que CMPCLAF.jcl (voir Partie 1, Exercice 3). Seuls le nom du job (ROCHA06) et le membre source (PRGAJT) changent.

Structure du programme

Paragraphe	Fonction
0000-PRINCIPAL	Point d'entrée, aiguillage selon EIBCALEN et EIBAUD
1000-PREMIER-PASSAGE	Affichage de l'écran vide pour saisie
2000-TRAITEMENT	Réception saisie, validations, écriture
2100-VALIDER-DONNEES	Contrôles de conformité des champs
2200-VERIFIER-DOUBLURE	READ pour vérifier que le client n'existe pas
2300-PREPARER-ENREGISTREMENT	Transfert WS-SAISIE vers ENR-CLIENT
2400-ECRIRE-CLIENT	WRITE VSAM avec gestion erreurs
9000-FIN-PROGRAMME	Message de fin et RETURN sans TRANSID

Commandes CICS utilisées

Commande	Usage
SEND MAP	Envoyer l'écran (avec ERASE pour effacer)
RECEIVE MAP	Recevoir la saisie avec RESP pour MAPFAIL
READ FILE	Vérifier si client existe (doublure) - NOTFND attendu
WRITE FILE	Écrire le nouveau client
RETURN TRANSID	Retour pseudo-conversationnel avec COMMAREA
SEND TEXT	Message de fin (sans MAP)

Messages d'erreur gérés

Message	Contexte
SAISIR LES DONNEES DU NOUVEAU CLIENT	Premier passage
AUCUNE DONNEE SAISIE	MAPFAIL - utilisateur a appuyé ENTER sans rien saisir
NUMERO DE COMPTE OBLIGATOIRE	Champ NUMCPT vide
NUMERO DE COMPTE DOIT ETRE NUMERIQUE	Caractères non numériques
CODE REGION OBLIGATOIRE	Champ CODREG vide
CODE REGION INVALIDE (01/02/03/04)	Code différent des valeurs autorisées
NOM OBLIGATOIRE	Champ NOM vide
SEXE OBLIGATOIRE	Champ SEXE vide

Message	Contexte
SEXE INVALIDE (M OU F)	Sexe différent de M ou F
SITUATION SOCIALE OBLIGATOIRE	Champ SITSO vide
SITUATION INVALIDE (C/M/D/V)	Situation non reconnue
POSITION OBLIGATOIRE	Champ POSIT vide
POSITION INVALIDE (DB OU CR)	Position non reconnue
ENREGISTREMENT EN DOUBLE	Client existe déjà (READ NORMAL ou WRITE DUPREC)
CLIENT AJOUTE AVEC SUCCES	WRITE VSAM réussi

Difficultés rencontrées et solutions

Problème 1 : Écrasement des données saisies par LOW-VALUES

Symptôme : Après le **RECEIVE MAP**, les données saisies étaient perdues lors du **MOVE LOW-VALUES TO MAPAJTO** dans le paragraphe de validation.

Cause : Avec **MODE=INOUT** et **STORAGE=AUTO** dans la définition BMS, les zones input (suffixe I) et output (suffixe O) partagent la même zone mémoire (voir Partie 1, Exercice 2 pour les explications sur la structure DSECT).

Solution : Sauvegarder les données saisies dans des variables Working-Storage (préfixe WS-) **immédiatement après** le **RECEIVE MAP**, avant tout **MOVE LOW-VALUES** :

```
* Juste après RECEIVE MAP, avant toute autre opération
MOVE NUMCPTI    TO WS-NUMCPT
MOVE NUMCPTL    TO WS-NUMCPTL
MOVE SEXEI      TO WS-SEXE
MOVE SEXEL      TO WS-SEXEL
...
```

Problème 2 : Validations ignorées - le client était ajouté malgré les erreurs

Symptôme : Même avec des données invalides (sexe = 'X'), le client était ajouté dans le fichier.

Cause : Le **GO TO paragraphe-FIN** dans les validations sortait de la plage du **PERFORM**, ce qui faisait continuer le programme **séquentiellement** vers les paragraphes suivants (2200, 2300, 2400) au lieu de retourner à l'appelant.

Illustration du problème :

```
* Code problématique
PERFORM 2100-VALIDER-DONNEES    ← Sans THRU, la plage s'arrête à 2100-
```

```

VALIDER-DONNEES

2100-VALIDER-DONNEES.
    ...
    GO TO 2100-FIN          ← Sort de la plage du PERFORM !
    ...
2100-FIN.                  ← Hors de la plage, le programme continue
séquentiellement
    EXIT.

2200-VERIFIER-DOUBLURE.    ← Exécuté même si erreur de validation !

```

Solution : Utiliser la clause **THRU** pour inclure le paragraphe FIN dans la plage du PERFORM :

```

* Code corrigé
PERFORM 2100-VALIDER-DONNEES THRU 2100-FIN    ← La plage inclut 2100-FIN

2100-VALIDER-DONNEES.
    ...
    GO TO 2100-FIN          ← Reste dans la plage du PERFORM
    ...
2100-FIN.                  ← Dans la plage, EXIT retourne à l'appelant
    EXIT.

```

Avec **THRU**, le **GO TO 2100-FIN** reste dans la plage du PERFORM, et après le **EXIT** de 2100-FIN, le contrôle retourne correctement à l'appelant (2000-TRAITEMENT).

Problème 3 : Message d'erreur non visible sans CEDF

Symptôme : Le message d'erreur de validation s'affichait dans CEDF mais pas sur l'écran normal.

Cause : Le **SEND MAP** sans **ERASE** ne rafraîchissait pas l'écran complet, causant des artefacts visuels.

Solution : Ajouter **ERASE** au **SEND MAP** d'erreur pour rafraîchir l'écran :

```

IF ERREUR-DETECTEE
    EXEC CICS SEND MAP('MAPAJT')
        MAPSET('CLIAJT')
        ERASE          ← Efface l'écran avant réaffichage
    END-EXEC
    GO TO 2000-FIN
END-IF

```

Amélioration future : Conservation des données lors des erreurs

Problème identifié : Actuellement, lorsqu'une erreur de validation se produit, l'écran est effacé (**ERASE**) et seul le message d'erreur s'affiche. L'utilisateur doit ressaisir toutes les données, ce qui n'est pas ergonomique.

Cause technique : L'option **ERASE** dans la commande **SEND MAP** efface l'écran entier avant d'afficher la MAP. Comme le programme fait **MOVE LOW-VALUES TO MAPAJTO** pour préparer l'affichage, les données ne sont pas renvoyées à l'écran.

Solution envisagée : Lors d'une erreur de validation, il faudrait :

1. **Ne pas faire** **MOVE LOW-VALUES TO MAPAJTO** pour conserver les données
2. **Ou** recopier les données sauvegardées (WS-SAISIE) vers les zones output (MAPAJTO) avant le **SEND MAP**
3. Utiliser **SEND MAP ... DATAONLY** au lieu de **ERASE** pour ne rafraîchir que les données sans effacer l'écran

```
* Amélioration : recopier les données avant affichage erreur
IF ERREUR-DETECTEE
    MOVE WS-NUMCPT  TO NUMCPTO
    MOVE WS-CODREG  TO CODREGO
    MOVE WS-NOM      TO NOMO
    ...
    EXEC CICS SEND MAP('MAPAJT')
        MAPSET('CLIAJT')
        DATAONLY          ← Ne rafraîchit que les données
    END-EXEC
END-IF
```

Note : Cette amélioration n'a pas été implémentée dans la version actuelle du projet. Elle constitue une évolution possible pour améliorer l'expérience utilisateur.

Captures d'écran

Compilation du programme PRGAJT

Après soumission du JCL CMPAJT, le job de compilation s'exécute avec succès.

```
* Statistics for COBOL program PRGAJT:
*   Source records = 739
*   Data Division statements = 249
*   Procedure Division statements = 151
End of compilation 1, program PRGAJT, no statements flagged.
Return code 0
***** BOTTOM OF DATA *****
```

Le job de compilation COBOL retourne RC=0, confirmant que le programme PRGAJT a été correctement compilé. On note 739 enregistrements sources traités.

Définition du programme dans CICS

Après la compilation réussie, on définit le programme dans CICS avec CEDA.

```

DEFINE PROGRAM(PRGAJT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFINE PROGram( PRGAJT  )
  PROGram      : PRGAJT
  Group        : CLIGROUP
  DEScription  ==> -
  Language     ==> -                CObol ! Assembler ! Le370 ! C ! PlI
  REload       ==> No                No ! Yes
  RESident     ==> No                No ! Yes
  USAge        ==> Normal            Normal ! Transient
  USElpacopy   ==> No                No ! Yes
  Status       ==> Enabled           Enabled ! Disabled
  RSl          : 00                 0-24 ! Public
  CEdf         ==> Yes               Yes ! No
  DAtalocation ==> Below             Below ! Any
  EXECKey      ==> User              User ! Cics
  COncurrency  ==> Quasirent         Quasirent ! Threadsafe ! Required
  Api          ==> Cicsapi           Cicsapi ! Openapi
REMOTE ATTRIBUTES
+ Dynamic      ==> No                No ! Yes

```

La commande `CEDA DEFINE PROGRAM(PRGAJT) GROUP(CLIGROUP)` crée la définition du programme. On voit les options disponibles : Language (Cobol par défaut), Status (Enabled), CEdf (Yes pour le débogage).

Installation du programme PRGAJT

```

INSTALL PROGRAM(PRGAJT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==> -
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  DOctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+ MApset      ==>

                                SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL              TIME: 17.32.00 DATE: 01/19/26

```

La commande `CEDA INSTALL PROGRAM(PRGAJT)` charge le programme compilé en mémoire CICS. Le message "INSTALL SUCCESSFUL" confirme l'activation.

Vérification avec CEDA VIEW

La commande `CEDA VIEW` permet de visualiser les caractéristiques du programme enregistré dans CICS.

```

OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View PROGram( PRGAJT  )
  PROGram      : PRGAJT
  Group        : CLIGROUP
  DEScription   :
  Language     : CObol          CObol ! Assembler ! Le370 ! C ! PLi
  REload       : No             No ! Yes
  RESident     : No             No ! Yes
  USAge        : Normal         Normal ! Transient
  USElpacopy   : No             No ! Yes
  Status       : Enabled        Enabled ! Disabled
  RSl          : 00             0-24 ! Public
  CEdf         : Yes            Yes ! No
  DAtalocation : Below          Below ! Any
  EXECKey      : User           User ! Cics
  COncurrency  : Quasirent      Quasirent ! Threadsafe ! Required
  Api          : Cicsapi        Cicsapi ! Openapi
REMOTE ATTRIBUTES
+ DYNamic      : No             No ! Yes

                                SYSID=S670 APPLID=CICSTS51

```

Vue de la définition du programme PRGAJT dans le groupe CLIGROUP. On note le langage COBOL et le statut Enabled.

Vérification avec CEMT

La commande CEMT INQ permet de vérifier que le programme est bien actif dans CICS.

```

INQUIRE PROGRAM(PRGAJT)
STATUS: RESULTS - OVERTYPE TO MODIFY
  Prog(PRGAJT  ) Leng(0000000000) Cob Pro Ena Pri      Ced
      Res(0000) Use(0000000000) Bel Uex Ful Qua Cic

```

Le programme PRGAJT est correctement installé : "Cob Pro Ena" indique un programme COBOL (Cob), compilé et prêt (Pro), et activé (Ena).

Exercice 8 : Transaction d'ajout

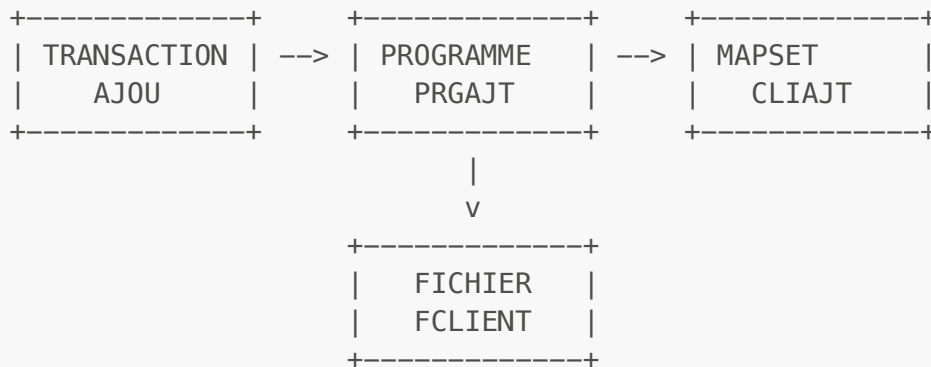
Énoncé

Suivre cette opération par l'ajout d'une nouvelle Transaction dans le GROUP et activer la transaction en mode debugger CEDF et sans debugger.

Mon travail

La transaction AJOU est le point d'entrée utilisateur pour l'ajout de clients. Comme pour AFFI, elle fait le lien entre le code saisi par l'utilisateur et le programme COBOL-CICS à exécuter.

Architecture CICS - Liaison des ressources



Pour que la transaction fonctionne, quatre ressources doivent être définies et installées dans CICS :

1. **FILE FCLIENT** : Le fichier VSAM contenant les données (défini dans l'exercice 1)
2. **MAPSET CLIAJT** : L'écran BMS compilé (défini et installé dans l'exercice 6)
3. **PROGRAM PRGAJT** : Le programme COBOL-CICS compilé (défini et installé dans l'exercice 7)
4. **TRANSACTION AJOU** : Le code de 4 caractères qui lance le programme

À ce stade, seule la **transaction** reste à définir. Les autres ressources ont été créées dans les exercices précédents.

Résolution

Définition et installation de la transaction :

Comme pour la transaction AFFI (voir Partie 1, Exercice 4 pour les explications détaillées sur CEDA), on définit puis on installe la transaction :

```

CEDA DEFINE TRANSACTION(AJOU) GROUP(CLIGROUP) PROGRAM(PRGAJT)
CEDA INSTALL TRANSACTION(AJOU) GROUP(CLIGROUP)
  
```

Vérification :

```
CEMT INQ TRAN(AJOU)
```

Résultat attendu : **Tra(AJOU) Pro(PRGAJT) Ena** confirmant que la transaction est active et liée au bon programme.

Tableau récapitulatif du groupe CLIGROUP après exercice 8

Type	Nom	Description	Défini dans
FILE	FCLIENT	Fichier VSAM CLIENT	Exercice 1
MAPSET	CLIAFF	Écran d'affichage	Exercice 4
PROGRAM	PRGCLIA	Programme d'affichage	Exercice 4

Type	Nom	Description	Défini dans
TRANSACTION	AFFI	Transaction d'affichage	Exercice 4
MAPSET	CLIAJT	Écran d'ajout	Exercice 8
PROGRAM	PRGAJT	Programme d'ajout	Exercice 8
TRANSACTION	AJOU	Transaction d'ajout	Exercice 8

Test de la transaction

Test avec CEDF (voir Partie 1, Exercice 5 pour la navigation CEDF) :

CEDF
AJOU

Points d'arrêt observés pour un ajout complet :

Étape	Commande CICS	RESP attendu	Description
1	SEND MAP	NORMAL	Affichage écran vide
2	RETURN TRANSID	-	Fin premier passage
3	RECEIVE MAP	NORMAL	Réception saisie
4	READ FILE	NOTFND	Vérification doublure
5	WRITE FILE	NORMAL	Écriture client
6	SEND MAP	NORMAL	Message succès
7	RETURN TRANSID	-	Fin traitement

Note importante sur NOTFND : Lors du READ FILE (étape 4), CEDF affiche souvent une réponse **NOTFND**. C'est le comportement **attendu et normal** ! Ce READ sert à vérifier que le client n'existe pas déjà (contrôle de doublure). Si NOTFND est retourné, cela signifie que le numéro de compte est disponible et qu'on peut procéder à l'écriture.

Test sans debugger :

AJOU

Scénarios de test :

Scénario	Action	Résultat attendu
Ajout normal	Saisir toutes les données valides	"CLIENT AJOUTE AVEC SUCCES"
Doublon	Saisir un numéro existant	"ENREGISTREMENT EN DOUBLE"

Scénario	Action	Résultat attendu
Sexe invalide	Saisir SEXE = 'X'	"SEXE INVALIDE (M OU F)"
Champ vide	Laisser NOM vide	"NOM OBLIGATOIRE"
Aucune saisie	Appuyer ENTER sans rien saisir	"AUCUNE DONNEE SAISIE"
Quitter	Appuyer PF3	Fin de la transaction

Vérification de l'ajout :

Après un ajout réussi, utiliser la transaction AFFI pour vérifier que le client a bien été créé :

AFFI

Saisir le numéro du client ajouté → Les données doivent s'afficher.

Captures d'écran

Définition de la transaction AJOU

La commande CEDA DEFINE crée la liaison entre le code transaction et le programme.

```

DEFINE TRANSACTION(AJOU) GROUP(CLIGROUP) PROGRAM(PRGAJT)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFine TRAnsAction( AJOU )
  TRAnsAction      : AJOU
  Group            : CLIGROUP
  DEscription      ==> _
  PROgram          ==> PRGAJT
  TWAsize          ==> 00000          0-32767
  PROFile          ==> DFHCICST
  PARtitionset     ==>
  SStatus          ==> Enabled        Enabled ! Disabled
  PRIMedsize       : 00000          0-65520
  TASKDATAloc     ==> Below          Below ! Any
  TASKDATAKey      ==> User          User ! Cics
  STorageclear     ==> No            No ! Yes
  RUNaway          ==> System        System ! 0 ! 500-2700000
  SHutdown         ==> Disabled      Disabled ! Enabled
  ISolate          ==> Yes           Yes ! No
  Brexit           ==>
+ REMOTE ATTRIBUTES

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                          TIME: 17.34.22 DATE: 01/19/26

```

La commande `CEDA DEFINE TRANSACTION(AJOU) GROUP(CLIGROUP) PROGRAM(PRGAJT)` associe le code "AJOU" au programme PRGAJT. On voit les paramètres : `PROFile (DFHCICST)`, `SStatus (Enabled)`, `TWAsize`, etc.

Installation de la transaction AJOU

```

INSTALL TRANSACTION(AJOU) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Engmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+ MAPset      ==>

                                SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL             TIME: 17.34.58 DATE: 01/19/26

```

La commande `CEDA INSTALL TRANSACTION(AJOU)` rend la transaction accessible aux utilisateurs. Le message "INSTALL SUCCESSFUL" confirme l'activation.

Vérification de la transaction avec CEMT

La commande `CEMT INQ` permet de vérifier que la transaction est bien active.

```

INQUIRE TRANSACTION(AJOU)
STATUS: RESULTS - OVERTYPE TO MODIFY
Tra(AJOU) Pri( 001 ) Pro(PRGAJT ) Tcl( DFHTCL00 ) Ena Sta
          Prf(DFHCICST) Uda Bel Iso          Bac Wai

```

La transaction `AJOU` est correctement installée et activée (`Ena Sta`). Elle est liée au programme `PRGAJT`.

Vérification avec CEDA VIEW

La commande `CEDA VIEW` affiche les caractéristiques détaillées de la transaction.

```

VIEW TRANSACTION(AJOU) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View TRANSAction( AJOU )
  TRANSAction    : AJOU
  Group          : CLIGROUP
  DEScription    :
  PROGram       : PRGAJT
  TWAsize        : 00000                      0-32767
  PROFile       : DFHCICST
  PArtitionset   :
  STATus        : Enabled                      Enabled ! Disabled
  PRIMedsize    : 00000                      0-65520
  TASKDATA Loc  : Below                       Below ! Any
  TASKDATAKey   : User                        User ! Cics
  STOrageclear  : No                          No ! Yes
  RUNaway       : System                      System ! 0 ! 500-2700000
  SHutdown      : Disabled                    Disabled ! Enabled
  ISolate       : Yes                         Yes ! No
  Brexit        :
+ REMOTE ATTRIBUTES

                                           SYSID=S670 APPLID=CICSTS51

```

Vue complète de la définition de la transaction AJOU : elle appartient au groupe CLIGROUP, est liée au programme PRGAJT, avec un statut Enabled.

Premier passage - Écran vide

Lorsque l'utilisateur lance la transaction AJOU, l'écran de saisie s'affiche vide.

```

*** AJOUT CLIENT ***
-----
NUMERO COMPTE :
CODE REGION   :      (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:      (AAAAMMJJ)
SEXE          :      (M OU F)
ACTIVITE PRO  :
SITUATION SOC :      (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      :      (DB OU CR)
-----
MESSAGE :  SAISIR LES DONNEES DU NOUVEAU CLIENT ET VALIDER

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER

```

L'écran de saisie MAPAJT s'affiche avec tous les champs vides et le message d'instruction "SAISIR LES DONNEES DU NOUVEAU CLIENT ET VALIDER".

Messages d'erreur de validation

Le programme valide les données saisies et affiche des messages d'erreur appropriés.

Aucune donnée saisie

```
*** AJOUT CLIENT ***
-----
NUMERO COMPTE : 
CODE REGION   :   (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 
NOM           : 
PRENOM        : 
DATE NAISSANCE:   (AAAAMMJJ)
SEXE          :   (M OU F)
ACTIVITE PRO  : 
SITUATION SOC :   (C/M/D/V)
ADRESSE       : 
SOLDE         : 
POSITION      :   (DB OU CR)
-----
MESSAGE :  AUCUNE DONNEE SAISIE - VEUILLEZ REMPLIR

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "AUCUNE DONNEE SAISIE - VEUILLEZ REMPLIR" lorsque l'utilisateur appuie sur ENTER sans avoir saisi de données (MAPFAIL).

Client existant (doublon)

```
*** AJOUT CLIENT ***
-----
NUMERO COMPTE : 
CODE REGION   :   (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 
NOM           : 
PRENOM        : 
DATE NAISSANCE:   (AAAAMMJJ)
SEXE          :   (M OU F)
ACTIVITE PRO  : 
SITUATION SOC :   (C/M/D/V)
ADRESSE       : 
SOLDE         : 
POSITION      :   (DB OU CR)
-----
MESSAGE :  ENREGISTREMENT EN DOUBLE - CE CLIENT EXISTE DEJA

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "ENREGISTREMENT EN DOUBLE - CE CLIENT EXISTE DEJA" lorsque le numéro de compte existe déjà dans le fichier VSAM.

Numéro de compte obligatoire

```
*** AJOUT CLIENT ***

-----

NUMERO COMPTE : -
CODE REGION   : (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE: (AAAAMMJJ)
SEXE          : (M OU F)
ACTIVITE PRO  :
SITUATION SOC : (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      : (DB OU CR)

-----

MESSAGE : NUMERO DE COMPTE OBLIGATOIRE

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "NUMERO DE COMPTE OBLIGATOIRE" lorsque le champ NUMCPT est vide.

Code région invalide

```
*** AJOUT CLIENT ***

-----

NUMERO COMPTE : -
CODE REGION   : (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE: (AAAAMMJJ)
SEXE          : (M OU F)
ACTIVITE PRO  :
SITUATION SOC : (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      : (DB OU CR)

-----

MESSAGE : CODE REGION INVALIDE (01/02/03/04)

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "CODE REGION INVALIDE (01/02/03/04)" lorsque le code région n'est pas une valeur autorisée.

Sexe invalide

```
*** AJOUT CLIENT ***
-----
NUMERO COMPTE : 
CODE REGION   :   (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 
NOM           : 
PRENOM        : 
DATE NAISSANCE:      (AAAAMMJJ)
SEXE          :   (M OU F)
ACTIVITE PRO  : 
SITUATION SOC :   (C/M/D/V)
ADRESSE       : 
SOLDE         : 
POSITION      :   (DB OU CR)
-----
MESSAGE :  SEXE INVALIDE (M OU F)

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "SEXE INVALIDE (M OU F)" lorsque le sexe saisi n'est pas M ou F.

Situation sociale invalide

```
*** AJOUT CLIENT ***
-----
NUMERO COMPTE : 
CODE REGION   :   (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 
NOM           : 
PRENOM        : 
DATE NAISSANCE:      (AAAAMMJJ)
SEXE          :   (M OU F)
ACTIVITE PRO  : 
SITUATION SOC :   (C/M/D/V)
ADRESSE       : 
SOLDE         : 
POSITION      :   (DB OU CR)
-----
MESSAGE :  SITUATION INVALIDE (C/M/D/V)

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "SITUATION INVALIDE (C/M/D/V)" lorsque la situation sociale n'est pas reconnue.

Position invalide


```
*** AJOUT CLIENT ***

-----

NUMERO COMPTE : _
CODE REGION   : (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE: (AAAAMMJJ)
SEXE          : (M OU F)
ACTIVITE PRO  :
SITUATION SOC : (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      : (DB OU CR)

-----

MESSAGE : POSITION INVALIDE (DB OU CR)

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "POSITION INVALIDE (DB OU CR)" lorsque la position n'est pas DB (débiteur) ou CR (créditeur).

Test d'ajout réussi - Premier client

Ajout du client RONALDO CRISTIANO avec toutes les données valides.

```
*** AJOUT CLIENT ***

-----

NUMERO COMPTE : 222222
CODE REGION   : 01 (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 10
NOM           : RONALDO
PRENOM        : CRISTIANO
DATE NAISSANCE: 19901212 (AAAAMMJJ)
SEXE          : M (M OU F)
ACTIVITE PRO  : 10
SITUATION SOC : C (C/M/D/V)
ADRESSE       : PORTUGAL
SOLDE         : 9999999999
POSITION      : DB (DB OU CR)

-----

MESSAGE : CLIENT AJOUTE AVEC SUCCES - NOUVEAU OU PF3

ENTER=VALIDER  PF3=QUITTER  CLEAR=EFFACER
```

Message "CLIENT AJOUTE AVEC SUCCES - NOUVEAU OU PF3" après l'ajout du client 222222 (RONALDO CRISTIANO, Paris, Célibataire, Débiteur).

Vérification avec AFFI

Après l'ajout, on vérifie que le client existe bien en utilisant la transaction d'affichage.

```
*** AFFICHAGE CLIENT ***
-----
NUMERO COMPTE : 222222
CODE REGION   : 01                                01 - PARIS
NATURE COMPTE : 10
NOM           : RONALDO
PRENOM        : CRISTIANO
DATE NAISSANCE: 19901212
SEXE          : M                                MASCULIN
ACTIVITE PRO  : 10
SITUATION SOC : C                                CELIBATAIRE
ADRESSE       : PORTUGAL
SOLDE         : 9999999999
POSITION      : DB                                DEBITEUR
-----
MESSAGE : CLIENT TROUVE - PF3=QUITTER OU NOUVELLE RECHERCHE

ENTER=RECHERCHER  PF3=QUITTER  CLEAR=EFFACER
```

Le client 222222 (RONALDO CRISTIANO) s'affiche correctement dans la transaction AFFI, confirmant que l'ajout a bien été effectué dans le fichier VSAM.

Session de débogage CEDF

Le débogueur CEDF permet de suivre l'exécution des commandes CICS pas à pas.

CEDF - SEND MAP (affichage écran)

```
TRANSACTION: AJOU PROGRAM: PRGAJT TASK: 0000152 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS SEND MAP
MAP ('MAPAJT ')
FROM ('.....t.....')
LENGTH (175)
MAPSET ('CLIAJT ')
TERMINAL
ERASE

OFFSET:X'0009DA' LINE:00135 EIBFN=X'1804'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur la commande SEND MAP : envoi de MAPAJT depuis le mapset CLIAJT. RESPONSE: NORMAL indique le succès de l'opération.

Saisie d'un nouveau client

```
*** AJOUT CLIENT ***
-----
NUMERO COMPTE : 333333
CODE REGION : 02 (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 10
NOM : GIL
PRENOM : GILBERTO
DATE NAISSANCE: 19851212 (AAAAMMJJ)
SEXE : M (M OU F)
ACTIVITE PRO : 10
SITUATION SOC : V (C/M/D/V)
ADRESSE : BRESIL
SOLDE : 8888888888
POSITION : CR (DB OU CR)
-----
MESSAGE : SAISIR LES DONNEES DU NOUVEAU CLIENT ET VALIDER

ENTER=VALIDER PF3=QUITTER CLEAR=EFFACER
```

Saisie des données pour un nouveau client : 333333, région 02 (Marseille), GIL GILBERTO, Veuf, Crédeur.

CEDF - RECEIVE MAP (réception saisie)

```

TRANSACTION: AJOU PROGRAM: PRGAJT TASK: 0000167 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS RECEIVE MAP
MAP ('MAPAJT ')
INTO ('.....333333...02...10...GIL          ...GILBERTO ...1985'...)
MAPSET ('CLIAJT ')
TERMINAL
NOHANDLE

```

OFFSET:X'000A46'
RESPONSE: **NORMAL**

LINE:00148
EIBRESP=0

EIBFN=X'1802'

ENTER: **CONTINUE**

PF1 : UNDEFINED

PF2 : SWITCH HEX/CHAR

PF3 : END EDF SESSION

PF4 : SUPPRESS DISPLAYS

PF5 : WORKING STORAGE

PF6 : USER DISPLAY

PF7 : SCROLL BACK

PF8 : SCROLL FORWARD

PF9 : STOP CONDITIONS

PF10: PREVIOUS DISPLAY

PF11: EIB DISPLAY

PF12: ABEND USER TASK

Point d'arrêt CEDF sur la commande RECEIVE MAP : réception des données saisies. On voit les valeurs transmises (333333, 02, 10, GIL, GILBERTO...). RESPONSE: NORMAL.

CEDF - READ FILE (vérification doublon)

```

TRANSACTION: AJOU PROGRAM: PRGAJT TASK: 0000167 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS READ
FILE ('FCLIENT ')
INTO ('333333 ...CLIENT TROUVE - PF3=QUITTER OU NOUVELL....CHERNHE      '...)
LENGTH (80)
RIDFLD ('333333')
EQUAL
NOHANDLE

```

OFFSET:X'000F44'
RESPONSE: **NOTFND**

LINE:00317
EIBRESP=13

EIBFN=X'0602'

ENTER: **CONTINUE**

PF1 : UNDEFINED

PF2 : SWITCH HEX/CHAR

PF3 : END EDF SESSION

PF4 : SUPPRESS DISPLAYS

PF5 : WORKING STORAGE

PF6 : USER DISPLAY

PF7 : SCROLL BACK

PF8 : SCROLL FORWARD

PF9 : STOP CONDITIONS

PF10: PREVIOUS DISPLAY

PF11: EIB DISPLAY

PF12: ABEND USER TASK

Point d'arrêt CEDF sur la commande READ FILE avec RIDFLD('333333'). **RESPONSE: NOTFND** (EIBRESP=13) est le résultat **attendu** : le client n'existe pas encore, on peut procéder à l'écriture.

CEDF - WRITE FILE (écriture client)

```
TRANSACTION: AJOU PROGRAM: PRGAJT TASK: 0000167 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS WRITE FILE
FILE ('FCLIENT ')
FROM ('3333330210GIL GILBERTO 19851212M10VBRESIL 8888888888CR'...)
LENGTH (80)
RIDFLD ('333333')
NOHANDLE

OFFSET:X'00105E' LINE:00358 EIBFN=X'0604'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur la commande WRITE FILE : écriture de l'enregistrement complet du client. On voit les données (3333330210GIL GILBERTO 19851212M10VBRESIL 8888888888CR). RESPONSE: NORMAL confirme l'écriture réussie.

Ajout réussi - Deuxième client

```
*** AJOUT CLIENT ***

-----
NUMERO COMPTE : 333333
CODE REGION : 02 (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 10
NOM : GIL
PRENOM : GILBERTO
DATE NAISSANCE: 19851212 (AAAAMMJJ)
SEXE : M (M OU F)
ACTIVITE PRO : 10
SITUATION SOC : V (C/M/D/V)
ADRESSE : BRESIL
SOLDE : 8888888888
POSITION : CR (DB OU CR)

-----
MESSAGE : CLIENT AJOUTE AVEC SUCCES - NOUVEAU OU PF3

ENTER=VALIDER PF3=QUITTER CLEAR=EFFACER
```

Message "CLIENT AJOUTE AVEC SUCCES" après l'ajout du client 333333 (GIL GILBERTO, Marseille, Veuf, Créditeur).

CEDF - SEND MAP final

```
TRANSACTION: AJOU PROGRAM: PRGAJT TASK: 0000167 APPLID: CICSTS51 DISPLAY: 00
- STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS SEND MAP
  MAP ('MAPAJT ')
  FROM ('.....t.....'...)
  LENGTH (175)
  MAPSET ('CLIAJT ')
  TERMINAL

OFFSET:X'000CEE' LINE:00214 EIBFN=X'1804'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur le SEND MAP final : envoi du message de succès à l'écran. RESPONSE: NORMAL.

Partie 2b : Opérations de Mise à Jour (REWRITE)

Cette section couvre les exercices 9 à 11 : MAP de mise à jour, programme de modification avec la commande REWRITE, et définition de la transaction MAJO.

WRITE vs REWRITE : Deux commandes d'écriture différentes

Après l'ajout (WRITE) dans la Partie 2a, cette section introduit la mise à jour (REWRITE). Ces deux commandes écrivent dans le fichier mais avec des logiques très différentes :

Aspect	WRITE (Ajout)	REWRITE (Mise à jour)
Action	Créer un nouvel enregistrement	Modifier un enregistrement existant
Client	Ne doit PAS exister	DOIT exister
Clé	Nouvelle (sera insérée)	Existante (non modifiable)
Prérequis	Aucun	READ UPDATE obligatoire
Verrouillage	Non	Oui (pendant READ UPDATE)
Erreur typique	DUPREC (doublon)	NOTFND (inexistant)

Point clé : Le REWRITE nécessite un READ UPDATE préalable dans la même unité de travail (UOW). En mode pseudo-conversationnel, cela signifie que les deux commandes doivent être exécutées dans le même passage du programme.

Exercice 9 : MAP pour mise à jour

Énoncé

Créer ou adapter la MAP précédente pour une opération de mise à jour de CLIENT dans le Data Set CLIENT.

Mon travail

J'ai créé une nouvelle MAP BMS (CLIMAJ) basée sur CLIAJT mais avec une particularité importante : la **gestion dynamique des attributs** du champ clé.

Pourquoi une gestion dynamique des attributs ?

En mise à jour, contrairement à l'ajout, le numéro de compte change de comportement au cours de la transaction :

1. **Phase 1 (Recherche)** : L'utilisateur doit pouvoir saisir le numéro du client à modifier → NUMCPT doit être **saisissable (UNPROT)**

2. **Phase 2 (Affichage)** : Une fois le client trouvé, son numéro s'affiche mais ne peut pas être modifié (la clé VSAM est immuable) → NUMCPT doit passer en **lecture seule (ASKIP)**
3. **Phase 3 (Modification)** : L'utilisateur modifie les autres champs et valide → NUMCPT reste **protégé (ASKIP)**

Cette gestion dynamique se fait dans le programme COBOL via le **suffixe 'A'** (Attribut) du copybook généré (voir Partie 1, Exercice 2 pour la structure DSECT et les suffixes L, F, A, I, O).

Comment modifier un attribut à l'exécution ?

Le copybook généré par l'assemblage BMS contient pour chaque champ nommé une variable suffixée **A** qui permet de changer son attribut dynamiquement :

```
* Après affichage du client, protéger le numéro de compte
* DFHBMASK = X'20' = ASKIP (protégé, intensité normale)
MOVE DFHBMASK TO NUMCPTA
```

Important : Les constantes d'attribut (DFHBMASK, DFHBMUNN, etc.) sont définies dans le copybook système **DFHBMSCA**. Il faut l'inclure dans le programme avec **COPY DFHBMSCA**.

Résolution

MAP BMS : CLIMAJ.bms

Le code source est stocké dans **ROCHA.CICS.SOURCE(CLIMAJ)**. La structure reprend les mêmes concepts BMS que CLIAFF et CLIAJT (voir Partie 1, Exercice 2 pour les explications sur DFHMDS, DFHMDI, DFHMDJ et les attributs).

Extrait du code BMS - En-tête avec commentaires explicatifs :

```
*****
* MAPSET : CLIMAJ - Mise a jour Client
* Transaction : MAJO
*
* PARTICULARITE MISE A JOUR :
* -----
* Le numero de compte est d'abord saisissable (recherche),
* puis passe en lecture seule apres affichage des donnees.
* Cette gestion dynamique des attributs se fait dans le programme
* COBOL via le suffixe 'A' (ex: NUMCPTA pour modifier l'attribut).
*
* Attribut DFHBMASK = X'20' = ASKIP (protege, normal)
* Attribut DFHBMUNN = X'4C' = UNPROT + NUM (saisie numerique)
*****
CLIMAJ    DFHMDS TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,                X
          STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
```

Extrait - Champ clé avec indication "non modifiable" :


```

*-----
* NUMERO DE COMPTE - CHAMP CLE
* Commence en UNPROT pour la saisie initiale (recherche)
* Le programme passera l'attribut a ASKIP apres affichage
*-----
          DFHMD F POS=(4,2),LENGTH=16,ATTRB=ASKIP,                                X
          INITIAL='NUMERO COMPTE : '
NUMCPT   DFHMD F POS=(4,19),LENGTH=6,ATTRB=(UNPROT,NUM,IC)
          DFHMD F POS=(4,26),LENGTH=20,ATTRB=ASKIP,                                X
          INITIAL='(Cle - non modifiable) '

```

Différence clé avec CLIAJT : L'indication "(Clé - non modifiable)" rappelle à l'utilisateur que la clé VSAM ne peut pas être changée après création du client. Le programme gère dynamiquement le passage de UNPROT à ASKIP via **MOVE DFHBMASK TO NUMCPTA**.

Zones de la MAP :

Zone	Longueur	Attribut initial	Comportement dynamique
NUMCPT	6	UNPROT,NUM,IC	Devient ASKIP après recherche (via NUMCPTA)
CODREG	2	UNPROT,NUM	Reste saisissable
NATCPT	2	UNPROT,NUM	Reste saisissable
NOM	10	UNPROT	Reste saisissable
PRENOM	10	UNPROT	Reste saisissable
DATNA	8	UNPROT,NUM	Reste saisissable
SEXE	1	UNPROT	Reste saisissable
ACTPRO	2	UNPROT,NUM	Reste saisissable
SITSO	1	UNPROT	Reste saisissable
ADRESSE	10	UNPROT	Reste saisissable
SOLDE	10	UNPROT,NUM	Reste saisissable
POSIT	2	UNPROT	Reste saisissable
MSG	60	ASKIP,BRT	Zone message (affichage seul)

Constantes d'attribut BMS (DFHBMSCA)

Le copybook **DFHBMSCA** contient les constantes hexadécimales pour modifier les attributs à l'exécution :

Constante	Valeur	Description	Usage typique
DFHBMASK	X'20'	ASKIP - Protégé, intensité normale	Protéger un champ
DFHBMPRF	X'28'	ASKIP - Protégé, brillant	Mise en évidence

Constante	Valeur	Description	Usage typique
DFHBMUNN	X'4C'	UNPROT + NUM - Saisie numérique	Rendre saisissable (chiffres)
DFHBMUNP	X'40'	UNPROT - Saisie alphanumérique	Rendre saisissable (texte)
DFHBMFSE	X'08'	MDT forcé	Forcer la transmission
DFHBMPRO	X'20'	PROT - Protégé	Synonyme de DFHBMASK

Rappel MDT : Le MDT (Modified Data Tag) indique si un champ a été modifié. Avec FRSET dans la MAP, les MDT sont remis à zéro au SEND MAP. Seuls les champs modifiés par l'utilisateur sont transmis au RECEIVE MAP (voir Partie 1, Exercice 2).

JCL d'assemblage : ASMMAJ.jcl

Le JCL d'assemblage suit la même structure que ASMCLAF.jcl (voir Partie 1, Exercice 2). Seuls le nom du job (ROCHA09) et le membre source (CLIMAJ) changent.

Définition CICS

La définition et l'installation du mapset suivent le même processus que pour les mapsets précédents (voir Partie 1, Exercice 4 pour les explications sur CEDA) :

```
CEDA DEFINE MAPSET(CLIMAJ) GROUP(CLIGROUP)
CEDA INSTALL MAPSET(CLIMAJ) GROUP(CLIGROUP)
```

Vérification

```
CEDA VIEW MAPSET(CLIMAJ) GROUP(CLIGROUP)
```

Note : **CEMT INQ MAPSET** n'existe pas dans CICS. Pour vérifier un mapset, utiliser **CEDA VIEW**.

Captures d'écran

Résultat de l'assemblage BMS

Après soumission du JCL ASMMAJ, le job d'assemblage s'exécute avec succès.

```

      No Statements Flagged in this Assembly
HIGH LEVEL ASSEMBLER, 5696-234, RELEASE 6.0, PTF UK92594
SYSTEM: z/OS 01.13.00          JOBNAME: ROCHA09      STEPNAME: ASSEM      PRO
Data Sets Allocated for this Assembly
Con DDname   Data Set Name                               Volume  Member
P1 SYSIN     SYS26015.T112513.RA000.ROCHA09.TEMPM.H01
L1 SYSLIB    DFH510.CICS.SDFHMAC                          FDC511
L2           SYS1.MACLIB                                    FDRES1
      SYSPRINT IBMUSER.ROCHA09.JOB00153.D0000102.?
      SYSPUNCH SYS26015.T112513.RA000.ROCHA09.MAP.H01

32100K allocated to Buffer Pool      Storage required   1124K
 115 Primary Input Records Read      6922 Library Records Read
   0 ASMAOPT Records Read            897 Primary Print Records Written
  31 Object Records Written          0 ADATA Records Written
Assembly Start Time: 11.25.13 Stop Time: 11.25.13 Processor Time: 00.00.00.3441
Return Code 000
***** BOTTOM OF DATA *****

```

Le job ROCHA09 (assemblage BMS) retourne Return Code 000. On note 115 Primary Input Records Read et 31 Object Records Written, confirmant la génération correcte du mapset CLIMAJ.

Définition du mapset CLIMAJ dans CICS

Après l'assemblage BMS réussi, on définit le mapset dans CICS avec CEDA.

```

DEFINE MAPSET(CLIMAJ) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFine MApset( CLIMAJ )
  Mapset      : CLIMAJ
  Group       : CLIGROUP
  DEscription ==> _
  REsident    ==> No           No ! Yes
  USAge       ==> Normal      Normal ! Transient
  USElpacopy  ==> No           No ! Yes
  Status      ==> Enabled     Enabled ! Disabled
  RSl         : 00            0-24 ! Public
DEFINITION SIGNATURE
DEFinetime   : 01/15/26 11:30:18
CHANGETime   : 01/15/26 11:30:18
CHANGEUsrid  : CICSUSER
CHANGEAGEnt  : CSDApi         CSDApi ! CSDBatch
CHANGEAGRel  : 0680

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 11.30.18 DATE: 01/15/26
PF 1 HELP 2 COM 3 END                        6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA DEFINE MAPSET(CLIMAJ) GROUP(CLIGROUP)` crée la définition du mapset. Le message "DEFINE SUCCESSFUL" confirme la création.

Installation du mapset CLIMAJ

```

INSTALL MAPSET(CLI MAJ) GROUP(CLI GROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==>
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  J0urnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSRpool     ==>
+ MAPset      ==> CLIMAJ

                                SYSID=S670 APPLID=CICSTS51
                                TIME: 11.31.36 DATE: 01/15/26
INSTALL SUCCESSFUL
PF 1 HELP      3 END      6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA INSTALL MAPSET(CLI MAJ)` charge le mapset en mémoire CICS. Le message "INSTALL SUCCESSFUL" indique que le mapset est prêt à être utilisé.

Vérification de la définition

```

VIEW MAPSET(CLI MAJ) GROUP(CLI GROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View Mapset( CLIMAJ )
  MAPset      : CLIMAJ
  Group       : CLIGROUP
  DEScription :
  RESident    : No                No ! Yes
  USAge       : Normal            Normal ! Transient
  USElpacopy  : No                No ! Yes
  Status      : Enabled            Enabled ! Disabled
  RSL         : 00                0-24 ! Public
DEFINITION SIGNATURE
  DEFInetime  : 01/15/26 11:30:18
  CHANGETime  : 01/15/26 11:30:18
  CHANGEUsrid : CICSUSER
  CHANGEAGEnt : CSDApi            CSDApi ! CSDBatch
  CHANGEAGRel : 0680

```

`CEDA VIEW` permet de consulter tous les paramètres du mapset : nom, groupe, résidence, et statut d'installation.

Exercice 10 : Programme de mise à jour (REWRITE)

Énoncé

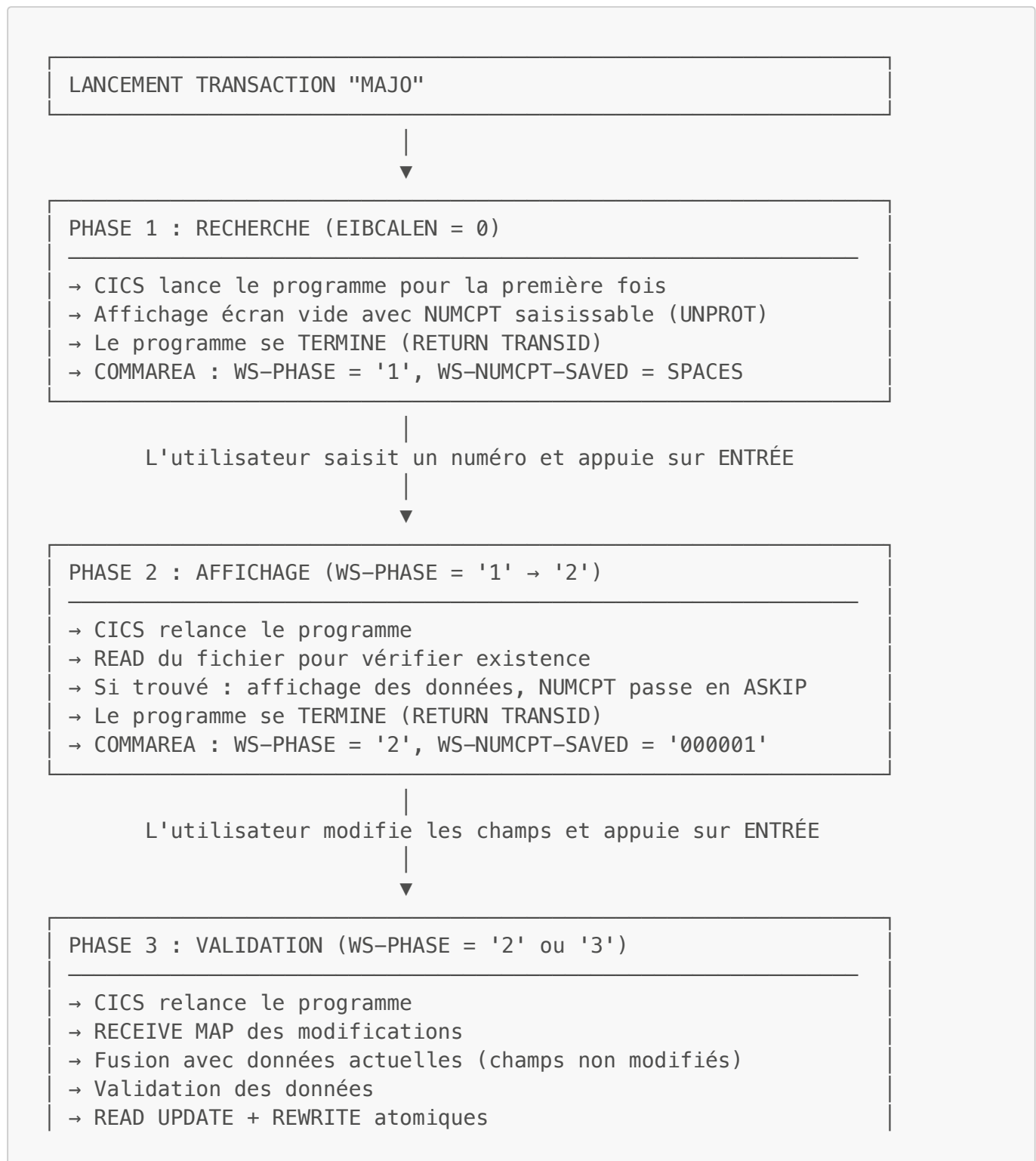
Créer le PROGRAMME pour une opération de mise à jour d'un CLIENT dans le Data Set CLIENT. Un contrôle de conformité de donnée et d'existence doit être effectué.

Mon travail

J'ai développé le programme PRGMAJ qui gère la mise à jour des clients existants. Ce programme présente plusieurs différences importantes par rapport à PRGAJT (ajout).

Pourquoi un mode pseudo-conversationnel à 3 phases ?

Contrairement à l'ajout (2 phases), la mise à jour nécessite 3 phases distinctes car l'utilisateur doit d'abord **rechercher** le client avant de le **modifier** :



→ Retour en phase 1 pour nouveau client

Voir Partie 1, Exercice 3 pour les explications détaillées sur le mode pseudo-conversationnel et les variables EIB.

Pourquoi une COMMAREA étendue ?

En mise à jour, la COMMAREA doit sauvegarder plus d'informations qu'en ajout :

Programme	Contenu COMMAREA	Raison
PRGAJT (Ajout)	WS-FLAG-INIT (1 octet)	Distinguer premier passage
PRGMAJ (MAJ)	WS-PHASE (1 octet) + WS-NUMCPT-MAJ (6 octets)	Phase + numéro protégé

Pourquoi sauvegarder le numéro de compte ?

Une fois le champ NUMCPT protégé (ASKIP), le terminal ne le transmet plus au programme lors du RECEIVE MAP. Or, on a besoin de ce numéro pour relire et modifier le client. La COMMAREA permet de le conserver entre les passages.

Pourquoi fusionner les modifications ?

C'est une différence majeure avec l'ajout. En mise à jour :

- L'utilisateur ne modifie que **certains** champs (ex: changer l'adresse uniquement)
- Les champs non modifiés ne sont pas transmis par le terminal (longueur = 0)
- Si on écrivait directement les valeurs reçues, on écraserait les autres champs avec des espaces !

Solution : Relire le client, puis ne remplacer que les champs dont la longueur > 0.

Pourquoi READ UPDATE + REWRITE dans le même paragraphe ?

En CICS, la commande **REWRITE** nécessite un **READ UPDATE** préalable dans la **même unité de travail (UOW)**.

Problème : En mode pseudo-conversationnel, chaque passage est une nouvelle tâche CICS → nouvelle UOW.

Conséquence : On ne peut PAS faire :

- Phase 2 : READ UPDATE (verrouillage)
- -- Fin de tâche --
- Phase 3 : REWRITE (échec car pas de verrouillage actif)

Solution : Faire les deux dans le même passage, juste avant l'écriture :

Phase 2 : READ simple (affichage) → Fin de tâche
 Phase 3 : READ UPDATE + REWRITE (atomique) → Fin de tâche

Résolution

Programme : PRGMAJ.cbl

Le code source est stocké dans **ROCHA.CICS.SOURCE(PRGMAJ)**. Voici les sections clés du programme.

Structure de la COMMAREA étendue (WORKING-STORAGE) :

```
*-----
* ZONE DE COMMUNICATION (COMMAREA)
* Sauvegarde la phase et le numéro de compte entre passages
*-----
01 WS-COMMAREA.
   05 WS-PHASE          PIC X(01) VALUE '1'.
   88 PHASE-RECHERCHE   VALUE '1'.
   88 PHASE-AFFICHAGE   VALUE '2'.
   88 PHASE-VALIDATION  VALUE '3'.
   05 WS-NUMCPT-MAJ     PIC X(06) VALUE SPACES.
```

LINKAGE SECTION (obligatoire pour recevoir la COMMAREA) :

```
LINKAGE SECTION.
*-----
* ZONE COMMAREA PASSEE PAR CICS
* OBLIGATOIRE pour accéder aux données du RETURN précédent
*-----
01 DFHCOMMAREA.
   05 LS-PHASE          PIC X(01).
   05 LS-NUMCPT-MAJ     PIC X(06).
```

Point d'entrée avec gestion des phases :

```
0000-PRINCIPAL.
  EVALUATE TRUE
    WHEN EIBCALEN = 0
      * Premier appel - Phase recherche
      PERFORM 1000-INIT-RECHERCHE
    WHEN EIBAID = DFHPF3
      * PF3 - Fin de transaction
      PERFORM 9000-FIN-PROGRAMME
    WHEN EIBAID = DFHCLEAR
      * CLEAR - Réinitialiser
      PERFORM 1000-INIT-RECHERCHE
```

```

        WHEN OTHER
*           Traitement selon la phase en cours
            MOVE DFHCOMMAREA TO WS-COMMAREA
            PERFORM 2000-TRAITEMENT THRU 2000-FIN
        END-EVALUATE

*   Retour pseudo-conversationnel
    EXEC CICS RETURN
        TRANSID('MAJO')
        COMMAREA(WS-COMMAREA)
        LENGTH(LENGTH OF WS-COMMAREA)
    END-EXEC.

```

Paragraphe d'affichage avec protection du NUMCPT :

```

3100-AFFICHER-CLIENT.
*-----
* Affiche les données du client dans la MAP
* NUMCPT passe en ASKIP (protégé) – clé non modifiable
*-----
    MOVE LOW-VALUES TO MAPMAJO

*   Transfert des données vers la MAP
    MOVE CLI-NUMCPT    TO NUMCPT0
    MOVE CLI-CODREG    TO CODREG0
    MOVE CLI-NATCPT    TO NATCPT0
    MOVE CLI-NOM       TO NOM0
    MOVE CLI-PRENOM    TO PRENOM0
    MOVE CLI-DATNAISS  TO DATNA0
    MOVE CLI-SEXE      TO SEXE0
    MOVE CLI-ACTPRO    TO ACTPRO0
    MOVE CLI-SITSO     TO SITSO0
    MOVE CLI-ADRESSE   TO ADRESSE0
    MOVE CLI-SOLDE     TO SOLDE0
    MOVE CLI-POSITION  TO POSITO

*   IMPORTANT : Protéger le numéro de compte (clé non modifiable)
*   DFHBMASK = X'20' = ASKIP (protégé, intensité normale)
    MOVE DFHBMASK TO NUMCPTA

    MOVE 'CLIENT TROUVE – MODIFIER ET VALIDER AVEC ENTER' TO MSG0

    EXEC CICS SEND MAP('MAPMAJ')
        MAPSET('CLIMAJ')
        ERASE
    END-EXEC.

```

Paragraphe de fusion des modifications :

4050-FUSIONNER-MODIFICATIONS.

```

*-----
* Fusionne les modifications de l'utilisateur avec les données
* actuelles du client. Seuls les champs modifiés (longueur > 0)
* remplacent les valeurs existantes.
*-----
*   Code région : si modifié, prendre la nouvelle valeur
      IF WS-CODREGL > 0
        MOVE WS-CODREG TO CLI-CODREG
      ELSE
        MOVE CLI-CODREG TO WS-CODREG
      END-IF

*   Nom : si modifié, prendre la nouvelle valeur
      IF WS-NOML > 0
        MOVE WS-NOM TO CLI-NOM
      ELSE
        MOVE CLI-NOM TO WS-NOM
      END-IF

*   Sexe : si modifié, prendre la nouvelle valeur
      IF WS-SEXEL > 0
        MOVE WS-SEXE TO CLI-SEXE
      ELSE
        MOVE CLI-SEXE TO WS-SEXE
      END-IF

*   ... (même logique pour tous les champs)

```

Paragraphe READ UPDATE + REWRITE atomique :

4300-ECRIRE-MODIFICATION.

```

*-----
* Mise à jour de l'enregistrement avec READ UPDATE + REWRITE
*
* IMPORTANT : Le REWRITE nécessite un READ UPDATE préalable
* dans la même unité de travail (UOW).
*-----
*   READ UPDATE pour verrouiller l'enregistrement
      EXEC CICS READ
        FILE('FCLIENT')
        INTO(ENR-CLIENT)
        RIDFLD(CLI-NUMCPT)
        UPDATE
        RESP(WS-RESP)
      END-EXEC

      IF WS-RESP NOT = DFHRESP(NORMAL)
        MOVE 'ERREUR VERROUILLAGE - REESSAYEZ' TO MSGO
        MOVE '0' TO WS-ERREUR
        GO TO 4300-FIN

```

```

END-IF

*   Réappliquer les modifications sur l'enregistrement lu
MOVE WS-CODREG      TO CLI-CODREG
MOVE WS-NATCPT      TO CLI-NATCPT
MOVE WS-NOM         TO CLI-NOM
MOVE WS-PRENOM      TO CLI-PRENOM
MOVE WS-DATNAISS    TO CLI-DATNAISS
MOVE WS-SEXE        TO CLI-SEXE
MOVE WS-ACTPRO      TO CLI-ACTPRO
MOVE WS-SITSO       TO CLI-SITSO
MOVE WS-ADRESSE     TO CLI-ADRESSE
MOVE WS-SOLDE       TO CLI-SOLDE
MOVE WS-POSITION    TO CLI-POSITION

*   REWRITE - Mise à jour effective
EXEC CICS REWRITE
      FILE('FCLIENT')
      FROM(ENR-CLIENT)
      RESP(WS-RESP)
END-EXEC

EVALUATE WS-RESP
  WHEN DFHRESP(NORMAL)
    MOVE LOW-VALUES TO MAPMAJO
    MOVE WS-NUMCPT TO NUMCPTO
    MOVE 'MISE A JOUR EFFECTUEE - NOUVEAU OU PF3'
      TO MSGO
*   Retour en phase recherche pour nouveau client
    MOVE '1' TO WS-PHASE
    MOVE SPACES TO WS-NUMCPT-MAJ
  WHEN OTHER
    MOVE 'ERREUR MISE A JOUR - CONTACTEZ SUPPORT' TO MSGO
    MOVE '0' TO WS-ERREUR
END-EVALUATE.

4300-FIN.
EXIT.

```

JCL de compilation : CMPMAJ.jcl

Le JCL de compilation suit la même structure que CMPCLAF.jcl (voir Partie 1, Exercice 3). Seuls le nom du job (ROCHA10) et le membre source (PRGMAJ) changent.

Note : Ce programme nécessite trois copybooks : **DFHAID** (touches fonction), **DFHBMSCA** (constantes attribut), et **CLIMAJ** (structure MAP).

Structure du programme

Paragraphe	Fonction
0000-PRINCIPAL	Point d'entrée, aiguillage selon EIBCALEN et EIBAUD

Paragraphe	Fonction
1000-INIT-RECHERCHE	Affichage écran vide pour saisie numéro
2000-TRAITEMENT	Aiguillage selon la phase en cours
3000-RECHERCHER-CLIENT	Phase 1→2 : Recherche et affichage
3100-AFFICHER-CLIENT	Transfert données vers MAP, protection NUMCPT
4000-VALIDER-MODIFICATION	Phase 2/3 : Réception et validation
4050-FUSIONNER-MODIFICATIONS	Fusion modifications/données actuelles
4100-VALIDER-DONNEES	Contrôles de conformité
4300-ECRIRE-MODIFICATION	READ UPDATE + REWRITE atomique
9000-FIN-PROGRAMME	Message de fin et RETURN sans TRANSID

Commandes CICS utilisées

Commande	Usage
SEND MAP	Envoyer l'écran (avec ERASE pour effacer)
RECEIVE MAP	Recevoir la saisie avec RESP pour MAPFAIL
READ FILE	Lecture simple (phase recherche/relecture)
READ UPDATE	Verrouillage pour REWRITE
REWRITE FILE	Mise à jour de l'enregistrement
RETURN TRANSID	Retour pseudo-conversationnel avec COMMAREA
SEND TEXT	Message de fin (sans MAP)

Messages d'erreur gérés

Message	Contexte
SAISIR LE NUMERO DE COMPTE A MODIFIER	Premier passage
VEUILLEZ SAISIR UN NUMERO DE COMPTE	MAPFAIL en phase recherche
NUMERO DE COMPTE OBLIGATOIRE	Champ NUMCPT vide
NUMERO DE COMPTE DOIT ETRE NUMERIQUE	Caractères non numériques
CLIENT INEXISTANT - VERIFIEZ LE NUMERO	NOTFND lors du READ
CLIENT TROUVE - MODIFIER ET VALIDER	Affichage réussi
AUCUNE MODIFICATION - ENTREZ DES DONNEES	MAPFAIL en phase validation
CODE REGION INVALIDE (01/02/03/04)	Code différent des valeurs autorisées
NOM OBLIGATOIRE	Champ NOM vide après fusion

Message	Contexte
SEXE INVALIDE (M OU F)	Sexe différent de M ou F
SITUATION INVALIDE (C/M/D/V)	Situation non reconnue
POSITION INVALIDE (DB OU CR)	Position non reconnue
ERREUR VERROUILLAGE - REESSAYEZ	Échec du READ UPDATE
MISE A JOUR EFFECTUEE - NOUVEAU OU PF3	REWRITE réussi

Définition CICS

```
CEDA DEFINE PROGRAM(PRGMAJ) GROUP(CLIGROUP) LANGUAGE(COBOL)
CEDA INSTALL PROGRAM(PRGMAJ) GROUP(CLIGROUP)
```

Vérification

```
CEMT INQ PROGRAM(PRGMAJ)
```

Résultat attendu : **Prog(PRGMAJ) Cob Ena**

Points importants

1. **COPY DFHBMSCA** : Copybook système contenant les constantes d'attribut (DFHBMASK, DFHBMUNN, etc.). Obligatoire pour modifier dynamiquement les attributs des champs.
2. **Sauvegarde du NUMCPT dans la COMMAREA** : Une fois protégé (ASKIP), le champ n'est plus transmis par le terminal. On doit le conserver dans WS-NUMCPT-SAVED pour les phases suivantes.
3. **READ simple en phase recherche** : La première lecture n'utilise pas UPDATE car le verrouillage ne persisterait pas après la fin de tâche (mode pseudo-conversationnel).
4. **READ UPDATE + REWRITE atomiques** : Les deux commandes doivent être dans le même paragraphe, exécutées séquentiellement, pour garantir que le verrouillage est actif au moment du REWRITE.
5. **Retour en phase 1 après succès** : Après une mise à jour réussie, le programme réinitialise la COMMAREA pour permettre la modification d'un autre client sans relancer la transaction.
6. **PERFORM THRU avec GO TO** : Comme pour PRGAJT (voir Partie 2a, Exercice 7), la clause THRU permet aux GO TO de rester dans la plage du PERFORM et de retourner correctement à l'appelant.

Difficultés rencontrées et solutions

Problème 1 : LINKAGE SECTION manquante pour DFHCOMMAREA

Symptôme : Après le premier passage, le message "ERREUR RELECTURE CLIENT" s'affichait car le numéro de compte sauvegardé (WS-NUMCPT-**SAVED**) était vide.

Cause : La LINKAGE SECTION était complètement absente du programme. Sans elle, les données passées via COMMAREA lors du RETURN précédent ne sont pas accessibles.

Solution : Ajouter la LINKAGE SECTION avec DFHCOMMAREA pour recevoir les données du passage précédent :

```
LINKAGE SECTION.
*-----
* ZONE COMMAREA PASSEE PAR CICS
* OBLIGATOIRE pour accéder aux données du RETURN précédent
*-----
01 DFHCOMMAREA.
   05 LS-PHASE          PIC X(01).
   05 LS-NUMCPT-SAVED  PIC X(06).
```

Rappel : En mode pseudo-conversationnel, chaque passage est une nouvelle tâche CICS. La LINKAGE SECTION est obligatoire pour récupérer le contexte sauvegardé.

Problème 2 : Données effacées lors de la mise à jour partielle

Symptôme : Quand l'utilisateur ne modifiait qu'un seul champ (ex: situation sociale), le message "CODE REGION OBLIGATOIRE" s'affichait malgré une région valide à l'écran.

Cause : Le terminal BMS ne transmet que les champs **modifiés** (longueur > 0). Les champs affichés mais non modifiés ont une longueur = 0, même s'ils contiennent des données visibles à l'écran.

Solution : Ajouter le paragraphe **4050-FUSIONNER-MODIFICATIONS** qui :

1. Relit le client pour récupérer les valeurs actuelles
2. Pour chaque champ : si longueur > 0, utilise la nouvelle valeur ; sinon, conserve l'existante

```
4050-FUSIONNER-MODIFICATIONS.
*   Code région : si modifié, prendre la nouvelle valeur
    IF WS-CODREGL > 0
        MOVE WS-CODREG TO CLI-CODREG
    ELSE
        MOVE CLI-CODREG TO WS-CODREG
    END-IF
*   ... (même logique pour tous les champs)
```

Problème 3 : CEDA INSTALL GROUP échoue

Symptôme : La commande **CEDA INSTALL GROUP(CLIGROUP)** retournait une erreur car le fichier FCLIENT était ouvert.

Cause : Réinstaller tout le groupe tente de réinstaller **toutes** les ressources, y compris les fichiers déjà ouverts.

Solution : Installer uniquement la ressource spécifique :

```
CEDA INSTALL TRANSACTION(MAJO) GROUP(CLIGROUP)
```

Bonne pratique : Toujours installer individuellement les nouvelles ressources plutôt que réinstaller tout le groupe.

Amélioration future : Conservation des données lors des erreurs

Problème identifié : Comme pour le programme d'ajout (PRGAJT), lorsqu'une erreur de validation se produit, l'écran est effacé (**ERASE**) et seul le message d'erreur s'affiche. L'utilisateur doit ressaisir les modifications, ce qui n'est pas ergonomique.

Cause technique : L'option **ERASE** dans la commande **SEND MAP** efface l'écran entier. Combiné avec **MOVE LOW-VALUES TO MAPMAJO**, les données ne sont pas renvoyées à l'écran.

Solution envisagée : Lors d'une erreur de validation en phase 3, recopier les données sauvegardées (WS-SAISIE) vers les zones output (MAPMAJO) avant le **SEND MAP**, et utiliser **DATAONLY** au lieu de **ERASE** :

```
* Amélioration : conserver les données lors d'une erreur
IF ERREUR-DETECTEE
    MOVE WS-NUMCPT TO NUMCPTO
    MOVE WS-CODREG TO CODREGO
    MOVE WS-NOM    TO NOMO
    ...
    EXEC CICS SEND MAP('MAPMAJ')
        MAPSET('CLIMAJ')
        DATAONLY          ← Ne rafraîchit que les données
    END-EXEC
END-IF
```

Note : Cette amélioration, commune à PRGAJT et PRGMAJ, n'a pas été implémentée dans la version actuelle. Elle constitue une évolution pour améliorer l'expérience utilisateur.

Captures d'écran

Définition du programme PRGMAJ dans CICS

Après la compilation COBOL réussie, on définit le programme dans CICS.

```

DEFINE PROGRAM(PRGMAJ) GROUP(CLIGROUP) LANGUAGE(COBOL)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA Define PROGram( PRGMAJ )
  PROGram      : PRGMAJ
  Group        : CLIGROUP
  DEScription   ==> _
  Language      ==> CObol          CObol ! Assembler ! Le370 ! C ! Pli
  REload        ==> No             No ! Yes
  RESident      ==> No             No ! Yes
  USAge         ==> Normal         Normal ! Transient
  USElpacopy    ==> No             No ! Yes
  Status        ==> Enabled        Enabled ! Disabled
  RSl           : 00              0-24 ! Public
  CEdf          ==> Yes            Yes ! No
  DATalocation  ==> Below          Below ! Any
  EXECKey       ==> User           User ! Cics
  CONcurrency   ==> Quasirent      Quasirent ! Threadsafe ! Required
  Api           ==> Cicsapi        Cicsapi ! Openapi
  REMOTE ATTRIBUTES
+  DYNamic      ==> No             No ! Yes

                                           SYSID=S670 APPLID=CICSTS51
  DEFINE SUCCESSFUL                        TIME: 12.16.13  DATE: 01/15/26
PF 1 HELP 2 COM 3 END                    6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA DEFINE PROGRAM(PRGMAJ) GROUP(CLIGROUP) LANGUAGE(COBOL)` crée la définition du programme. Le message "DEFINE SUCCESSFUL" confirme la création.

Installation du programme PRGMAJ

```

INSTALL PROGRAM(PRGMAJ) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice   ==> _
  Bundle        ==>
  CONnection    ==>
  CORbaserver   ==>
  DB2Conn       ==>
  DB2Entry      ==>
  DB2Tran       ==>
  DJar          ==>
  DOctemplate   ==>
  Enqmodel      ==>
  File          ==>
  Ipconn        ==>
  JOurnalmodel  ==>
  JVmserver     ==>
  LIBrary       ==>
  LSrpool       ==>
+  MAPset       ==>

                                           SYSID=S670 APPLID=CICSTS51
  INSTALL SUCCESSFUL                      TIME: 12.17.45  DATE: 01/15/26
PF 1 HELP      3 END                    6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA INSTALL PROGRAM(PRGMAJ)` charge le programme compilé en mémoire CICS. Le message "INSTALL SUCCESSFUL" indique que le programme est prêt.

Vérification avec CEMT

```
INQUIRE PROGRAM(PRGMAJ)
STATUS:  RESULTS - OVERTYPE TO MODIFY
  Prog(PRGMAJ  ) Leng(0000000000) Cob Pro Ena Pri      Ced
      Res(0000) Use(0000000000) Bel Uex Ful Qua Cic
```

CEMT INQ PROGRAM(PRGMAJ) affiche le statut du programme : "Cob" (COBOL), "Pro" (Protected), "Ena" (Enabled). Le programme est correctement installé et activé.

Compilation du programme PRGMAJ

Le job de compilation COBOL s'exécute avec succès.

```
* Statistics for COBOL program PRGMAJ:
*   Source records = 1134
*   Data Division statements = 328
*   Procedure Division statements = 249
End of compilation 1, program PRGMAJ, no statements flagged.
Return code 0
***** BOTTOM OF DATA *****
```

Statistiques de compilation du programme PRGMAJ : 1134 enregistrements sources, 328 instructions DATA DIVISION, 249 instructions PROCEDURE DIVISION. Return code 0 confirme la compilation réussie.

Test d'erreur - Client inexistant

Le programme gère correctement le cas où le client recherché n'existe pas.

*** MISE A JOUR CLIENT ***

```
-----
NUMERO COMPTE :      (CLE - NON MODIFIABL
CODE REGION   :      (01=PAR,02=MAR,03=LY0,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:      (AAAAMMJJ)
SEXE          :      (M OU F)
ACTIVITE PRO  :
SITUATION SOC :      (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      :      (DB OU CR)
-----
```

MESSAGE : CLIENT INEXISTANT - VERIFIEZ LE NUMERO

ENTER=VALIDER PF3=QUITTER CLEAR=REINIT.

Message "CLIENT INEXISTANT - VERIFIEZ LE NUMERO" lorsque l'utilisateur saisit un numéro de compte qui n'existe pas dans le fichier VSAM (NOTFND).

Exercice 11 : Transaction de mise à jour

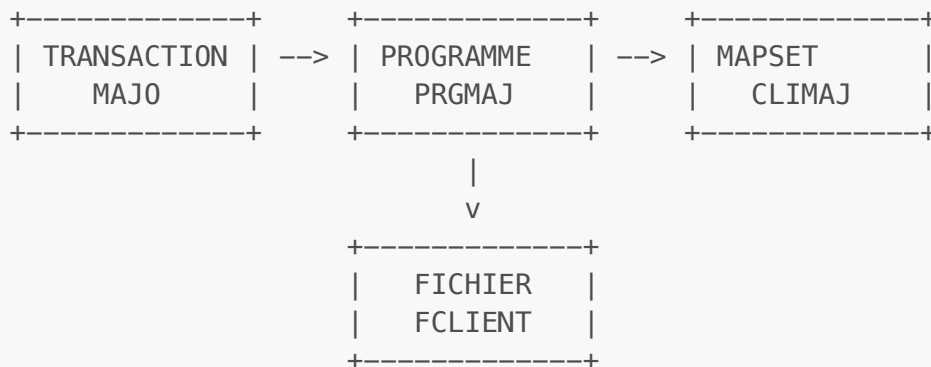
Énoncé

Définir une transaction indépendante de la précédente pour appeler le programme de mise à jour.

Mon travail

La transaction MAJO est le point d'entrée utilisateur pour la mise à jour des clients. Comme pour AFFI et AJOU, elle fait le lien entre le code saisi par l'utilisateur et le programme COBOL-CICS à exécuter.

Architecture CICS - Liaison des ressources



Résolution

Étape 1 : Définition de la transaction

```
CEDA DEFINE TRANSACTION(MAJO) GROUP(CLIGROUP) PROGRAM(PRGM AJ)
```

Paramètre	Valeur	Description
TRANSACTION	MAJO	Code transaction (4 caractères max)
GROUP	CLIGROUP	Groupe de ressources du projet
PROGRAM	PRGM AJ	Programme COBOL à exécuter

Étape 2 : Installation de la transaction

```
CEDA INSTALL TRANSACTION(MAJO) GROUP(CLIGROUP)
```

Bonne pratique : Installer uniquement la ressource ajoutée (**CEDA INSTALL TRANSACTION**) plutôt que tout le groupe (**CEDA INSTALL GROUP**). Réinstaller le groupe complet peut causer des erreurs si certaines ressources (comme FCLIENT) sont déjà ouvertes.

Vérification

```
CEDA VIEW TRANSACTION(MAJO) GROUP(CLIGROUP)
```

Résultat attendu : Affichage de la définition avec PROGRAM(PRGMAJ)

```
CEMT INQ TRAN(MAJO)
```

Résultat attendu : Tra(MAJO) Pro(PRGMAJ) Ena

Test de la transaction

Test sans debugger :

```
MAJO
```

Comportement attendu :

1. Écran de saisie du numéro de compte (NUMCPT saisissable)
2. Saisir un numéro existant (ex: 000001) et ENTER
3. Affichage des données du client (NUMCPT protégé/grisé)
4. Modifier les champs souhaités (ex: changer l'adresse)
5. ENTER pour valider → Message "MISE A JOUR EFFECTUEE"
6. L'écran revient en phase 1 pour un nouveau client
7. PF3 pour quitter

Test avec CEDF (voir Partie 1, Exercice 5 pour la navigation CEDF) :

```
CEDF  
MAJO
```

Points d'arrêt observés pour une mise à jour complète :

Étape	Commande CICS	RESP attendu	Phase
1	SEND MAP	NORMAL	1 - Écran recherche
2	RETURN TRANSID	-	Fin phase 1
3	RECEIVE MAP	NORMAL	2 - Réception numéro
4	READ FILE	NORMAL	2 - Vérification existence
5	SEND MAP	NORMAL	2 - Affichage client

Étape	Commande CICS	RESP attendu	Phase
6	RETURN TRANSID	-	Fin phase 2
7	RECEIVE MAP	NORMAL	3 - Réception modifications
8	READ FILE	NORMAL	3 - Relecture données
9	READ UPDATE	NORMAL	3 - Verrouillage
10	REWRITE	NORMAL	3 - Mise à jour
11	SEND MAP	NORMAL	3 - Message succès
12	RETURN TRANSID	-	Retour phase 1

Note : Si le client n'existe pas, l'étape 4 retourne NOTFND et le programme affiche un message d'erreur sans passer à la phase 2.

Ressources du groupe CLIGROUP après exercice 11

Type	Nom	Description	Défini dans
FILE	FCLIENT	Fichier VSAM clients	Exercice 1
MAPSET	CLIAFF	Écran affichage	Exercice 4
MAPSET	CLIAJT	Écran ajout	Exercice 8
MAPSET	CLIMAJ	Écran mise à jour	Exercice 9
PROGRAM	PRGCLIA	Programme affichage	Exercice 4
PROGRAM	PRGAJT	Programme ajout	Exercice 8
PROGRAM	PRGMAJ	Programme mise à jour	Exercice 10
TRANSACTION	AFFI	Transaction affichage	Exercice 4
TRANSACTION	AJOU	Transaction ajout	Exercice 8
TRANSACTION	MAJO	Transaction mise à jour	Exercice 11

Captures d'écran

Définition de la transaction MAJO

La transaction fait le lien entre le code utilisateur et le programme COBOL.

```

DEFINE TRANSACTION(MAJO) GROUP(CLIGROUP) PROGRAM(PRGMAJ)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFINE TRANsAction( MAJO )
  TRANsAction    : MAJO
  Group          : CLIGROUP
  DEscription    ==> _
  PROgram        ==> PRGMAJ
  TWasize        ==> 00000                0-32767
  PROfile        ==> DFHCICST
  PArtitionset   ==>
  STatus         ==> Enabled              Enabled ! Disabled
  PRIMedsize     : 00000                0-65520
  TASKDATAloc    ==> Below               Below ! Any
  TASKDATAkey    ==> User                 User ! Cics
  STorageclear   ==> No                  No ! Yes
  RUNaway        ==> System               System ! 0 ! 500-2700000
  SHutdown       ==> Disabled             Disabled ! Enabled
  ISolate        ==> Yes                  Yes ! No
  Brexit         ==>
+ REMOTE ATTRIBUTES

                                SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                TIME: 12.30.37 DATE: 01/15/26
PF 1 HELP 2 COM 3 END           6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA DEFINE TRANSACTION(MAJO) GROUP(CLIGROUP) PROGRAM(PRGMAJ)` associe le code "MAJO" au programme PRGMAJ. Le message "DEFINE SUCCESSFUL" confirme la création.

Installation de la transaction MAJO

```

INSTALL TRANSACTION(MAJO) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice    ==> _
  Bundle         ==>
  CONnection     ==>
  CORbaserver    ==>
  DB2Conn        ==>
  DB2Entry       ==>
  DB2Tran        ==>
  DJar           ==>
  D0ctemplate    ==>
  Enqmodel       ==>
  File           ==>
  Ipconn         ==>
  J0urnalmodel   ==>
  JVmserver      ==>
  LIBrary        ==>
  LSrpool        ==>
+ MApset         ==>

                                SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL              TIME: 12.31.52 DATE: 01/15/26

```

La commande `CEDA INSTALL TRANSACTION(MAJO)` rend la transaction accessible aux utilisateurs. Le message "INSTALL SUCCESSFUL" indique que la transaction est active.

Vérification de la définition

```

VIEW TRANSACTION(MAJO) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View TRANSAction( MAJO )
  TRANSAction      : MAJO
  Group            : CLIGROUP
  DEScription      :
  PROGram          : PRGMAJ
  TWasize          : 00000                      0-32767
  PROFile          : DFHCICST
  PARTitionset     :
  STATus           : Enabled                    Enabled ! Disabled
  PRIMedsize       : 00000                      0-65520
  TASKDATA Loc     : Below                      Below ! Any
  TASKDATA Key     : User                       User ! Cics
  STOrageclear     : No                         No ! Yes
  RUNaway          : System                     System ! 0 ! 500-2700000
  SHutdown         : Disabled                   Disabled ! Enabled
  ISolate          : Yes                       Yes ! No
  Brexit           :
+ REMOTE ATTRIBUTES

```

CEDA VIEW permet de vérifier les paramètres de la transaction : code MAJO, programme associé PRGMAJ, et groupe CLIGROUP.

Test fonctionnel - Phase 1 : Saisie du numéro

Après avoir tapé "MAJO" sur l'écran CICS, l'écran de mise à jour s'affiche.

```

*** MISE A JOUR CLIENT ***
-----
NUMERO COMPTE :      (CLE - NON MODIFIABL
CODE REGION   :      (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:      (AAAAMMJJ)
SEXE          :      (M OU F)
ACTIVITE PRO  :
SITUATION SOC :      (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      :      (DB OU CR)
-----
MESSAGE :  SAISIR LE NUMERO DE COMPTE A MODIFIER

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.

```

Phase 1 : L'écran s'affiche vide avec le curseur sur le champ NUMCPT. L'utilisateur doit saisir le numéro du client à modifier.

Test fonctionnel - Saisie du numéro de compte

```
*** MISE A JOUR CLIENT ***
-----
NUMERO COMPTE : 000001 (CLE - NON MODIFIABL
CODE REGION   : _ (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE: (AAAAMMJJ)
SEXE          : (M OU F)
ACTIVITE PRO  :
SITUATION SOC : (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      : (DB OU CR)
-----
MESSAGE : SAISIR LE NUMERO DE COMPTE A MODIFIER

ENTER=VALIDER PF3=QUITTER CLEAR=REINIT.
```

L'utilisateur a saisi le numéro de compte "000001" et appuie sur ENTER pour rechercher le client.

Test fonctionnel - Phase 2 : Affichage des données

```
*** MISE A JOUR CLIENT ***
-----
NUMERO COMPTE : 000001 (CLE - NON MODIFIABL
CODE REGION   : 01 (01=PAR,02=MAR,03=LYO,04=
NATURE COMPTE : 20
NOM           : DUPONT
PRENOM        : JEAN
DATE NAISSANCE: 19850315 (AAAAMMJJ)
SEXE          : M (M OU F)
ACTIVITE PRO  : 10
SITUATION SOC : m (C/M/D/V)
ADRESSE       : PARIS
SOLDE         : 0000150000
POSITION      : CR (DB OU CR)
-----
MESSAGE : CLIENT TROUVE - MODIFIER ET VALIDER AVEC ENTER

ENTER=VALIDER PF3=QUITTER CLEAR=REINIT.
```

Phase 2 : Le client a été trouvé. Toutes ses données sont affichées et le message "CLIENT TROUVE" confirme la recherche réussie. L'utilisateur peut maintenant modifier les champs souhaités et valider avec ENTER.

Session de débogage CEDF - Mise à jour complète

Le débogueur CEDF permet de suivre l'exécution des commandes CICS pas à pas lors d'une mise à jour.

CEDF - RECEIVE MAP (réception du numéro)

```

_TRANSACTION: MAJO PROGRAM: PRGMAJ TASK: 0000191 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS RECEIVE MAP
MAP ('MAPMAJ ')
INTO ('.....000001.....')
MAPSET ('CLIMAJ ')
TERMINAL
NOHANDLE

OFFSET:X'000CFE' LINE:00199 EIBFN=X'1802'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK

```

Point d'arrêt CEDF sur la commande RECEIVE MAP : réception du numéro de compte 000001 saisi par l'utilisateur. RESPONSE: NORMAL.

CEDF - READ FILE (lecture du client)

```

_TRANSACTION: MAJO PROGRAM: PRGMAJ TASK: 0000191 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS READ
FILE ('FCLIENT ')
INTO ('0000010110DUPONT .....19850315M..M.....CR'...)
LENGTH (80)
RIDFLD ('000001')
EQUAL
NOHANDLE

OFFSET:X'000F10' LINE:00242 EIBFN=X'0602'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK

```

Point d'arrêt CEDF sur la commande READ FILE avec RIDFLD('000001'). Le client DUPONT est trouvé (données visibles : 19850315M..M...CR). RESPONSE: NORMAL.

Écran - Client trouvé

```
*** MISE A JOUR CLIENT ***
-----
NUMERO COMPTE : 000001 (CLE - NON MODIFIABL
CODE REGION   : 01 (01=PAR,02=MAR,03=LY0,04=
NATURE COMPTE : 10
NOM            : DUPONT
PRENOM        : JOSUE
DATE NAISSANCE: 19850315 (AAAAMMJJ)
SEXE          : M (M OU F)
ACTIVITE PRO  :
SITUATION SOC : M (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      : CR (DB OU CR)
-----
MESSAGE : CLIENT TROUVE - MODIFIER ET VALIDER AVEC ENTER

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

L'écran affiche les données du client 000001 (DUPONT JOSUE). Le message "CLIENT TROUVE - MODIFIER ET VALIDER AVEC ENTER" invite l'utilisateur à effectuer ses modifications. Noter que le NUMCPT est maintenant protégé (clé non modifiable).

CEDF - SEND MAP (affichage client)


```

TRANSACTION: MAJO PROGRAM: PRGMAJ TASK: 0000191 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS SEND MAP
MAP ('MAPMAJ ')
FROM ('.....t....0000001...01...10...DUPONT .....1985'...)
LENGTH (175)
MAPSET ('CLIMAJ ')
TERMINAL
ERASE

```

OFFSET:X'00110C'
RESPONSE: **NORMAL**

LINE:00307
EIBRESP=0

EIBFN=X'1804'

ENTER: **CONTINUE**

PF1 : UNDEFINED

PF2 : **SWITCH HEX/CHAR**

PF3 : **END EDF SESSION**

PF4 : **SUPPRESS DISPLAYS**

PF5 : **WORKING STORAGE**

PF6 : **USER DISPLAY**

PF7 : **SCROLL BACK**

PF8 : **SCROLL FORWARD**

PF9 : **STOP CONDITIONS**

PF10: **PREVIOUS DISPLAY**

PF11: **EIB DISPLAY**

PF12: **ABEND USER TASK**

Point d'arrêt CEDF sur la commande SEND MAP : envoi de l'écran avec les données du client (0000001...01...10...DUPONT...). RESPONSE: NORMAL.

CEDF - REWRITE (mise à jour VSAM)

```

TRANSACTION: MAJO PROGRAM: PRGMAJ TASK: 0000202 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS REWRITE
FILE ('FCLIENT ')
FROM ('0000010110DUPONT JOSUE 19850315M10M.....CR'...)
LENGTH (80)
NOHANDLE

```

OFFSET:X'00181C'
RESPONSE: **NORMAL**

LINE:00583
EIBRESP=0

EIBFN=X'0606'

ENTER: **CONTINUE**

PF1 : UNDEFINED

PF2 : **SWITCH HEX/CHAR**

PF3 : **END EDF SESSION**

PF4 : **SUPPRESS DISPLAYS**

PF5 : **WORKING STORAGE**

PF6 : **USER DISPLAY**

PF7 : **SCROLL BACK**

PF8 : **SCROLL FORWARD**

PF9 : **STOP CONDITIONS**

PF10: **PREVIOUS DISPLAY**

PF11: **EIB DISPLAY**

PF12: **ABEND USER TASK**

Point d'arrêt CEDF sur la commande REWRITE : écriture de l'enregistrement modifié dans le fichier FCLIENT. On voit les données complètes (000001 DUPONT JOSUE 19850315M10M...CR). RESPONSE: NORMAL confirme la mise à jour réussie.

Écran - Mise à jour effectuée

```
*** MISE A JOUR CLIENT ***

-----

NUMERO COMPTE : 000001 (CLE - NON MODIFIABL
CODE REGION   :      (01=PAR,02=MAR,03=LY0,04=
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:      (AAAAMMJJ)
SEXE          :      (M OU F)
ACTIVITE PRO  :
SITUATION SOC :      (C/M/D/V)
ADRESSE       :
SOLDE         :
POSITION      :      (DB OU CR)

-----

MESSAGE : MISE A JOUR EFFECTUEE - NOUVEAU OU PF3

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

Message "MISE A JOUR EFFECTUEE - NOUVEAU OU PF3" confirmant le succès de l'opération. L'écran est réinitialisé pour permettre la modification d'un autre client.

CEDF - SEND MAP (message de succès)

```
TRANSACTION: MAJO PROGRAM: PRGMAJ TASK: 0000202 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS SEND MAP
MAP ('MAPMAJ ')
FROM ('.....t....0000001.....')
LENGTH (175)
MAPSET ('CLIMAJ ')
TERMINAL
ERASE

OFFSET:X'00149E' LINE:00402 EIBFN=X'1804'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur le SEND MAP final : envoi du message de succès à l'utilisateur. RESPONSE: NORMAL.

Vérification avec AFFI

Après la mise à jour, on peut vérifier les modifications avec la transaction d'affichage.

```
*** AFFICHAGE CLIENT ***
-----
NUMERO COMPTE : 000001
CODE REGION   : 01                01 - PARIS
NATURE COMPTE : 10
NOM           : DUPONT
PRENOM        : JOSUE
DATE NAISSANCE : 19850315
SEXE          : M                MASCULIN
ACTIVITE PRO  : 10
SITUATION SOC : M                MARIE(E)
ADRESSE       :
SOLDE         :
POSITION      : CR                CREDITEUR
-----
MESSAGE : CLIENT TROUVE - PF3=QUITTER OU NOUVELLE RECHERCHE

ENTER=RECHERCHER  PF3=QUITTER  CLEAR=EFFACER
```

Vérification avec la transaction AFFI : le client 000001 (DUPONT JOSUE) s'affiche avec ses données mises à jour. On note la situation sociale "M" traduite en "MARIE(E)" et la position "CR" traduite en "CREDITEUR".

Partie 2c : Opérations de Suppression (DELETE)

Cette section couvre les exercices 12 à 15 : MAP de suppression, programme de suppression avec la commande DELETE, et définition de transaction.

La commande DELETE : Supprimer des enregistrements VSAM

Après avoir maîtrisé READ (lecture), WRITE (ajout) et REWRITE (mise à jour), cette partie introduit la dernière opération CRUD : la suppression avec DELETE.

Caractéristiques de DELETE

Aspect	DELETE	Comparaison avec REWRITE
Fonction	Supprimer un enregistrement	Modifier un enregistrement
Prérequis	Aucun	READ UPDATE obligatoire
Verrouillage	Non nécessaire	Obligatoire
Erreur si absent	NOTFND	NOTFND
Atomicité	Oui (opération unique)	Non (READ + REWRITE)

Point clé : Contrairement à REWRITE, la commande DELETE est autonome et ne nécessite pas de READ UPDATE préalable. Elle supprime directement l'enregistrement identifié par RIDFLD.

Variantes de suppression en CICS

CICS offre plusieurs façons de supprimer des enregistrements :

Variante	Syntaxe	Usage
DELETE simple	<code>DELETE FILE(...) RIDFLD(clé)</code>	Supprime un enregistrement par sa clé exacte
DELETE avec GENERIC	<code>DELETE FILE(...) RIDFLD(préfixe) KEYLENGTH(...) GENERIC</code>	Supprime tous les enregistrements dont la clé commence par le préfixe
DELETE en browse	<code>DELETE FILE(...) RIDFLD(...) (après READNEXT)</code>	Supprime l'enregistrement courant lors d'un parcours

Dans cette partie, nous implémentons le **DELETE simple** avec confirmation visuelle. Le **DELETE avec GENERIC** sera traité dans la Partie 3 (Exercice 17) pour la suppression de plusieurs clients en une seule opération.

Mon choix de conception

L'énoncé original prévoyait deux programmes distincts :

- **Exercice 13** : Suppression directe (DELETE sans affichage préalable)
- **Exercice 15** : Suppression avec lecture préalable (READ + affichage + DELETE)

J'ai fait le choix de développer directement la version complète (avec lecture et confirmation) dès l'exercice 13, car c'est la bonne pratique en environnement de production. On ne supprime jamais de données sans permettre à l'utilisateur de vérifier visuellement ce qu'il supprime.

Ce qui était prévu	Ce que j'ai implémenté	Justification
Ex 13 : DELETE direct	DELETE avec confirmation	Sécurité des données
Ex 15 : READ + DELETE	Déjà couvert par Ex 13	Évite code redondant

Bonne pratique mainframe : En production, une suppression accidentelle peut avoir des conséquences graves. L'affichage préalable et la confirmation explicite (O/N) sont des garde-fous essentiels.

Exercice 12 : MAP pour suppression

Énoncé

Créer ou adapter la MAP précédente pour une opération de suppression de CLIENT dans le Data Set CLIENT.

Mon travail

J'ai créé une nouvelle MAP de suppression (CLISUP) qui combine les caractéristiques des MAPs précédentes.

Pourquoi une MAP spécifique pour la suppression ?

La suppression nécessite un écran hybride :

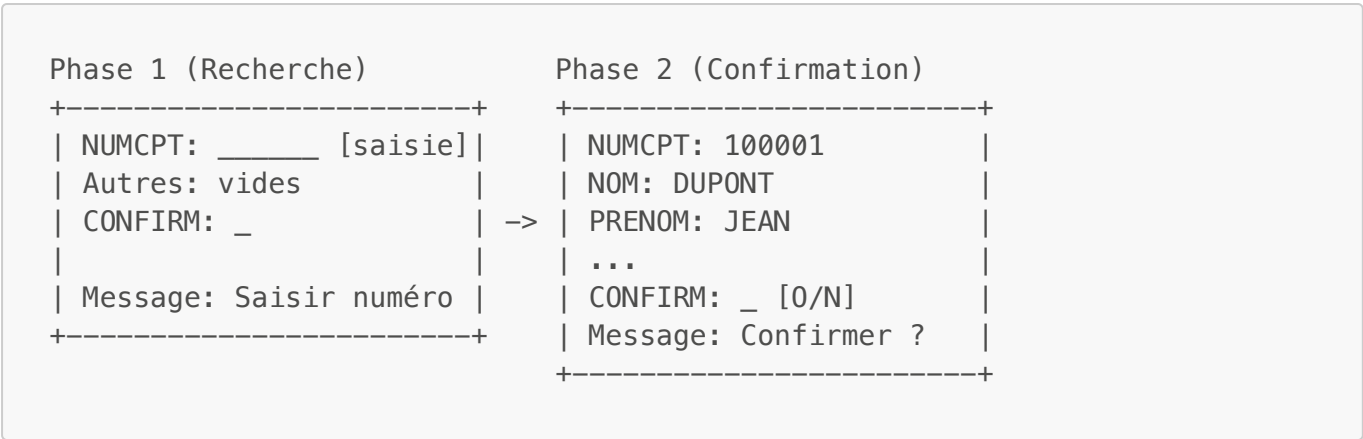
1. **Phase recherche** : Le numéro de compte est saisissable (UNPROT) pour identifier le client
2. **Phase confirmation** : Les données sont affichées en lecture seule (ASKIP,BRT) pour que l'utilisateur vérifie qu'il supprime le bon client
3. **Champ CONFIRM** : Un nouveau champ (O/N) permet de valider ou annuler la suppression

Différences avec les autres MAPs

Aspect	CLIAFF (Affichage)	CLIAJT (Ajout)	CLIMAJ (Maj)	CLISUP (Suppression)
NUMCPT	UNPROT (saisie)	UNPROT (saisie)	UNPROT→ASKIP	UNPROT (saisie)
Autres champs	ASKIP (affichage)	UNPROT (saisie)	UNPROT (modif)	ASKIP (affichage)
Confirmation	Non	Non	Non	Oui (O/N)

Aspect	CLIAFF (Affichage)	CLIAJT (Ajout)	CLIMAJ (Maj)	CLISUP (Suppression)
Libellés	Oui (LIBREG...)	Non	Non	Oui (LIBREG...)

Flux de suppression en 2 phases



Résolution

MAP BMS : CLISUP.bms

Le code source est stocké dans `ROCHA.CICS.SOURCE(CLISUP)`. La structure reprend les mêmes concepts BMS que les MAPs précédentes (voir Partie 1, Exercice 2 pour les explications sur DFHMSD, DFHMDI, DFHMDF et les attributs).

Extrait du code BMS - En-tête avec commentaires explicatifs :

```
*****
* MAPSET : CLISUP - Suppression Client
* Transaction : SUPP / SULE
*
* PARTICULARITE SUPPRESSION :
* -----
* Le numero de compte est saisi pour rechercher le client.
* Les donnees sont affichees en lecture seule pour confirmation.
* Un champ CONFIRM (O/N) permet de valider la suppression.
*****
CLISUP    DFHMSD TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,                                X
          STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
```

Extrait - Zones d'affichage en lecture seule (ASKIP,BRT) :

```
*-----
* ZONES D'AFFICHAGE - DONNEES CLIENT (LECTURE SEULE)
*-----
          DFHMDF POS=(6,2),LENGTH=16,ATTRB=ASKIP,                                X
          INITIAL='CODE REGION    :'
```

```

CODREG  DFHMDF POS=(6,19),LENGTH=2,ATTRB=(ASKIP,BRT)
        DFHMDF POS=(6,25),LENGTH=20,ATTRB=ASKIP
LIBREG  DFHMDF POS=(6,46),LENGTH=15,ATTRB=(ASKIP,BRT)

```

Différence clé avec les autres MAPs : Tous les champs de données sont en **ASKIP, BRT** (protégés, brillants) car l'utilisateur ne peut que visualiser, pas modifier. Seuls NUMCPT (recherche) et CONFIRM (O/N) sont saisissables.

Extrait - Zone de confirmation (élément spécifique à la suppression) :

```

*-----
* ZONE DE CONFIRMATION
*-----
        DFHMDF POS=(18,2),LENGTH=30,ATTRB=(ASKIP,BRT),
        INITIAL='CONFIRMER SUPPRESSION (O/N) : '
CONFIRM DFHMDF POS=(18,33),LENGTH=1,ATTRB=UNPROT
        DFHMDF POS=(18,35),LENGTH=1,ATTRB=ASKIP

```

Élément distinctif : Le champ CONFIRM est unique à cette MAP. Il permet une validation explicite avant la suppression irréversible.

Zones de la MAP :

Zone	Longueur	Attribut	Description
NUMCPT	6	UNPROT,NUM,IC	Numéro de compte (clé de recherche)
CODREG	2	ASKIP,BRT	Code région (affichage)
LIBREG	15	ASKIP,BRT	Libellé région
NOM	10	ASKIP,BRT	Nom client (affichage)
PRENOM	10	ASKIP,BRT	Prénom client (affichage)
...	...	ASKIP,BRT	Autres champs en lecture seule
CONFIRM	1	UNPROT	Confirmation O/N (saisie)
MSG	60	ASKIP,BRT	Zone message

Note : Contrairement à CLIMAJ, le champ NUMCPT ne passe pas dynamiquement en ASKIP après la recherche. Il reste techniquement saisissable mais l'utilisateur se concentre sur le champ CONFIRM.

JCL d'assemblage : ASMSUP.jcl

Le JCL d'assemblage suit la même structure que ASMCLAF.jcl (voir Partie 1, Exercice 2). Seuls le nom du job (ROCHA12) et le membre source (CLISUP) changent.

Définition CICS

La définition et l'installation du mapset suivent le même processus que pour les mapsets précédents (voir Partie 1, Exercice 4 pour les explications sur CEDA) :

```
CEDA DEFINE MAPSET(CLISUP) GROUP(CLIGROUP)
CEDA INSTALL MAPSET(CLISUP) GROUP(CLIGROUP)
```

Vérification

```
CEDA VIEW MAPSET(CLISUP) GROUP(CLIGROUP)
```

Captures d'écran

Définition du mapset CLISUP dans CICS

Après l'assemblage BMS réussi, on définit le mapset dans CICS avec CEDA.

```
DEFINE MAPSET(CLISUP) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEfine MApset( CLISUP  )
  MApset      : CLISUP
  Group       : CLIGROUP
  DEScription ==> _
  RESident    ==> No           No ! Yes
  USAge       ==> Normal      Normal ! Transient
  USElpacopy  ==> No           No ! Yes
  Status      ==> Enabled     Enabled ! Disabled
  RSL         : 00            0-24 ! Public
DEFINITION SIGNATURE
DEFinetime   : 01/15/26 15:51:20
CHANGETime   : 01/15/26 15:51:20
CHANGEUsrid  : CICSUSER
CHANGEAGent  : CSDApi         CSDApi ! CSDBatch
CHANGEAGRel  : 0680

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                          TIME: 15.51.20 DATE: 01/15/26
```

La commande `CEDA DEFINE MAPSET(CLISUP) GROUP(CLIGROUP)` crée la définition du mapset de suppression. Le message "DEFINE SUCCESSFUL" confirme la création.

Installation du mapset CLISUP


```

INSTALL MAPSET(CLISUP) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==>
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+  MApset      ==> CLISUP

                                SYSID=S670 APPLID=CICSTS51
                                TIME: 15.52.35 DATE: 01/15/26

INSTALL SUCCESSFUL

```

La commande CEDA INSTALL MAPSET(CLISUP) charge le mapset en mémoire CICS. Le message "INSTALL SUCCESSFUL" indique que le mapset est prêt.

Vérification de la définition

```

VIEW MAPSET(CLISUP) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS
CEDA View MApset( CLISUP )
  MApset      : CLISUP
  Group       : CLIGROUP
  DEScription :
  RESident    : No          No ! Yes
  USAge       : Normal      Normal ! Transient
  USElpacopy  : No          No ! Yes
  Status      : Enabled     Enabled ! Disabled
  RSl         : 00          0-24 ! Public
DEFINITION SIGNATURE
  DEFInetime  : 01/15/26 15:51:20
  CHANGETime  : 01/15/26 15:51:20
  CHANGEUsrid : CICSUSER
  CHANGEAGEnt : CSDApi      CSDApi ! CSDBatch
  CHANGEAGRel : 0680

                                SYSID=S670 APPLID=CICSTS51

```

CEDA VIEW permet de consulter tous les paramètres du mapset : nom, groupe, résidence, et statut d'installation.

Résultat de l'assemblage BMS

```

      No Statements Flagged in this Assembly
HIGH LEVEL ASSEMBLER, 5696-234, RELEASE 6.0, PTF UK92594
SYSTEM: z/OS 01.13.00          JOBNAME: ROCHA12      STEPNAME: ASSEM      PRO
Data Sets Allocated for this Assembly
Con DDname    Data Set Name                                Volume  Member
P1 SYSIN      SYS26015.T154852.RA000.ROCHA12.TEMPM.H01
L1 SYSLIB     DFH510.CICS.SDFHMAC                          FDC511
L2            SYS1.MACLIB                                  FDRES1
      SYSPRINT IBMUSER.ROCHA12.JOB00160.D0000102.?
      SYSPUNCH SYS26015.T154852.RA000.ROCHA12.MAP.H01

      32100K allocated to Buffer Pool      Storage required    1144K
      114 Primary Input Records Read      6922 Library Records Read
      0 ASMAOPT Records Read              958 Primary Print Records Written
      34 Object Records Written           0 ADATA Records Written
Assembly Start Time: 15.48.52 Stop Time: 15.48.52 Processor Time: 00.00.00.3450
Return Code 000
***** BOTTOM OF DATA *****

```

Le job ROCHA12 (assemblage BMS) retourne Return Code 000. On note 114 Primary Input Records Read et 34 Object Records Written, confirmant la génération correcte du mapset CLISUP.

Exercice 13 : Programme de suppression (DELETE)

Énoncé

Créer le PROGRAMME pour une opération de suppression d'un CLIENT dans le Data Set CLIENT en précisant le code CLIENT. Un contrôle de conformité de donnée et d'existence doit être effectué.

Mon travail

J'ai développé le programme PRGSUP qui gère la suppression de clients avec confirmation visuelle.

Note : J'ai directement implémenté la version complète avec lecture préalable et affichage des données (prévue initialement pour l'exercice 15). Cette approche est la bonne pratique en production.

Pourquoi DELETE ne nécessite pas READ UPDATE ?

C'est une différence importante avec REWRITE :

Commande	Prérequis	Raison
REWRITE	READ UPDATE obligatoire	L'enregistrement doit être verrouillé pour la modification
DELETE	Aucun	La suppression est atomique, pas besoin de verrouillage préalable

La commande DELETE supprime directement par la clé (RIDFLD). Si le client n'existe pas, elle retourne NOTFND.

Pourquoi un mode à 2 phases avec confirmation ?

PHASE 1 : RECHERCHE (EIBCALEN = 0 ou WS-PHASE = '1')

- L'utilisateur saisit un numéro de compte
- READ pour vérifier existence et récupérer les données
- Affichage des données pour confirmation visuelle
- WS-PHASE passe à '2'

|
L'utilisateur voit les données et répond 0 ou N
|
▼

PHASE 2 : CONFIRMATION (WS-PHASE = '2')

- Réception de la réponse 0/N
- Si N : Message "SUPPRESSION ANNULÉE", retour phase 1
- Si 0 : DELETE RIDFLD, message "CLIENT SUPPRIMÉ"
- Retour en phase 1 pour nouveau client

Voir Partie 1, Exercice 3 pour les explications détaillées sur le mode pseudo-conversationnel et les variables EIB.

Résolution

Programme : PRGSUP.cbl

Le code source est stocké dans `ROCHA.CICS.SOURCE(PRGSUP)`. Voici les extraits clés spécifiques à la suppression.

Structure de la COMMAREA :

```
01  WS-COMMAREA.
    05  WS-PHASE          PIC X(01) VALUE '1'.
        88  PHASE-RECHERCHE VALUE '1'.
        88  PHASE-CONFIRM  VALUE '2'.
    05  WS-NUMCPT-MAJ      PIC X(06) VALUE SPACES.
```

La COMMAREA contient la phase et le numéro de compte sauvegardé pour la suppression.

Paragraphe de confirmation avec validation O/N :

```
4000-CONFIRMER-SUPPRESSION.
*-----
* Phase 2 : Réception de la confirmation et suppression
*-----
EXEC CICS RECEIVE MAP('MAPSUP')
```

```

        MAPSET('CLISUP')
        RESP(WS-RESP)
    END-EXEC

    IF WS-RESP = DFHRESP(MAPFAIL)
        MOVE LOW-VALUES TO MAPSUPO
        MOVE WS-NUMCPT-MAVED TO NUMCPTO
        MOVE DFHBMASK TO NUMCPTA
        MOVE 'VEUILLEZ REPONDRE O OU N' TO MSGO
        EXEC CICS SEND MAP('MAPSUP')
            MAPSET('CLISUP')
            ERASE
        END-EXEC
        GO TO 4000-FIN
    END-IF

```

```

*   Sauvegarde de la confirmation
    MOVE CONFIRMI TO WS-CONFIRM
    MOVE CONFIRML TO WS-CONFIRML

```

```

*   Vérification de la réponse (accepte O/N majuscules et
    minuscules)

```

```

    IF WS-CONFIRM NOT = 'O' AND WS-CONFIRM NOT = 'N'
        AND WS-CONFIRM NOT = 'o' AND WS-CONFIRM NOT = 'n'
        MOVE LOW-VALUES TO MAPSUPO
        MOVE WS-NUMCPT-MAVED TO NUMCPTO
        MOVE DFHBMASK TO NUMCPTA
        MOVE 'REPONSE INVALIDE - SAISIR O OU N' TO MSGO
        EXEC CICS SEND MAP('MAPSUP')
            MAPSET('CLISUP')
            ERASE
        END-EXEC
        GO TO 4000-FIN
    END-IF

```

```

*   Si N ou n : Annulation
    IF WS-CONFIRM = 'N' OR WS-CONFIRM = 'n'
        MOVE LOW-VALUES TO MAPSUPO
        MOVE 'SUPPRESSION ANNULEE - NOUVEAU NUMERO OU PF3' TO MSGO
        MOVE '1' TO WS-PHASE
        MOVE SPACES TO WS-NUMCPT-MAVED
        EXEC CICS SEND MAP('MAPSUP')
            MAPSET('CLISUP')
            ERASE
        END-EXEC
        GO TO 4000-FIN
    END-IF

```

```

*   Si O ou o : Suppression
    PERFORM 4100-SUPPRIMER-CLIENT THRU 4100-FIN.

```

```

4000-FIN.
EXIT.

```

Paragraphe de suppression avec DELETE :

```

4100-SUPPRIMER-CLIENT.
*-----
* Suppression effective de l'enregistrement
* La commande DELETE ne nécessite PAS de READ UPDATE préalable
*-----
      MOVE WS-NUMCPT-SAVED TO CLI-NUMCPT

      EXEC CICS DELETE
            FILE('FCLIENT')
            RIDFLD(CLI-NUMCPT)
            RESP(WS-RESP)
      END-EXEC

      EVALUATE WS-RESP
        WHEN DFHRESP(NORMAL)
          MOVE LOW-VALUES TO MAPSUPO
          MOVE 'CLIENT SUPPRIME - NOUVEAU NUMERO OU PF3' TO MSGO
          Retour en phase recherche
          MOVE '1' TO WS-PHASE
          MOVE SPACES TO WS-NUMCPT-SAVED
        WHEN DFHRESP(NOTFND)
          MOVE LOW-VALUES TO MAPSUPO
          MOVE 'ERREUR : CLIENT DEJA SUPPRIME' TO MSGO
          MOVE '1' TO WS-PHASE
          MOVE SPACES TO WS-NUMCPT-SAVED
        WHEN OTHER
          MOVE LOW-VALUES TO MAPSUPO
          MOVE WS-NUMCPT-SAVED TO NUMCPTO
          MOVE DFHBMASK TO NUMCPTA
          MOVE 'ERREUR SUPPRESSION - CONTACTEZ SUPPORT' TO MSGO
      END-EVALUATE

      EXEC CICS SEND MAP('MAPSUP')
            MAPSET('CLISUP')
      ERASE
      END-EXEC.

4100-FIN.
EXIT.

```

JCL de compilation : CMPSUP.jcl

Le JCL de compilation suit la même structure que CMPCLAF.jcl (voir Partie 1, Exercice 3). Seuls le nom du job (ROCHA13) et le membre source (PRGSUP) changent.

Structure du programme

Paragraphe	Fonction
0000-PRINCIPAL	Point d'entrée, aiguillage pseudo-conversationnel
1000-INIT-RECHERCHE	Affichage écran vide
2000-TRAITEMENT	Aiguillage selon la phase
3000-RECHERCHER-CLIENT	Phase 1 : Saisie et lecture du client
3100-AFFICHER-CLIENT	Affichage des données avec libellés
4000-CONFIRMER-SUPPRESSION	Phase 2 : Réception confirmation O/N
4100-SUPPRIMER-CLIENT	Exécution de la commande DELETE
9000-FIN-PROGRAMME	Fin de transaction (PF3)

Commandes CICS utilisées

Commande	Usage
SEND MAP	Envoyer l'écran (avec ERASE)
RECEIVE MAP	Recevoir la saisie avec RESP pour MAPFAIL
READ FILE	Vérifier existence et afficher les données
DELETE FILE	Supprimer l'enregistrement (sans READ UPDATE)
RETURN TRANSID	Retour pseudo-conversationnel

Messages d'erreur gérés

Message	Contexte
SAISIR LE NUMERO DE COMPTE A SUPPRIMER	Premier passage
VEUILLEZ SAISIR UN NUMERO DE COMPTE	MAPFAIL ou champ vide
NUMERO DE COMPTE DOIT ETRE NUMERIQUE	Caractères non numériques
CLIENT INEXISTANT - VERIFIEZ LE NUMERO	READ retourne NOTFND
CLIENT TROUVE - CONFIRMER SUPPRESSION (O/N) ?	Client affiché, attente confirmation
VEUILLEZ REpondre O OU N	MAPFAIL en phase confirmation
REPONSE INVALIDE - SAISIR O OU N	Confirmation différente de O/N
SUPPRESSION ANNULEE - NOUVEAU NUMERO OU PF3	Utilisateur a saisi N
CLIENT SUPPRIME - NOUVEAU NUMERO OU PF3	DELETE réussi
ERREUR : CLIENT DEJA SUPPRIME	DELETE retourne NOTFND

Définition CICS

```
CEDA DEFINE PROGRAM(PRGSUP) GROUP(CLIGROUP) LANGUAGE(COBOL)
CEDA INSTALL PROGRAM(PRGSUP) GROUP(CLIGROUP)
```

Vérification

```
CEMT INQ PROGRAM(PRGSUP)
```

Résultat attendu : **Prog(PRGSUP) Cob Ena**

Points importants

1. **DELETE sans READ UPDATE** : Contrairement à REWRITE, la commande DELETE n'a pas besoin de verrouillage préalable. Elle supprime directement par la clé.
2. **Confirmation obligatoire** : L'utilisateur doit explicitement répondre O ou N. Toute autre réponse est rejetée.
3. **Acceptation majuscules/minuscules** : Le programme accepte O/o et N/n pour plus de convivialité.
4. **PERFORM THRU avec GO TO** : Comme pour les autres programmes (voir Partie 2a, Exercice 7), la clause THRU permet aux GO TO de rester dans la plage du PERFORM.

Difficultés rencontrées et solutions

Le programme PRGSUP a bénéficié des leçons apprises lors du développement des programmes précédents. Les difficultés suivantes ont été anticipées et évitées :

Problème potentiel	Prévention appliquée
GO TO hors plage PERFORM	Utilisation systématique de PERFORM ... THRU paragraphe-FIN
Écrasement données après RECEIVE	Sauvegarde immédiate dans WS-NUMCPT-SAVED et WS-CONFIRM
Message non visible	ERASE systématique sur tous les SEND MAP
Données non transmises (NUMCPT protégé)	Sauvegarde du numéro dans COMMAREA (WS-NUMCPT-SAVED)

Note : L'approche avec confirmation visuelle (READ + affichage + DELETE) évite les suppressions accidentelles, contrairement à un DELETE direct qui aurait été techniquement plus simple mais risqué.

Captures d'écran

Définition du programme PRGSUP dans CICS

Après la compilation COBOL réussie, on définit le programme dans CICS.

```

DEFINE PROGRAM(PRGSUP) GROUP(CLIGROUP) LANGUAGE(COBOL)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEfIne PROGram( PRGSUP )
  PROGram      : PRGSUP
  Group        : CLIGROUP
  DEScription  ==> _
  Language     ==> CObol          CObol ! Assembler ! Le370 ! C ! PlI
  RELoad       ==> No             No ! Yes
  RESident     ==> No             No ! Yes
  USAge        ==> Normal         Normal ! Transient
  USElpacopy   ==> No             No ! Yes
  Status       ==> Enabled        Enabled ! Disabled
  RSL          : 00              0-24 ! Public
  CEdf         ==> Yes            Yes ! No
  DATalocation ==> Below          Below ! Any
  EXECKey      ==> User           User ! Cics
  COncurrency  ==> Quasirent      Quasirent ! Threadsafe ! Required
  Api          ==> Cicsapi        Cicsapi ! Openapi
  REMOTE ATTRIBUTES
+  DYnamic      ==> No            No ! Yes

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                          TIME: 17.11.17 DATE: 01/15/26

```

La commande `CEDA DEFINE PROGRAM(PRGSUP) GROUP(CLIGROUP) LANGUAGE(COBOL)` crée la définition du programme. Le message "DEFINE SUCCESSFUL" confirme la création.

Installation du programme PRGSUP

```

INSTALL PROGRAM(PRGSUP) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  J0urnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSRpool     ==>
+  MAPset      ==>

                                           SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL                          TIME: 17.12.03 DATE: 01/15/26
PF 1 HELP          3 END          6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA INSTALL PROGRAM(PRGSUP)` charge le programme compilé en mémoire CICS. Le message "INSTALL SUCCESSFUL" indique que le programme est prêt.

Vérification avec CEMT

```
INQUIRE PROGRAM(PRGSUP)
STATUS: RESULTS - OVERTYPE TO MODIFY
Prog(PRGSUP ) Leng(0000000000) Cob Pro Ena Pri      Ced
      Res(0000) Use(0000000000) Bel Uex Ful Qua Cic
```

CEMT INQ PROGRAM(PRGSUP) affiche le statut du programme : "Cob" (COBOL), "Pro" (Protected), "Ena" (Enabled). Le programme est correctement installé et activé.

Compilation du programme PRGSUP

```
* Statistics for COBOL program PRGSUP:
*   Source records = 996
*   Data Division statements = 339
*   Procedure Division statements = 183
End of compilation 1, program PRGSUP, no statements flagged.
Return code 0
***** BOTTOM OF DATA *****
```

Statistiques de compilation du programme PRGSUP : 996 enregistrements sources, 339 instructions DATA DIVISION, 183 instructions PROCEDURE DIVISION. Return code 0 confirme la compilation réussie.

Exercice 14 : Transaction de suppression

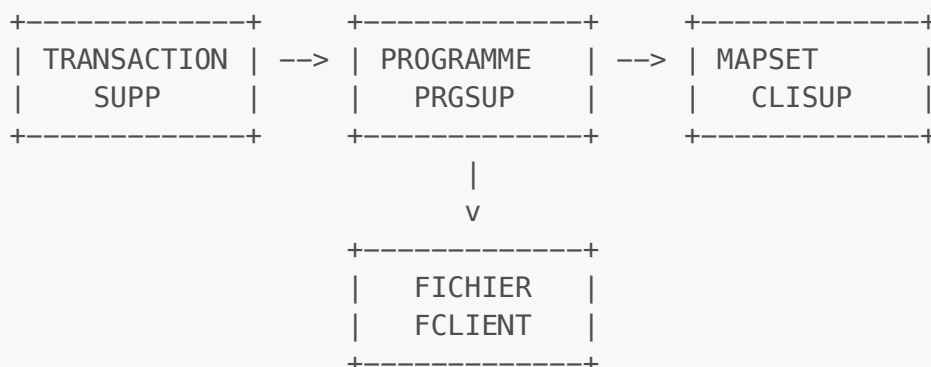
Énoncé

Définir une transaction indépendante des précédentes pour appeler le programme de suppression.

Mon travail

La transaction SUPP est le point d'entrée utilisateur pour la suppression de clients.

Architecture CICS - Liaison des ressources



Résolution

Définition de la transaction :

```
CEDA DEFINE TRANSACTION(SUPP) GROUP(CLIGROUP) PROGRAM(PRGSUP)
```

Paramètre	Valeur	Description
TRANSACTION	SUPP	Code transaction (4 caractères max)
GROUP	CLIGROUP	Groupe de ressources du projet
PROGRAM	PRGSUP	Programme COBOL à exécuter

Installation de la transaction :

```
CEDA INSTALL TRANSACTION(SUPP) GROUP(CLIGROUP)
```

Bonne pratique : Installer uniquement la ressource ajoutée plutôt que tout le groupe. Réinstaller le groupe peut causer des problèmes si FCLIENT est ouvert.

Vérification

```
CEDA VIEW TRANSACTION(SUPP) GROUP(CLIGROUP)  
CEMT INQ TRAN(SUPP)
```

Résultat attendu : **Tra(SUPP) Pro(PRGSUP) Ena**

Test de la transaction

Test sans debugger :

```
SUPP
```

Comportement attendu :

1. Écran de saisie du numéro de compte
2. Saisir un numéro existant (ex: 100005)
3. Affichage des données du client avec demande de confirmation
4. Saisir O pour confirmer ou N pour annuler
5. Si O : Message "CLIENT SUPPRIME"
6. Si N : Message "SUPPRESSION ANNULEE"

Test avec CEDF (voir Partie 1, Exercice 5 pour la navigation CEDF) :

```
CEDF
```

SUPP

Points d'arrêt observés :

Étape	Commande CICS	RESP attendu	Description
1	SEND MAP	NORMAL	Affichage écran recherche
2	RETURN TRANSID	-	Fin phase 1
3	RECEIVE MAP	NORMAL	Réception numéro
4	READ FILE	NORMAL	Lecture client
5	SEND MAP	NORMAL	Affichage pour confirmation
6	RETURN TRANSID	-	Fin phase 1bis
7	RECEIVE MAP	NORMAL	Réception confirmation
8	DELETE FILE	NORMAL	Suppression
9	SEND MAP	NORMAL	Message succès

Ressources du groupe CLIGROUP après exercice 14

Type	Nom	Description	Défini dans
FILE	FCLIENT	Fichier VSAM clients	Exercice 1
MAPSET	CLIAFF	Écran affichage	Exercice 4
MAPSET	CLIAJT	Écran ajout	Exercice 8
MAPSET	CLIMAJ	Écran mise à jour	Exercice 9
MAPSET	CLISUP	Écran suppression	Exercice 12
PROGRAM	PRGCLIA	Programme affichage	Exercice 4
PROGRAM	PRGAJT	Programme ajout	Exercice 8
PROGRAM	PRGMAJ	Programme mise à jour	Exercice 10
PROGRAM	PRGSUP	Programme suppression	Exercice 13
TRANSACTION	AFFI	Transaction affichage	Exercice 4
TRANSACTION	AJOU	Transaction ajout	Exercice 8
TRANSACTION	MAJO	Transaction mise à jour	Exercice 11
TRANSACTION	SUPP	Transaction suppression	Exercice 14

Captures d'écran

Définition de la transaction SUPP

La transaction fait le lien entre le code utilisateur et le programme COBOL.

```

DEFINE TRANSACTION(SUPP) GROUP(CLIGROUP) PROGRAM(PRGSUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA Define TRANsAction( SUPP )
  TRANsAction      : SUPP
  Group            : CLIGROUP
  DEscription      ==> _
  PROGram          ==> PRGSUP
  TWasize          ==> 00000                0-32767
  PROFile          ==> DFHCICST
  PARTitionset     ==>
  STatus           ==> Enabled              Enabled ! Disabled
  PRIMedsize       : 00000                0-65520
  TASKDATAloc     ==> Below                Below ! Any
  TASKDATAkey      ==> User                 User ! Cics
  STOrageclear     ==> No                   No ! Yes
  RUNaway          ==> System               System ! 0 ! 500-2700000
  SHutdown         ==> Disabled             Disabled ! Enabled
  ISolate          ==> Yes                  Yes ! No
  Brexit           ==>
+ REMOTE ATTRIBUTES

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                         TIME: 17.18.45  DATE: 01/15/26

```

La commande `CEDA DEFINE TRANSACTION(SUPP) GROUP(CLIGROUP) PROGRAM(PRGSUP)` associe le code "SUPP" au programme PRGSUP. Le message "DEFINE SUCCESSFUL" confirme la création.

Installation de la transaction SUPP

```

INSTALL TRANSACTION(SUPP) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice      ==>
  Bundle           ==>
  CONnection       ==>
  CORbaserver      ==>
  DB2Conn          ==>
  DB2Entry         ==>
  DB2Tran          ==>
  DJar             ==>
  D0ctemplate      ==>
  Enqmodel         ==>
  File             ==>
  Ipconn           ==>
  J0urnalmodel     ==>
  JVmserver        ==>
  LIBrary          ==>
  LSRpool          ==>
+ MAPset           ==>

                                           SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL                         TIME: 17.20.12  DATE: 01/15/26

```

La commande `CEDA INSTALL TRANSACTION(SUPP)` rend la transaction accessible aux utilisateurs. Le message "INSTALL SUCCESSFUL" confirme l'activation.

Test fonctionnel - Écran de suppression vide

Après avoir tapé "SUPP" sur l'écran CICS, l'écran de saisie s'affiche.

```
*** SUPPRESSION CLIENT ***
-----
NUMERO COMPTE :
CODE REGION   :
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:
SEXE          :
ACTIVITE PRO  :
SITUATION SOC :
ADRESSE       :
SOLDE         :
POSITION      :

CONFIRMER SUPPRESSION (O/N) :

-----
MESSAGE : SAISIR LE NUMERO DE COMPTE A SUPPRIMER

ENTER=RECHERCHER/CONFIRMER  PF3=QUITTER  CLEAR=EFFACER
```

Phase 1 : L'écran de suppression s'affiche vide avec le message "SAISIR LE NUMERO DE COMPTE A SUPPRIMER". L'utilisateur doit saisir un numéro de compte existant.

Session de débogage CEDF - Suppression complète

Le débogueur CEDF permet de suivre l'exécution des commandes CICS pas à pas lors d'une suppression.

CEDF - RECEIVE MAP (réception du numéro)

```
TRANSACTION: SUPP PROGRAM: PRGSUP TASK: 0000254 APPLID: CICSTS51 DISPLAY: 00
- STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS RECEIVE MAP
  MAP ('MAPSUP ')
  INTO ('.....6.....333333.....'....)
  MAPSET ('CLISUP ')
  TERMINAL
  NOHANDLE

OFFSET:X'000CCA' LINE:00170 EIBFN=X'1802'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur la commande RECEIVE MAP : réception du numéro de compte 333333 saisi par l'utilisateur. RESPONSE: NORMAL.

CEDF - READ FILE (lecture du client)

```
TRANSACTION: SUPP PROGRAM: PRGSUP TASK: 0000254 APPLID: CICSTS51 DISPLAY: 00
- STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS READ
  FILE ('FCLIENT ')
  INTO ('3333330210GIL GILBERTO 19851212M10VBRESIL 8888888888CR'...)
  LENGTH (80)
  RIDFLD ('333333')
  EQUAL
  NOHANDLE

OFFSET:X'000EE8' LINE:00213 EIBFN=X'0602'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur la commande READ FILE avec RIDFLD('333333'). Le client GIL GILBERTO est trouvé (données visibles : 19851212M10VBRESIL 8888888888CR). RESPONSE: NORMAL.

Écran - Client trouvé, demande de confirmation

```
*** SUPPRESSION CLIENT ***

-----

NUMERO COMPTE : 333333

CODE REGION   : 02                MARSEILLE
NATURE COMPTE : 10                AUTRE
NOM           : GIL
PRENOM        : GILBERTO
DATE NAISSANCE : 19851212
SEXE          : M                MASCULIN
ACTIVITE PRO  : 10
SITUATION SOC : V                VEUF (VE)
ADRESSE       : BRESIL
SOLDE         : 8888888888
POSITION      : CR                CREDITEUR

CONFIRMER SUPPRESSION (O/N) :

-----

MESSAGE : CLIENT TROUVE - CONFIRMER SUPPRESSION (O/N) ?

ENTER=RECHERCHER/CONFIRMER  PF3=QUITTER  CLEAR=EFFACER
```

L'écran affiche les données complètes du client 333333 (GIL GILBERTO, MARSEILLE, VEUF, CREDITEUR). Le message "CLIENT TROUVE - CONFIRMER SUPPRESSION (O/N) ?" invite l'utilisateur à confirmer ou annuler.

CEDF - SEND MAP (affichage pour confirmation)

```
TRANSACTION: SUPP PROGRAM: PRGSUP TASK: 0000254 APPLID: CICSTS51 DISPLAY: 00
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS SEND MAP
  MAP ('MAPSUP ')
  FROM ('.....6....0333333...02...MARSEILLE      ...10...AUTRE      '...)
  LENGTH (258)
  MAPSET ('CLISUP ')
  TERMINAL
  ERASE

OFFSET:X'001290'      LINE:00337      EIBFN=X'1804'
RESPONSE: NORMAL      EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED      PF2 : SWITCH HEX/CHAR      PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS      PF5 : WORKING STORAGE      PF6 : USER DISPLAY
PF7 : SCROLL BACK      PF8 : SCROLL FORWARD      PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY      PF11: EIB DISPLAY      PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur la commande SEND MAP : envoi de l'écran avec les données du client (333333, 02, MARSEILLE...). RESPONSE: NORMAL.

CEDF - DELETE FILE (suppression VSAM)

```
TRANSACTION: SUPP PROGRAM: PRGSUP TASK: 0000265 APPLID: CICSTS51 DISPLAY: 00
- STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS DELETE FILE
  FILE ('FCLIENT ')
  RIDFLD ('333333')
  NOHANDLE

OFFSET:X'00156A' LINE:00409 EIBFN=X'0608'
RESPONSE: NORMAL EIBRESP=0

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: ABEND USER TASK
```

Point d'arrêt CEDF sur la commande DELETE FILE avec RIDFLD('333333'). **RESPONSE: NORMAL** confirme que l'enregistrement a été supprimé du fichier VSAM.

Écran - Suppression effectuée

```
*** SUPPRESSION CLIENT ***
-----
NUMERO COMPTE : _
CODE REGION :
NATURE COMPTE :
NOM :
PRENOM :
DATE NAISSANCE:
SEXE :
ACTIVITE PRO :
SITUATION SOC :
ADRESSE :
SOLDE :
POSITION :

CONFIRMER SUPPRESSION (O/N) :
-----
MESSAGE : CLIENT SUPPRIME - NOUVEAU NUM OU PF3
ENTER=RECHERCHER/CONFIRMER PF3=QUITTER CLEAR=EFFACER
```


Message "CLIENT SUPPRIME - NOUVEAU NUM OU PF3" confirmant le succès de l'opération. L'écran est réinitialisé pour permettre une nouvelle suppression.

Test d'erreur - Client déjà supprimé

```
*** SUPPRESSION CLIENT ***
-----
NUMERO COMPTE :  _
CODE REGION   :
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:
SEXE          :
ACTIVITE PRO  :
SITUATION SOC :
ADRESSE       :
SOLDE         :
POSITION      :

CONFIRMER SUPPRESSION (O/N) :

-----
MESSAGE : CLIENT INEXISTANT - VERIFIEZ LE NUMERO

ENTER=RECHERCHER/CONFIRMER  PF3=QUITTER  CLEAR=EFFACER
```

En tentant de supprimer à nouveau le client 333333, le programme affiche "CLIENT INEXISTANT - VERIFIEZ LE NUMERO" car l'enregistrement n'existe plus.

Vérification avec AFFI

```
*** AFFICHAGE CLIENT ***
-----
NUMERO COMPTE : 333333
CODE REGION   :
NATURE COMPTE :
NOM           :
PRENOM        :
DATE NAISSANCE:
SEXE          :
ACTIVITE PRO  :
SITUATION SOC :
ADRESSE       :
SOLDE        :
POSITION     :
-----
MESSAGE : CLIENT INEXISTANT - VERIFIEZ LE NUMERO
-----
ENTER=RECHERCHER  PF3=QUITTER  CLEAR=EFFACER
```

Vérification avec la transaction AFFI : le client 333333 n'existe plus. Le message "CLIENT INEXISTANT - VERIFIEZ LE NUMERO" confirme que la suppression a bien été effectuée.

Exercice 15 : Suppression avec lecture préalable

Énoncé

Reprendre cette opération de suppression en la précédant par une opération de lecture. Définir une transaction indépendante de la précédente.

Mon travail

Exercice déjà couvert : Le programme PRGSUP (exercice 13) implémente déjà la suppression avec lecture préalable. J'ai anticipé cette fonctionnalité en développant directement la version complète.

Pourquoi avoir anticipé ?

Le programme PRGSUP réalise exactement ce que demande l'exercice 15 :

1. **READ** pour vérifier l'existence et récupérer les données
2. **Affichage** des données du client pour confirmation visuelle
3. **Confirmation O/N** avant suppression
4. **DELETE** uniquement si l'utilisateur confirme

Comparaison : Ce qui était prévu vs ce qui a été fait

Élément	Prévu (Ex 13 + Ex 15)	Réalisé
Ex 13	DELETE direct (sans affichage)	DELETE avec READ + affichage
Ex 15	READ + DELETE (avec affichage)	Déjà couvert par Ex 13
Transaction SUPP	Programme simple	Programme complet
Transaction SULE	Programme avec lecture	Non nécessaire (alias possible)

Résolution

Option 1 : Ne rien faire - L'exercice est déjà couvert par PRGSUP.

Option 2 : Créer une transaction alias (optionnel)

Si on souhaite avoir les deux codes transaction (SUPP et SULE) pointant vers le même programme :

```
CEDA DEFINE TRANSACTION(SULE) GROUP(CLIGROUP) PROGRAM(PRGSUP)
CEDA INSTALL TRANSACTION(SULE) GROUP(CLIGROUP)
```

Cela permet d'utiliser indifféremment **SUPP** ou **SULE** pour accéder à la suppression avec confirmation visuelle.

Conclusion

En implémentant directement la version sécurisée (avec lecture préalable) dans l'exercice 13, j'ai :

- Appliqué les bonnes pratiques de développement mainframe
- Évité la création d'un programme moins sécurisé (DELETE sans vérification)
- Couvert les objectifs des exercices 13 et 15 en une seule implémentation

Note : Toutes les captures d'écran nécessaires sont présentées dans les exercices 13 et 14.

Partie 3 : Opérations Avancées

Cette section couvre les exercices 16 à 19 : création de clients génériques, navigation VSAM avec STARTBR/READNEXT/ENDBR, et statistiques par région avec AIX/PATH.

Au-delà du CRUD : La navigation séquentielle

Dans les parties précédentes, nous avons maîtrisé les quatre opérations CRUD sur un enregistrement à la fois :

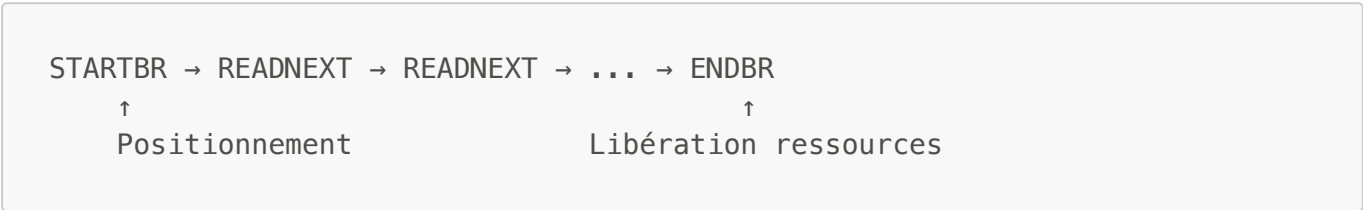
- **Partie 1** : READ (lecture par clé exacte)
- **Partie 2a** : WRITE (ajout)
- **Partie 2b** : REWRITE (mise à jour)
- **Partie 2c** : DELETE (suppression)

Cette partie introduit une nouvelle dimension : **le parcours séquentiel** d'un fichier VSAM. Au lieu de travailler sur un enregistrement spécifique, on parcourt plusieurs enregistrements correspondant à un critère.

Commandes CICS pour la navigation VSAM

Commande	Usage	Comparaison avec READ
STARTBR	Positionner le curseur de browse	Comme READ mais sans récupérer les données
READNEXT	Lire l'enregistrement suivant	Avance automatiquement dans le fichier
READPREV	Lire l'enregistrement précédent	Navigation arrière
ENDBR	Terminer le parcours	Libère les ressources (obligatoire !)

Cycle de vie d'un browse :



Point clé : Contrairement à READ qui travaille sur un enregistrement unique, le browse permet de traiter **tous les enregistrements** correspondant à un préfixe de clé. C'est indispensable pour des opérations comme "supprimer tous les clients 111xxx" ou "lister tous les clients d'une région".

Exercice 16 : Création de clients génériques

Énoncé

Sachant que le CODE CLIENT est sur six caractères, créer cinq CLIENT avec une partie de leur code générique commençant par '111...', de même '444...' et '777...'.

Mon choix de conception

L'énoncé demande de créer manuellement des clients génériques. J'ai fait le choix de **préparer ces données dès le début du projet** pour plusieurs raisons :

Ce qui était prévu	Ce que j'ai implémenté	Justification
Création manuelle ici	Données pré-chargées (LOADVSAM.jcl)	Gain de temps pour les tests
Uniquement 111xxx, 444xxx, 777xxx	Également 222xxx	Plus de variété pour les démonstrations
Via ISPF ou transaction	Via JCL initial + AJOU	Traçabilité et reproductibilité

Anticipation : En préparant les données de test dès la Partie 0, j'ai pu tester immédiatement les transactions AFFI, AJOU, MAJO et SUPP avec des clients "prêts à l'emploi".

Mon travail

J'ai anticipé cet exercice lors des phases précédentes du projet.

Pourquoi des clients génériques ?

Les clients avec des préfixes communs (111xxx, 222xxx, etc.) sont nécessaires pour tester les commandes de navigation VSAM :

- **STARTBR** avec une clé partielle (ex: 111) se positionne sur le premier client correspondant
- **READNEXT** lit séquentiellement tous les clients 111xxx jusqu'à rencontrer une clé différente

Résolution

Clients pré-chargés via LOADVSAM.jcl (voir Partie 1, Exercice 1) :

Numéro	Nom	Région	Position
222001	LEROY Michel	Paris	CR
222002	ROUX Nathalie	Marseille	DB
222003	DAVID François	Lyon	CR
222004	BERTRAND Isabelle	Lille	DB
222005	MOREL Philippe	Paris	CR

Clients créés via AJOU (exercices 7-8) :

Plusieurs clients 111xxx ont été créés lors des tests de la transaction d'ajout.

Création supplémentaire (optionnel) :

Pour créer les clients 444xxx et 777xxx, utiliser la transaction AJOU :

AJOU

- Saisir numéro 444001, remplir les champs, valider
- Répéter pour 444002, 444003, etc.

Vérification

AFFI

- Saisir 222001 → Client affiché
- Saisir 111001 → Client affiché (si créé)

Captures d'écran

Vérification des clients génériques avec DITTO/ESA

Avant de tester les fonctionnalités de navigation VSAM, on vérifie la présence des clients génériques dans le fichier.

The screenshot shows the 'DITTO/ESA for MVS' interface in 'VB - VSAM Browse' mode. It displays a catalog of clients stored in the dataset 'ROCHA.CICS.CLIENT'. The catalog lists records with their Relative Block Address (RBA), length (Len), key, and data. The data field contains a combination of generic client codes (111xxx) and test strings (TEST, STRASBOURG, CAAAAAAAAA, EEEEEEEEEE, RRRRRRRRRR). The interface includes navigation controls like 'Command ===>' and 'Scroll PAGE'.

RBA	Len	Key	DSNAME	Format
880	80	111120110TEST	ROCHA.CICS.CLIENT	99999999
960	80	111130110TEST	ROCHA.CICS.CLIENT	99999999
1040	80	111140110TEST	ROCHA.CICS.CLIENT	99999999
1120	80	111150110AAAAAAAAAAAAAAAAAAAA	ROCHA.CICS.CLIENT	99999999
1200	80	111160110TEST	ROCHA.CICS.CLIENT	99999999
1280	80	111170110ZZZZZZZZZZZZZZZZZZZZ	ROCHA.CICS.CLIENT	99999999
1360	80	111180110EEEEEEEEEEEEEEEEEEEE	ROCHA.CICS.CLIENT	99999999
1440	80	111190110EEEEEEEEEEEEEEEEEEEE	ROCHA.CICS.CLIENT	99999999
1520	80	111220110TEST	ROCHA.CICS.CLIENT	99999999
1600	80	111770110RRRRRRRRRRRRRRRRRRR	ROCHA.CICS.CLIENT	99999999
1680	80	111880110TEST	ROCHA.CICS.CLIENT	99999999

L'utilitaire DITTO/ESA en mode VSAM Browse montre le contenu du fichier ROCHA.CICS.CLIENT. On voit les clients avec des préfixes génériques (111xxx) créés via la transaction AJOU lors des exercices précédents.

Exercice 17 : Suppression par code générique (STARTBR)

Énoncé

En utilisant les commandes adéquates, supprimer les CLIENT dont le code générique est '111...!.

Mon choix de conception

L'énoncé attendait probablement l'utilisation de la commande `DELETE ... GENERIC KEYLENGTH(...)` native de CICS, où l'utilisateur saisit **la clé ET sa longueur** :

```
*      Solution attendue (DELETE GENERIC natif)
EXEC CICS DELETE FILE('FCLIENT')
        RIDFLD(W$-REC-KEY) KEYLENGTH(W$-KEY-LEN)
        GENERIC NUMREC(W$-DEL-REC)
END-EXEC
```

J'ai fait le choix d'une **approche différente** pour améliorer l'ergonomie :

Approche attendue	Mon approche	Différence
Saisie clé + longueur	Saisie clé uniquement	Longueur calculée automatiquement
DELETE GENERIC natif	STARTBR/READNEXT + DELETE	Plus de contrôle sur le processus
Suppression immédiate	Comptage + confirmation	Sécurité des données

Pourquoi ce choix ? Demander à l'utilisateur de saisir la longueur de la clé est source d'erreurs. En calculant automatiquement la longueur à partir des caractères saisis (espaces ignorés), l'interface est plus intuitive. De plus, le comptage préalable et la confirmation évitent les suppressions accidentelles.

Mon travail

J'ai implémenté une alternative au DELETE GENERIC natif avec :

- **Calcul automatique de la longueur** : l'utilisateur saisit juste le préfixe
- **Phase de comptage** : affiche le nombre de clients avant suppression
- **Confirmation obligatoire** : l'utilisateur doit valider avec O/N

Pourquoi ne pas faire DELETE pendant le browse ?

C'est un point technique crucial. On ne peut **pas** faire `DELETE RIDFLD` pendant un browse actif (STARTBR/READNEXT) car cela provoque un **deadlock** :

- Le browse tient un verrou lecture sur le fichier
- Le DELETE demande un verrou exclusif sur le même enregistrement
- CICS freeze

Solution adoptée : Collecte puis suppression

La technique consiste à séparer le browse de la suppression en deux étapes distinctes :

ÉTAPE 1 : COLLECTE (pendant le browse)

```
STARTBR → READNEXT → stocker clé en table → READNEXT → ...
```

Table WS-CLES : [111001] [111002] [111003] ... (max 100)

ENDBR ← Fermeture OBLIGATOIRE avant les DELETE

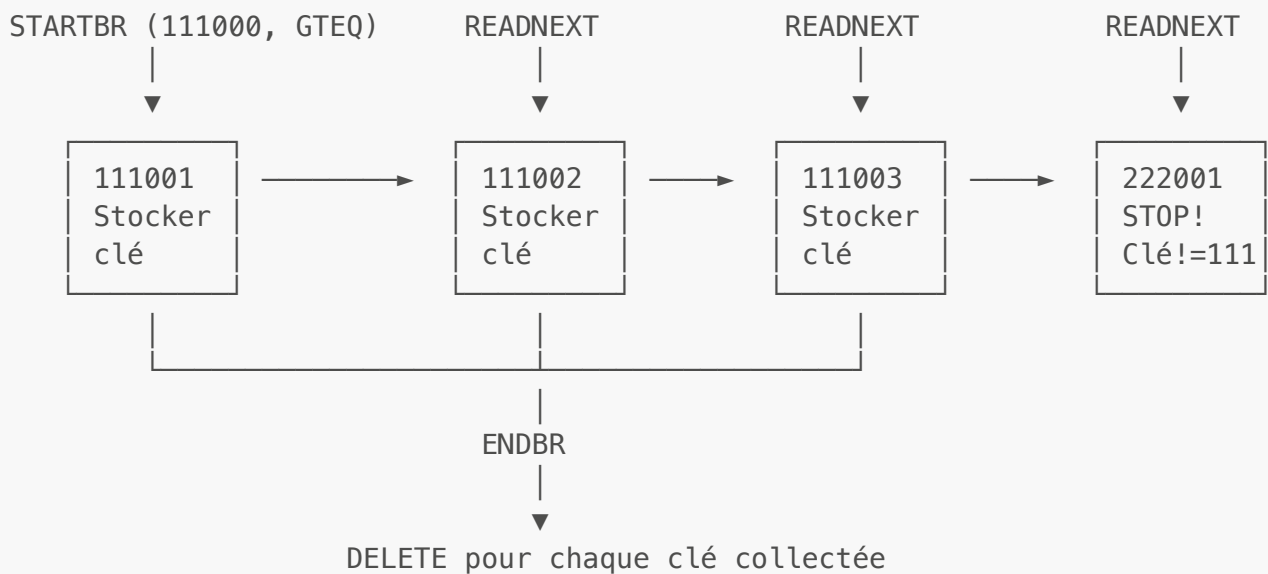
ÉTAPE 2 : SUPPRESSION (après le browse)

Pour chaque clé dans la table :
DELETE FILE('FCLIENT') RIDFLD(clé)

Plus de conflit : le browse est fermé, DELETE a le champ libre

Limite : La table est dimensionnée à 100 entrées. Si plus de 100 clients correspondent au préfixe, l'utilisateur doit relancer la transaction pour supprimer le reste.

Principe de la navigation VSAM pour suppression générique



Mode pseudo-conversationnel à 2 phases

PHASE 1 : COMPTAGE

1. Saisie préfixe (1-5 car) ou clé complète (6 car)
2. STARTBR/READNEXT pour compter les clients
3. Affichage : "X client(s) trouvé(s)"



PHASE 2 : CONFIRMATION ET SUPPRESSION

4. L'utilisateur répond 0 ou N
5. Si N : Retour phase 1
6. Si 0 : Suppression en 2 étapes (évite deadlock)
 - a) STARTBR/READNEXT → collecter clés en table (max 100)
 - b) ENDBR (fermer browse)
 - c) Pour chaque clé : DELETE RIDFLD

Résolution

MAP BMS : CLIDEL.bms

Le code source est stocké dans `ROCHA.CICS.SOURCE(CLIDEL)`.

En-tête du MAPSET avec commentaires :

```
*****
* MAPSET : CLIDEL – Suppression Generique Client
* Transaction : DELG
* Fil Rouge CICS – Exercice 17
*
* PARTICULARITE :
* -----
* Permet la suppression par prefixe (1 a 5 car) ou cle complete (6 car)
* - Prefixe : Supprime tous les clients correspondants
* - Cle complete : Supprime un seul client
*
* Le champ PREFIXE est en PIC X (pas NUM) pour eviter la
* justification a droite des valeurs numeriques.
*****
CLIDEL  DFHMSD TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,          X
        STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
```

Champs de saisie et d'affichage :

```
*-----
* ZONE DE SAISIE – PREFIXE OU CLE COMPLETE
*-----
PREFIXE  DFHMD F POS=(5,28),LENGTH=6,ATTRB=(UNPROT,IC)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        PIC X (pas NUM) : évite la justification à droite

*-----
* ZONE D'INFORMATION – NOMBRE DE CLIENTS
*-----
NBCLI    DFHMD F POS=(8,28),LENGTH=5,ATTRB=(ASKIP,BRT)
        ^^^^^         ^^^^^^^^^^^^^^^^^
        Résultat du comptage      ASKIP,BRT : lecture seule,
surbrillance
```

```

*-----
* ZONE DE CONFIRMATION
*-----
CONFIRM  DFHMD F POS=(10,28),LENGTH=1,ATTRB=UNPROT
          ^^^^^^                               ^^^^^^^^^^
          0 ou N                               Saisissable

```

Programme : PRGDELG.cbl - Extraits clés

Table de stockage des clés (Working-Storage) :

```

*-----
* TABLE DES CLES A SUPPRIMER (max 100 clients)
*-----
01  WS-TABLE-CLES.
    05 WS-NB-CLES          PIC 9(03) VALUE 0.
    05 WS-CLES OCCURS 100 TIMES.
        10 WS-CLE-SUP      PIC X(06).
01  WS-IDX-SUP            PIC 9(03) VALUE 0.

```

Paragraphe de parcours pour comptage :

```

3100-PARCOURIR-FICHER.
*-----
* Parcours du fichier pour compter les clients correspondants
*-----
    MOVE 0 TO WS-COMPTEUR
    MOVE 'N' TO WS-FIN-BROWSE

*   Construction de la clé de début (préfixe)
    MOVE SPACES TO WS-CLE-DEBUT
    MOVE WS-PREFIXE(1:WS-LONGUEUR) TO WS-CLE-DEBUT

*   Positionnement sur le premier client >= préfixe
    EXEC CICS STARTBR
        FILE('FCLIENT')
        RIDFLD(WS-CLE-DEBUT)
        GTEQ
        RESP(WS-RESP)
    END-EXEC

*   Boucle de lecture
    PERFORM UNTIL FIN-BROWSE
        EXEC CICS READNEXT
            FILE('FCLIENT')
            INTO(ENR-CLIENT)
            RIDFLD(WS-CLE-COURANTE)
            RESP(WS-RESP)
    END-EXEC

```

```

        EVALUATE TRUE
            WHEN WS-RESP = DFHRESP(ENDFILE)
                MOVE '0' TO WS-FIN-BROWSE
            WHEN WS-CLE-COURANTE(1:WS-LONGUEUR) NOT =
                WS-PREFIXE(1:WS-LONGUEUR)
*              Clé ne correspond plus au préfixe
                MOVE '0' TO WS-FIN-BROWSE
            WHEN OTHER
*              Client correspondant trouvé
                ADD 1 TO WS-COMPTEUR
        END-EVALUATE
    END-PERFORM

*    Fermeture du browse
    EXEC CICS ENDBR FILE('FCLIENT') END-EXEC.

```

Paragraphe de suppression en 2 étapes :

```

4100-SUPPRIMER-CLIENTS.
*-----
* Suppression en 2 étapes pour éviter le deadlock
* Étape 1 : Collecter les clés pendant le browse
* Étape 2 : Supprimer après ENDBR
*-----
    MOVE 0 TO WS-NB-CLES

    EXEC CICS STARTBR
        FILE('FCLIENT')
        RIDFLD(WS-CLE-DEBUT)
        GTEQ
        RESP(WS-RESP)
    END-EXEC

*    ETAPE 1 : Collecter les clés (sans DELETE)
    PERFORM UNTIL FIN-BROWSE
        EXEC CICS READNEXT ... END-EXEC

        EVALUATE TRUE
            WHEN WS-RESP = DFHRESP(ENDFILE)
                MOVE '0' TO WS-FIN-BROWSE
            WHEN WS-CLE-COURANTE(1:WS-LONGUEUR) NOT =
                WS-PREFIXE(1:WS-LONGUEUR)
                MOVE '0' TO WS-FIN-BROWSE
            WHEN WS-NB-CLES >= 100
*              Table pleine
                MOVE '0' TO WS-FIN-BROWSE
            WHEN OTHER
*              Stocker la clé dans la table
                ADD 1 TO WS-NB-CLES
                MOVE WS-CLE-COURANTE TO WS-CLE-SUP(WS-NB-CLES)
        END-EVALUATE

```

```
END-PERFORM
```

```
* Fermer le browse AVANT les suppressions
EXEC CICS ENDBR FILE('FCLIENT') END-EXEC

* ETAPE 2 : Supprimer chaque clé collectée
PERFORM VARYING WS-IDX-SUP FROM 1 BY 1
  UNTIL WS-IDX-SUP > WS-NB-CLES
  MOVE WS-CLE-SUP(WS-IDX-SUP) TO WS-CLE-COURANTE
  EXEC CICS DELETE
    FILE('FCLIENT')
    RIDFLD(WS-CLE-COURANTE)
    RESP(WS-RESP)
  END-EXEC
  IF WS-RESP = DFHRESP(NORMAL)
    ADD 1 TO WS-COMPTEUR-SUP
  END-IF
END-PERFORM.
```

Définition CICS :

```
CEDA DEFINE MAPSET(CLIDEL) GROUP(CLIGROUP)
CEDA INSTALL MAPSET(CLIDEL)

CEDA DEFINE PROGRAM(PRGDELG) GROUP(CLIGROUP) LANGUAGE(COBOL)
CEDA INSTALL PROGRAM(PRGDELG)

CEDA DEFINE TRANSACTION(DELG) GROUP(CLIGROUP) PROGRAM(PRGDELG)
CEDA INSTALL TRANSACTION(DELG)
```

Note : L'installation individuelle de chaque ressource permet de vérifier leur bon fonctionnement au fur et à mesure. On peut ensuite utiliser **CEDA DISPLAY GROUP(CLIGROUP)** pour visualiser l'ensemble des ressources du groupe.

Points importants

1. **Champ PREFIXE en PIC X** : Défini sans attribut NUM pour éviter la justification à droite. Ainsi **1** reste **1_____** et non **____1**.
2. **Table limitée à 100 clés** : Si plus de 100 clients correspondent, l'utilisateur doit relancer la transaction pour supprimer le reste.
3. **GTEQ** : Greater Than or Equal. Le STARTBR se positionne sur le premier enregistrement dont la clé est **>=** au préfixe.

Difficultés rencontrées et solutions

Problème 1 : Deadlock lors de la suppression pendant le browse

Symptôme : Le programme se figeait (freeze CICS) lors de l'exécution du DELETE pendant le parcours STARTBR/READNEXT.

Cause : Le browse tient un verrou de lecture sur le fichier. La commande DELETE demande un verrou exclusif sur le même enregistrement. CICS détecte un deadlock et freeze la transaction.

Solution : Implémenter une suppression en **deux phases** :

```
*   ETAPE 1 : Collecter les clés (pendant le browse)
      PERFORM UNTIL FIN-BROWSE
          EXEC CICS READNEXT ... END-EXEC
          ADD 1 TO WS-NB-CLES
          MOVE WS-CLE-COURANTE TO WS-CLE-SUP(WS-NB-CLES)
      END-PERFORM

*   Fermer le browse AVANT les suppressions
      EXEC CICS ENDBR FILE('FCLIENT') END-EXEC

*   ETAPE 2 : Supprimer chaque clé collectée
      PERFORM VARYING WS-IDX-SUP FROM 1 BY 1
          UNTIL WS-IDX-SUP > WS-NB-CLES
          EXEC CICS DELETE FILE('FCLIENT') RIDFLD(...) END-EXEC
      END-PERFORM
```

Problème 2 : Logique de browse incorrecte et position du curseur

Symptôme : Le programme comptait des enregistrements qui ne correspondaient pas au préfixe, ou le curseur ne se positionnait pas correctement après une recherche.

Cause : La condition d'arrêt du browse comparait mal les préfixes, et le SEND MAP ne repositionnait pas le curseur sur le champ de saisie.

Solution : Corriger la comparaison avec une référence modifiée et ajouter le positionnement curseur :

```
*   Comparaison correcte avec référence modifiée
      IF WS-CLE-COURANTE(1:WS-LONGUEUR) NOT =
          WS-PREFIXE(1:WS-LONGUEUR)
          MOVE '0' TO WS-FIN-BROWSE
      END-IF
```

Captures d'écran

Résultats des compilations

Assemblage BMS CLIDEL

```

      No Statements Flagged in this Assembly
HIGH LEVEL ASSEMBLER, 5696-234, RELEASE 6.0, PTF UK92594
SYSTEM: z/OS 01.13.00          JOBNAME: ROCHA09      STEPNAME: ASSEM      PRO
Data Sets Allocated for this Assembly
Con DDname   Data Set Name                               Volume  Member
P1 SYSIN     SYS26016.T11119.RA000.ROCHA09.TEMPM.H01
L1 SYSLIB    DFH510.CICS.SDFHMAC                          FDC511
L2           SYS1.MACLIB                                    FDRES1
      SYSPRINT IBMUSER.ROCHA09.JOB00170.D0000102.?
      SYSPUNCH SYS26016.T11119.RA000.ROCHA09.MAP.H01

      32100K allocated to Buffer Pool      Storage required      880K
      69 Primary Input Records Read      6922 Library Records Read
      0 ASMAOPT Records Read              497 Primary Print Records Written
      17 Object Records Written           0 ADATA Records Written
Assembly Start Time: 11.11.20 Stop Time: 11.11.20 Processor Time: 00.00.00.2434
Return Code 000
***** BOTTOM OF DATA *****

```

Le job ROCHA09 (assemblage BMS) retourne Return Code 000. On note 69 Primary Input Records Read et 17 Object Records Written, confirmant la génération correcte du mapset CLIDEL (plus léger que CLISUP car moins de champs).

Compilation du programme PRGDELG

```

* Statistics for COBOL program PRGDELG:
*   Source records = 1018
*   Data Division statements = 280
*   Procedure Division statements = 199
End of compilation 1, program PRGDELG, no statements flagged.
Return code 0
***** BOTTOM OF DATA *****

```

Statistiques de compilation du programme PRGDELG : 1018 enregistrements sources, 280 instructions DATA DIVISION, 199 instructions PROCEDURE DIVISION. Return code 0 confirme la compilation réussie.

Définition des ressources CICS pour DELG

La transaction de suppression générique nécessite trois ressources : MAPSET, PROGRAM et TRANSACTION.

```

DEFINE MAPSET(CLIDEL) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA  DEfIne MApset( CLIDEL  )
  MApset      : CLIDEL
  Group       : CLIGROUP
  DEscription ==> _
  REsident    ==> No                No ! Yes
  USAge       ==> Normal            Normal ! Transient
  USElpacopy  ==> No                No ! Yes
  Status      ==> Enabled            Enabled ! Disabled
  RSl         : 00                  0-24 ! Public
DEFINITION SIGNATURE
DEFInetime   : 01/16/26 11:12:15
CHANGETime   : 01/16/26 11:12:15
CHANGEUsrid  : CICSUSER
CHANGEAGent  : CSDApi              CSDApi ! CSDBatch
CHANGEAGRel  : 0680

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                         TIME: 11.12.15  DATE: 01/16/26

```

La commande `CEDA DEFINE MAPSET(CLIDEL) GROUP(CLIGROUP)` crée la définition du mapset de suppression générique. Le message "DEFINE SUCCESSFUL" confirme la création.

```

INSTALL MAPSET(CLIDEL) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA  Install
  AToMservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  DOctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+  MApset      ==> CLIDEL

                                           SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL                         TIME: 11.13.19  DATE: 01/16/26

```

La commande `CEDA INSTALL MAPSET(CLIDEL)` charge le mapset en mémoire CICS.

```

DEFINE PROGRAM(PRGDELG) GROUP(CLIGROUP) LANGUAGE(COBOL)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA Define PRoGram( PRGDELG )
  PRoGram      : PRGDELG
  Group        : CLIGROUP
  DEscription  ==>
  Language     ==> CObol          CObol ! Assembler ! Le370 ! C ! Pli
  REload       ==> No             No ! Yes
  RESident     ==> No             No ! Yes
  USAge        ==> Normal         Normal ! Transient
  USElpacopy   ==> No             No ! Yes
  Status       ==> Enabled        Enabled ! Disabled
  RSl          : 00              0-24 ! Public
  CEdf         ==> Yes            Yes ! No
  DAlocation   ==> Below          Below ! Any
  EXEKey       ==> User           User ! Cics
  COncurrency  ==> Quasirent      Quasirent ! Threadsafe ! Required
  Api          ==> Cicsapi        Cicsapi ! Openapi
  REMOTE ATTRIBUTES
+  DYnamic     ==> No             No ! Yes

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                        TIME: 11.56.18  DATE: 01/16/26

```

La commande `CEDA DEFINE PROGRAM(PRGDELG) GROUP(CLIGROUP) LANGUAGE(COBOL)` crée la définition du programme de suppression générique.

```

INSTALL PROGRAM(PRGDELG) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  DOctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+  MApset     ==>

                                           SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL                        TIME: 11.57.04  DATE: 01/16/26

```

La commande `CEDA INSTALL PROGRAM(PRGDELG)` charge le programme en mémoire.


```

VIEW MAPSET(CLIDEL) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View Mapset( CLIDEL )
  Mapset      : CLIDEL
  Group       : CLIGROUP
  Description  :
  Resident    : No                No ! Yes
  USAge       : Normal            Normal ! Transient
  USElpacopy   : No                No ! Yes
  Status      : Enabled            Enabled ! Disabled
  RSL         : 00                0-24 ! Public
DEFINITION SIGNATURE
  DEFInetime   : 01/16/26 11:12:15
  CHANGETime   : 01/16/26 11:12:15
  CHANGEUsrid  : CICSUSER
  CHANGEAGEnt  : CSDApi            CSDApi ! CSDBatch
  CHANGEAGRel  : 0680

```

SYSID=S670 APPLID=CICSTS51

CEDA VIEW permet de vérifier la définition du mapset CLIDEL.

```

INQUIRE PROGRAM(PRGDELG)
STATUS: RESULTS - OVERTYPE TO MODIFY
  Prog(PRGDELG ) Leng(0000000000) Cob Pro Ena Pri      Ced
      Res(0000) Use(0000000000) Bel Uex Ful Qua Cic

```

La commande CEDA DEFINE TRANSACTION(DELG) GROUP(CLIGROUP) PROGRAM(PRGDELG) associe le code "DELG" au programme PRGDELG.

```

DEFINE TRANSACTION(DELG) GROUP(CLIGROUP) PROGRAM(PRGDELG)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFine TRANSAction( DELG )
  TRANSAction  : DELG
  Group        : CLIGROUP
  Description   ==> _
  PROGram      ==> PRGDELG
  TWasize      ==> 00000                0-32767
  PROFile      ==> DFHCICST
  PartitioNset ==>
  STATus       ==> Enabled                Enabled ! Disabled
  PRIMedsize   : 00000                0-65520
  TASKDATAloc  ==> Below                Below ! Any
  TASKDATAKey  ==> User                  User ! Cics
  STOrageclear ==> No                    No ! Yes
  RUNaway      ==> System                System ! 0 ! 500-2700000
  SHutdown     ==> Disabled              Disabled ! Enabled
  ISolate      ==> Yes                   Yes ! No
  Brexit       ==>
+ REMOTE ATTRIBUTES

```

SYSID=S670 APPLID=CICSTS51
TIME: 12.00.01 DATE: 01/16/26

DEFINE SUCCESSFUL

La commande CEDA INSTALL TRANSACTION(DELG) rend la transaction accessible aux utilisateurs.

Vérification des ressources du groupe

```
INSTALL TRANSACTION(DELG) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  CORbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Enqmodel    ==>
  File        ==>
  Ipconn      ==>
  JOurnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+ MAPset      ==>

                                SYSID=S670 APPLID=CICSTS51
                                TIME: 12.00.46 DATE: 01/16/26
INSTALL SUCCESSFUL
```

CEDA DISPLAY GROUP(CLIGROUP) affiche toutes les ressources définies dans le groupe.

```
*** SUPPRESSION GENERIQUE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : _      (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS :

CONFIRMER (O/N)        :

-----

MESSAGE : SAISIR PREFIXE (1-5 CAR) OU CLE COMPLETE (6 CAR)

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

Suite de la liste des ressources du groupe CLIGROUP.

Test fonctionnel - Phase 1 : Comptage

```
*** SUPPRESSION GENERALE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : 1          (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS : 00011

CONFIRMER (O/N)          :

-----

MESSAGE : 00011 CLIENT(S) TROUVE(S) - CONFIRMER SUPPRESSION (O/N) ?

-----

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

L'utilisateur saisit le préfixe "1" et appuie sur ENTER. Le programme parcourt le fichier VSAM avec STARTBR/READNEXT et compte 11 clients correspondants (tous ceux dont le numéro commence par "1").

```
*** SUPPRESSION GENERALE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : _          (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS :

CONFIRMER (O/N)          :

-----

MESSAGE : SUPPRESSION ANNULEE - NOUVEAU PREFIXE OU PF3

-----

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

L'utilisateur a répondu "N" à la confirmation. Le message "NOUVEAU PREFIXE OU PF3" indique que la suppression est annulée et qu'on peut saisir un nouveau préfixe.

Test fonctionnel - Suppression d'un client unique

```
*** SUPPRESSION GENERIQUE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : 111114  (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS   : 00001

CONFIRMER (O/N)          :

-----

MESSAGE : 00001 CLIENT(S) TROUVE(S) - CONFIRMER SUPPRESSION (O/N) ?

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

Avec le préfixe "111114" (6 caractères = clé complète), seul 1 client correspond.

```
*** SUPPRESSION GENERIQUE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : 111114  (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS   : 00001

CONFIRMER (O/N)          :

-----

MESSAGE : VEUILLEZ REPONDRE O OU N

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

Le programme demande une confirmation explicite. Si l'utilisateur appuie sur ENTER sans répondre, le message "VEUILLEZ REPONDRE O OU N" s'affiche.

```
*** SUPPRESSION GENERIQUE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : _      (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS  :

CONFIRMER (O/N)         :

-----

MESSAGE : 00001 CLIENT(S) SUPPRIME(S) - NOUVEAU PREFIXE OU PF3

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

Après confirmation "O", le client 111114 est supprimé. Le message "00001 CLIENT(S) SUPPRIME(S)" confirme l'opération.

Test fonctionnel - Suppression multiple

```
*** SUPPRESSION GENERIQUE CLIENT ***
-----

PREFIXE OU CLE COMPLETE : 11111      (1 A 6 CARACTERES)

CLIENTS CORRESPONDANTS  : 00006

CONFIRMER (O/N)         : o

-----

MESSAGE : 00006 CLIENT(S) TROUVE(S) - CONFIRMER SUPPRESSION (O/N) ?

ENTER=VALIDER  PF3=QUITTER  CLEAR=REINIT.
```

Avec le préfixe "11111" (5 caractères), 6 clients correspondent. L'utilisateur répond "O" pour confirmer la suppression.

Les 6 clients ont été supprimés. Le programme a collecté les clés dans une table, fermé le browse avec ENDBR, puis exécuté DELETE pour chaque clé (évitant ainsi le deadlock).

[illegible]

Après les suppressions, DITTO/ESA montre que les clients 11111x ont bien été supprimés du fichier VSAM. Seuls les autres clients (000001, 222xxx, etc.) restent.

150 / 188

Énoncé

Faire une lecture successive des CLIENT dont le code générique est '222...' en utilisant la commande READNEXT et ENDBR.

Mon choix de conception

L'énoncé attendait probablement une navigation **un enregistrement à la fois** avec STARTBR/READNEXT :

APPROCHE ATTENDUE : Navigation séquentielle (1 par 1)

ENTER → Affiche client 222001

ENTER → Affiche client 222002

ENTER → Affiche client 222003

...

ENTER → "FIN DE FICHIER"

Mode pseudo-conversationnel simple avec COMMAREA pour garder la position courante dans le browse.

J'ai fait le choix d'une **approche différente** avec affichage de plusieurs clients à la fois :

Approche attendue	Mon approche	Différence
1 client par écran	10 clients par écran	Vue d'ensemble
ENTER = suivant	ENTER = nouvelle recherche	Interaction différente
Navigation linéaire	PF7/PF8 (avant/arrière)	Navigation bidirectionnelle
Pas de compteur	Total clients + page X/Y	Information contextuelle

Pourquoi ce choix ? Afficher un seul client par écran oblige à faire beaucoup d'ENTER pour parcourir une liste. Avec 10 clients par page et la navigation PF7/PF8, l'utilisateur a une vue d'ensemble et peut revenir en arrière, ce qui est plus ergonomique pour une liste de résultats.

Mon travail

J'ai implémenté une **liste paginée complète** au lieu d'un simple READ GENERIC :

- Affichage de **tous les clients** correspondant au préfixe
- **10 clients par page** (limite d'un écran 3270)
- Navigation **PF7** (précédent) / **PF8** (suivant)
- Compteur total et indicateur de page (X/Y)

Algorithme de pagination

ENTER : Nouvelle recherche

1. Compter tous les clients correspondants
2. Calculer le nombre de pages (total / 10)
3. Afficher la première page



PF8 : Page suivante

1. STARTBR au début du préfixe
2. READNEXT pour sauter (page - 1) × 10 enregistrements
3. READNEXT × 10 pour remplir l'écran
4. ENDBR

Résolution

MAP BMS : CLILIST.bms

Le code source est stocké dans `ROCHA.CICS.SOURCE(CLILIST)`.

En-tête du MAPSET :

```
*****
* MAPSET : CLILIST - Liste Generique des Clients
* Transaction : LGEN
* Fil Rouge CICS - Exercice 18
*****
CLILIST  DFHMSD TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,          X
          STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
```

Structure en 10 lignes répétitives :

```
*-----
* EN-TETE DES COLONNES
*-----
          DFHMD F POS=(5,1),LENGTH=50,ATTRB=(ASKIP,BRT),      X
          INITIAL='NUMCPT RG NOM          PRENOM          SOLDE          POS '
```

*-----

* LIGNE 1 (lignes 2-10 suivent le même pattern)

*-----

```
L1NUM    DFHMD F POS=(7,1),LENGTH=6,ATTRB=ASKIP              <- Numéro client
L1REG    DFHMD F POS=(7,8),LENGTH=2,ATTRB=ASKIP              <- Code région
L1NOM    DFHMD F POS=(7,11),LENGTH=10,ATTRB=ASKIP            <- Nom
```



```

L1PRE      DFHMDF POS=(7,22),LENGTH=10,ATTRB=ASKIP    <- Prénom
L1SOL      DFHMDF POS=(7,33),LENGTH=10,ATTRB=ASKIP    <- Solde
L1POS      DFHMDF POS=(7,44),LENGTH=2,ATTRB=ASKIP     <- Position (DB/CR)
...
L10NUM     DFHMDF POS=(16,1),LENGTH=6,ATTRB=ASKIP     <- Ligne 10

```

Zone de pagination :

```

*-----
* ZONE INFORMATIONS PAGINATION
*-----
          DFHMDF POS=(18,2),LENGTH=6,ATTRB=ASKIP,INITIAL='PAGE : '
PAGNUM    DFHMDF POS=(18,9),LENGTH=3,ATTRB=(ASKIP,BRT)  <- Page courante
          DFHMDF POS=(18,13),LENGTH=1,ATTRB=ASKIP,INITIAL='/'
PAGTOT    DFHMDF POS=(18,15),LENGTH=3,ATTRB=(ASKIP,BRT) <- Total pages
          DFHMDF POS=(18,22),LENGTH=7,ATTRB=ASKIP,INITIAL='TOTAL : '
CLITOT    DFHMDF POS=(18,30),LENGTH=5,ATTRB=(ASKIP,BRT) <- Total clients

```

Conception : Cette MAP utilise 60 champs (10 lignes × 6 colonnes) avec des noms courts (L1NUM, L2NUM...) pour respecter les limites de l'assembleur BMS. Les touches PF7/PF8 permettent la navigation entre les pages.

Programme : PRGLGEN.cbl - Extraits clés

Structure COMMAREA (contexte de navigation) :

```

*-----
* ZONE DE COMMUNICATION (COMMAREA)
* Sauvegarde le contexte de pagination entre passages
*-----
01  WS-COMMAREA.
    05 WS-PREFIXE-MAJ    PIC X(06) VALUE SPACES.
    05 WS-PREFIXE-MIN    PIC X(06) VALUE SPACES.
    05 WS-PREFIXE-DEB    PIC X(06) VALUE SPACES.
    05 WS-PREFIXE-FIN    PIC X(06) VALUE SPACES.
    05 WS-PAGE-COURANTE  PIC 9(03) VALUE 0.
    05 WS-TOTAL-CLIENTS  PIC 9(05) VALUE 0.
    05 WS-TOTAL-PAGES    PIC 9(03) VALUE 0.
    05 WS-FIN-FICHER     PIC X(01) VALUE 'N'.

```

Logique de pagination (saut d'enregistrements) :

```

6000-AFFICHER-PAGE.
*-----
* Affiche la page courante (10 clients)
* Technique : sauter les enregistrements des pages précédentes
*-----
*      Sauter les enregistrements des pages precedentes

```

```

        COMPUTE WS-COMPTEUR = (WS-PAGE-COURANTE - 1) * 10
        PERFORM WS-COMPTEUR TIMES
            EXEC CICS READNEXT
                FILE('FCLIENT')
                INTO(ENR-CLIENT)
                RIDFLD(WS-CLE-COURANTE)
                RESP(WS-RESP)
            END-EXEC
        END-PERFORM

*   Lire les 10 clients de cette page
        PERFORM UNTIL FIN-BROWSE OR WS-LIGNE-COURANTE >= 10
            EXEC CICS READNEXT ... END-EXEC
            ADD 1 TO WS-LIGNE-COURANTE
            MOVE CLI-NUMCPT TO WS-CLI-NUM(WS-LIGNE-COURANTE)
            ...
        END-PERFORM

```

Technique de pagination : Pour afficher la page N, on effectue $(N-1) \times 10$ READNEXT "à vide" pour sauter les enregistrements des pages précédentes, puis 10 READNEXT pour remplir l'écran.

Définition CICS :

```

CEDA DEFINE MAPSET(CLILIST) GROUP(CLIGROUP)
CEDA INSTALL MAPSET(CLILIST)

CEDA DEFINE PROGRAM(PRGLGEN) GROUP(CLIGROUP) LANGUAGE(COBOL)
CEDA INSTALL PROGRAM(PRGLGEN)

CEDA DEFINE TRANSACTION(LGEN) GROUP(CLIGROUP) PROGRAM(PRGLGEN)
CEDA INSTALL TRANSACTION(LGEN)

```

Vérification : Utiliser `CEDA DISPLAY GROUP(CLIGROUP)` pour voir toutes les ressources installées du groupe.

Points importants

1. **COMMAREA pour pagination** : Sauvegarde le préfixe, la page courante, et le total pour permettre la navigation entre les pages.
2. **Calcul du nombre de pages** : $TOTAL-PAGES = (TOTAL-CLIENTS + 9) / 10$ (arrondi supérieur)
3. **Saut d'enregistrements** : Pour afficher la page N, on fait $(N-1) \times 10$ READNEXT "à vide" avant de commencer à afficher.

Difficultés rencontrées et solutions

Problème	Symptôme	Cause	Solution
Assemblage BMS	Erreurs cryptiques à l'assemblage	60 champs avec noms longs dépassant les limites	Noms courts (L1NUM, L2NUM...)
Format JCL	Jobs ASMLIST/CMPLGEN échouaient	Paramètres mal alignés	Respecter colonnes 1-71
COMMAREA non réinitialisée	PF7/PF8 utilisait l'ancienne recherche	Pas de reset si aucun client	INITIALIZE WS-COMMAREA
PERFORM sans THRU	Double affichage de messages	GO TO sortait du PERFORM	Ajouter THRU paragraphe-FIN
Curseur mal positionné	Curseur sur dernier champ après recherche	SEND MAP sans CURSOR	Ajouter FREEKB CURSOR

Correction COMMAREA (recherche sans résultat) :

```

      IF WS-TOTAL-CLIENTS = 0
*      Reinitialiser la COMMAREA AVANT le SEND MAP
      MOVE SPACES TO WS-PREFIXE-MADE
      MOVE 0 TO WS-LONGUEUR-MADE
      MOVE SPACES TO WS-DERNIERE-CLE
      MOVE 0 TO WS-PAGE-COURANTE
      MOVE 0 TO WS-TOTAL-PAGES
      MOVE 'N' TO WS-FIN-FICHER
      ...
      END-IF

```

Captures d'écran

Définition des ressources CICS pour LGEN

La transaction de liste générique nécessite trois ressources : MAPSET, PROGRAM et TRANSACTION.

```

DEFINE MAPSET(CLILIST) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEfINE MApset( CLILIST )
  MApset      : CLILIST
  Group       : CLIGROUP
  DEScription ==> _
  REsident    ==> No                No ! Yes
  USAge       ==> Normal            Normal ! Transient
  USElpacopy  ==> No                No ! Yes
  Status      ==> Enabled            Enabled ! Disabled
  RSl         : 00                  0-24 ! Public
DEFINITION SIGNATURE
  DEFInetime  : 01/16/26 15:34:33
  CHANGETime  : 01/16/26 15:34:33
  CHANGEUsrid : CICSUSER
  CHANGEAGEnt : CSDApi              CSDApi ! CSDBatch
  CHANGEAGRel : 0680

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 15.34.33 DATE: 01/16/26
PF 1 HELP 2 COM 3 END                        6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA DEFINE MAPSET(CLILIST) GROUP(CLIGROUP)` crée la définition du mapset de liste paginée. Le message "DEFINE SUCCESSFUL" confirme la création.

```

DEFINE PROGRAM(PRGLGEN) GROUP(CLIGROUP) LANGUAGE(COBOL)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEfINE PROGram( PRGLGEN )
  PROGram     : PRGLGEN
  Group       : CLIGROUP
  DEScription ==> _
  Language    ==> CObol              CObol ! Assembler ! Le370 ! C ! Pli
  RELoad      ==> No                No ! Yes
  RESident    ==> No                No ! Yes
  USAge       ==> Normal            Normal ! Transient
  USElpacopy  ==> No                No ! Yes
  Status      ==> Enabled            Enabled ! Disabled
  RSl         : 00                  0-24 ! Public
  CEdf        ==> Yes                Yes ! No
  DAtalocation ==> Below              Below ! Any
  EXECKey     ==> User                User ! Cics
  COncurrency ==> Quasirent           Quasirent ! Threadsafe ! Required
  Api         ==> Cicsapi             Cicsapi ! Openapi
REMOTE ATTRIBUTES
+ Dynamic     ==> No                No ! Yes

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 15.36.04 DATE: 01/16/26
PF 1 HELP 2 COM 3 END                        6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA DEFINE PROGRAM(PRGLGEN) GROUP(CLIGROUP) LANGUAGE(COBOL)` crée la définition du programme de liste générique.

```

DEFINE TRANSACTION(LGEN) GROUP(CLIGROUP) PROGRAM(PRGLGEN)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFINE TRANSAction( LGEN )
  TRANSAction      : LGEN
  Group            : CLIGROUP
  DEScription      ==> _
  PROGram          ==> PRGLGEN
  TWAsize          ==> 00000                0-32767
  PROFile          ==> DFHCICST
  PArtitionset     ==>
  STATus           ==> Enabled              Enabled ! Disabled
  PRIMedsize       : 00000                0-65520
  TASKDATAloc      ==> Below               Below ! Any
  TASKDATAKey      ==> User                 User ! Cics
  STOrageclear     ==> No                   No ! Yes
  RUNaway          ==> System               System ! 0 ! 500-2700000
  SHutdown         ==> Disabled             Disabled ! Enabled
  ISolate          ==> Yes                  Yes ! No
  Brexit           ==>
+ REMOTE ATTRIBUTES

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 15.37.08 DATE: 01/16/26
PF 1 HELP 2 COM 3 END                        6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande `CEDA DEFINE TRANSACTION(LGEN) GROUP(CLIGROUP) PROGRAM(PRGLGEN)` associe le code "LGEN" au programme PRGLGEN.

```

DISPLAY GROUP(CLIGROUP)
ENTER COMMANDS
NAME      TYPE      GROUP      LAST CHANGE
FCLIENT   FILE      CLIGROUP   01/13/26 10:54:32
CLIAFF    MAPSET     CLIGROUP   01/12/26 17:59:42
CLIAJT    MAPSET     CLIGROUP   01/13/26 14:31:46
CLIDEL    MAPSET     CLIGROUP   01/16/26 11:12:15
CLILIST   MAPSET     CLIGROUP *  INSTALL SUCCESSFUL
CLIMAJ    MAPSET     CLIGROUP   01/15/26 11:30:18
CLISUP    MAPSET     CLIGROUP   01/15/26 15:51:20
PRGAJT    PROGRAM    CLIGROUP   01/13/26 14:33:36
PRGCLIA   PROGRAM    CLIGROUP   01/12/26 17:06:28
PRGDELG   PROGRAM    CLIGROUP   01/16/26 11:56:18
PRGLGEN   PROGRAM    CLIGROUP *  INSTALL SUCCESSFUL
PRGMAJ    PROGRAM    CLIGROUP   01/15/26 12:16:13
PRGSUP    PROGRAM    CLIGROUP   01/15/26 17:11:17
AFFI      TRANSACTION CLIGROUP   01/12/26 17:09:12
AJOU      TRANSACTION CLIGROUP   01/13/26 14:34:55
DELG      TRANSACTION CLIGROUP   01/16/26 12:00:01
+ LGEN     TRANSACTION CLIGROUP *  INSTALL SUCCESSFUL

                                           SYSID=S670 APPLID=CICSTS51
RESULTS: 1 TO 17 OF 19                     TIME: 15.38.39 DATE: 01/16/26
PF 1 HELP 2 SIG 3 END 4 TOP 5 BOT 6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

Après installation du groupe, `CEDA DISPLAY` montre les ressources `CLILIST`, `PRGLGEN` et `LGEN` installées.

Test fonctionnel - Liste avec peu de résultats

```
*** LISTE GENERIQUE DES CLIENTS ***

PREFIXE :  1      (1 A 6 CARACTERES)

NUMCPT RG NOM      PRENOM      SOLDE      POS
111122 01 TEST      TEST      9999999999 DB
111177 01 RRRRRRRRRR RRRRRRRRRR 9999999999 DB
111188 01 TEST      TEST      9999999999 DB

PAGE : 001 / 001    TOTAL : 00003 CLIENT(S)

MESSAGE :  FIN DE LISTE - PF7 POUR REVENIR

ENTER=CHERCHER  PF7=PREC  PF8=SUIV  PF3=QUIT
```

L'utilisateur saisit le préfixe "1" et appuie sur ENTER. Le programme affiche les 3 clients restants (après les suppressions de l'exercice 17). Le message "FIN DE LISTE" indique qu'il n'y a pas d'autres pages.

Test fonctionnel - Liste avec plusieurs résultats

```
*** LISTE GENERIQUE DES CLIENTS ***

PREFIXE :  0      (1 A 6 CARACTERES)

NUMCPT RG NOM      PRENOM      SOLDE      POS
000001 01 DUPONT
000002 01 MARTIN    MARIE      0000080000 DB
000003 02 BERNARD    PIERRE     0000250000 CR
000004 02 PETIT      SOPHIE     0000045000 DB
000005 03 ROBERT     ALAIN      0000320000 CR
000006 03 RICHARD    CLAIRE     0000012000 DB
000007 04 DURAND     PAUL       0000180000 CR
000008 04 MOREAU     ANNE       0000095000 DB
000009 01 LAURENT    MARC       0000420000 CR
000010 02 SIMON      JULIE      0000067000 DB

PAGE : 001 / 001    TOTAL : 00010 CLIENT(S)

MESSAGE :  PF7=PREC  PF8=SUIV  PF3=QUITTER

ENTER=CHERCHER  PF7=PREC  PF8=SUIV  PF3=QUIT
```

Avec le préfixe "0", 10 clients sont affichés sur une seule page. Le format d'affichage montre pour chaque client : numéro, région, nom, prénom, solde et position (DB/CR).

Exercice 19 : Statistiques par région

Énoncé

Élaborer une transaction permettant de calculer pour une REGION le nombre de CLIENT, la somme des montants des CLIENT Débiteurs et leur nombre et la somme des montants des CLIENT Créditeurs et leur nombre. Cette transaction aura en entrée le code REGION et affichera les quatre informations spécifiées ci-dessus.

Mon choix de conception

L'énoncé demande d'utiliser un AIX/PATH pour accéder aux clients par région. Les deux approches (attendue et implémentée) utilisent donc le fichier PCLIENT défini sur le PATH. Les différences portent sur l'implémentation :

Approche attendue	Mon approche	Différence
Affichage basique des statistiques	Validation du code région (01-04)	Ergonomie utilisateur
Gestion DUPKEY implicite	Gestion explicite de DFHRESP(DUPKEY)	Robustesse du code
Mode conversationnel simple	Mode pseudo-conversationnel	Pattern cohérent avec les autres transactions

Point commun : Les deux approches nécessitent la définition de l'AIX, du PATH, et du FILE CICS PCLIENT. La manipulation dans ADCD.Z113F.PROCLIB est également requise (voir ci-dessous).

Mon travail

Cette transaction utilise l'**AIX (Alternate Index)** sur le champ CODREG comme demandé par l'énoncé. J'ai ajouté :

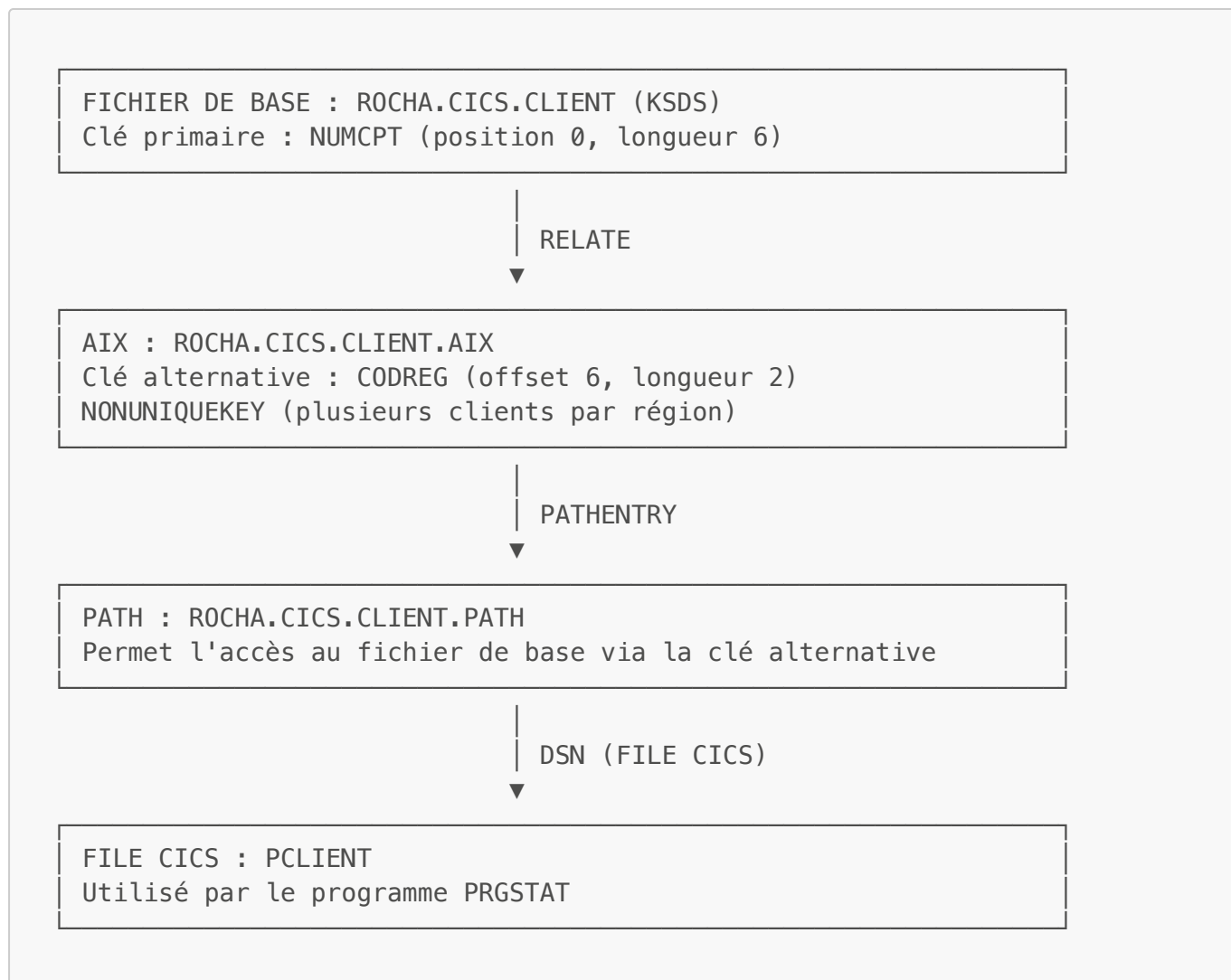
- **Validation du code région** : seules les valeurs 01-04 sont acceptées
- **Affichage du nom de la région** : Paris, Marseille, Lyon ou Lille
- **Gestion explicite de DFHRESP(DUPKEY)** : réponse normale pour un AIX avec NONUNIQUEKEY

Pourquoi un AIX/PATH est nécessaire ?

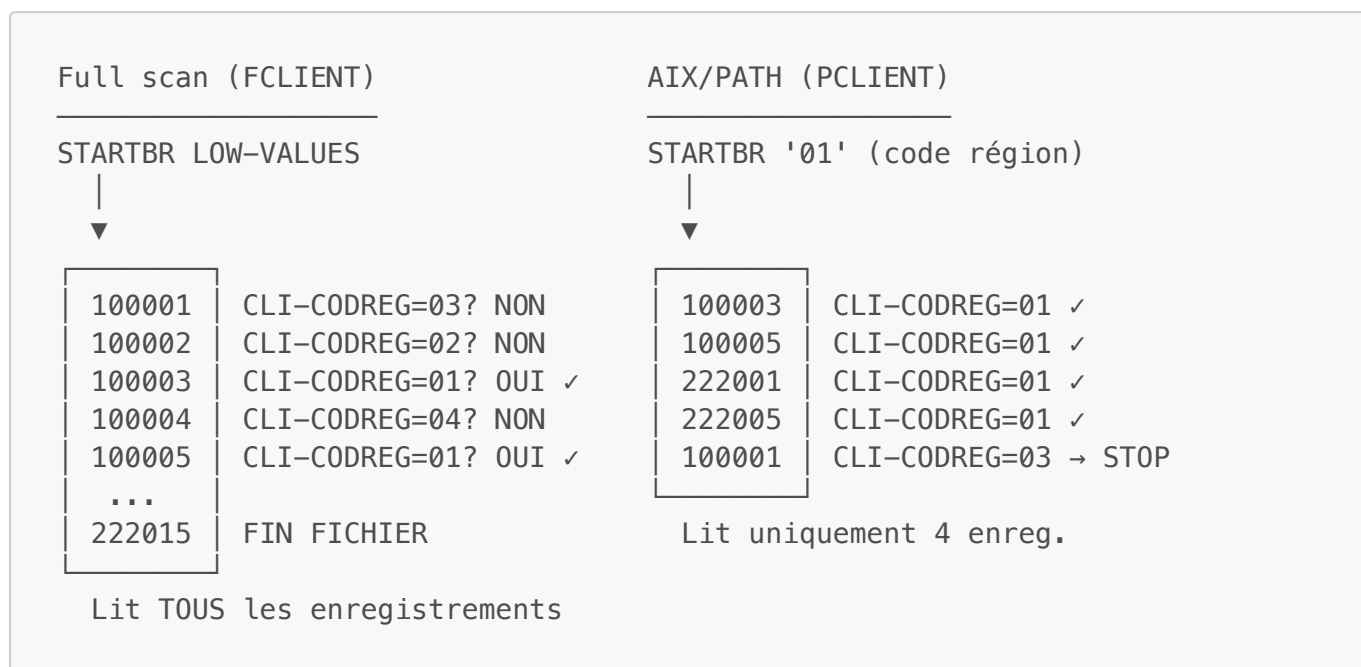
Approche	Avantages	Inconvénients
Full scan (sans AIX)	Simple, pas de configuration	Lit TOUT le fichier, inefficace
AIX/PATH sur CODREG	Accès direct par région, performant	Nécessite définition AIX + PATH + FILE CICS

L'énoncé impose l'utilisation d'un AIX pour des raisons pédagogiques et de performance. En production, avec des milliers de clients, le full scan serait très lent.

Architecture AIX/PATH



Différence Full scan vs AIX/PATH



Résolution

Étape 1 : Définition AIX et PATH (DEFPATH.jcl)

Ce JCL crée l'index alternatif sur le champ CODREG et le PATH associé :

```

/*-----*
/* ETAPE 2 : Definition de l'ALTERNATE INDEX (AIX)                *
/*          Cle alternative : CODREG (offset 6, longueur 2)        *
/*          NONUNIQUEKEY : plusieurs clients par region          *
/*-----*
//STEP2    EXEC PGM=IDCAMS
//SYSIN    DD *
  DEFINE ALTERNATEINDEX ( -
    NAME(ROCHA.CICS.CLIENT.AIX) -
    RELATE(ROCHA.CICS.CLIENT) -
    KEYS(2 6) -
    NONUNIQUEKEY -
    UPGRADE -
  ) ...
/*
/*-----*
/* ETAPE 3 : Construction de l'AIX (BLDINDEX)                    *
/*-----*
//STEP3    EXEC PGM=IDCAMS
//SYSIN    DD *
  BLDINDEX -
    INDATASET(ROCHA.CICS.CLIENT) -
    OUTDATASET(ROCHA.CICS.CLIENT.AIX)
/*
/*-----*
/* ETAPE 4 : Definition du PATH                                   *
/*-----*
//STEP4    EXEC PGM=IDCAMS
//SYSIN    DD *
  DEFINE PATH ( -
    NAME(ROCHA.CICS.CLIENT.PATH) -
    PATHENTRY(ROCHA.CICS.CLIENT.AIX) -
  )
/*
```

Paramètres clés de l'AIX :

Paramètre	Valeur	Explication
KEYS(2 6)	2 octets à l'offset 6	Position du champ CODREG dans l'enregistrement
NONUNIQUEKEY	-	Plusieurs clients peuvent avoir le même code région
UPGRADE	-	L'AIX est mis à jour automatiquement quand le fichier de base change
RELATE	ROCHA.CICS.CLIENT	Fichier de base associé

Étape 2 : MAP BMS (CLISTAT.bms)

Le code source est stocké dans **ROCHA.CICS.SOURCE(CLISTAT)**.

En-tête du MAPSET avec commentaires :

```
*****
* MAPSET : CLISTAT - Statistiques par Region
* Transaction : STAT
* Fil Rouge CICS - Exercice 19
*
* FONCTIONNALITE :
* -----
* Affiche les statistiques d'une region :
* - Nombre total de clients
* - Nombre et somme des clients debiteurs (DB)
* - Nombre et somme des clients crediters (CR)
*
* REGIONS DISPONIBLES :
* 01 - Paris      02 - Marseille
* 03 - Lyon       04 - Lille
*****
CLISTAT  DFHMSD TYPE=&SYSPARM,MODE=INOUT,LANG=COBOL,          X
          STORAGE=AUTO,CTRL=(FREEKB,FRSET),TIOAPFX=YES
```

Zone de saisie et zones de résultats :

```
*-----
* ZONE DE SAISIE - CODE REGION
*-----
CODREG   DFHMD F POS=(4,28),LENGTH=2,ATTRB=(UNPROT,NUM,IC)
          ~~~~~
          Code région (01-04) - UNPROT,NUM : numérique saisissable

*-----
* NOM DE LA REGION (affiché après saisie)
*-----
NOMREG   DFHMD F POS=(6,28),LENGTH=15,ATTRB=(ASKIP,BRT)
          Paris, Marseille, Lyon ou Lille

*-----
* STATISTIQUES
*-----
NBTOT    DFHMD F POS=(10,38),LENGTH=5,ATTRB=(ASKIP,BRT)  <- Total clients
NBDB     DFHMD F POS=(12,38),LENGTH=5,ATTRB=(ASKIP,BRT)  <- Nb débiteurs
MTDB     DFHMD F POS=(13,38),LENGTH=15,ATTRB=(ASKIP,BRT) <- Somme DB
NBCR     DFHMD F POS=(15,38),LENGTH=5,ATTRB=(ASKIP,BRT)  <- Nb créditeurs
MTCR     DFHMD F POS=(16,38),LENGTH=15,ATTRB=(ASKIP,BRT) <- Somme CR
```

Conception : Cette MAP affiche des statistiques calculées, tous les champs de résultat sont en ASKIP (lecture seule). Seul le code région est saisissable.

Étape 3 : Programme PRGSTAT.cbl - Extraits clés

Variables pour conversion solde (REDEFINES) :

```
*-----
* VARIABLES POUR CONVERSION SOLDE (REDEFINES)
* Note: FUNCTION NUMVAL non supporté sur IBM Enterprise COBOL
*-----
01  WS-SOLDE-ALPHA          PIC X(10) VALUE SPACES.
01  WS-SOLDE-NUM REDEFINES WS-SOLDE-ALPHA
                                PIC 9(10).
```

Note technique : La fonction **NUMVAL** n'est pas supportée dans le contexte MOVE sur IBM Enterprise COBOL. La solution est d'utiliser **REDEFINES** pour réinterpréter la zone alphanumérique comme numérique.

Paragraphe de calcul des statistiques :

```
3000-CALCULER-STATS.
*-----
* Parcours du fichier via AIX/PATH pour la région demandée
* L'AIX permet d'accéder directement aux clients de la région
*-----
    INITIALIZE WS-STATS
    MOVE 'N' TO WS-FIN-BROWSE

*   Positionner sur la clé AIX (code région)
    MOVE WS-CODE-REGION TO WS-CLE-AIX

    EXEC CICS STARTBR
        FILE('PCLIENT')
        RIDFLD(WS-CLE-AIX)
        RESP(WS-RESP)
    END-EXEC

*   Gestion explicite des erreurs STARTBR
    EVALUATE WS-RESP
        WHEN DFHRESP(NORMAL)
            CONTINUE
        WHEN DFHRESP(NOTFND)
*       Aucun client dans cette région
            GO TO 3000-FIN
        WHEN OTHER
            GO TO 3000-FIN
    END-EVALUATE

*   Boucle de lecture des enregistrements de la région
    PERFORM UNTIL FIN-BROWSE
        EXEC CICS READNEXT
            FILE('PCLIENT')
```

```

        INTO(ENR-CLIENT)
        RIDFLD(WS-CLE-AIX)
        RESP(WS-RESP)
    END-EXEC

    EVALUATE TRUE
        WHEN WS-RESP = DFHRESP(ENDFILE)
            MOVE '0' TO WS-FIN-BROWSE
        WHEN WS-RESP NOT = DFHRESP(NORMAL)
            AND WS-RESP NOT = DFHRESP(DUPKEY)
*           Erreur autre que DUPKEY (normal pour AIX)
            MOVE '0' TO WS-FIN-BROWSE
        WHEN CLI-CODREG NOT = WS-CODE-REGION
*           Changement de région = fin du browse
            MOVE '0' TO WS-FIN-BROWSE
        WHEN OTHER
*           Client de la région - comptabiliser
            ADD 1 TO WS-NB-TOTAL
            PERFORM 3100-CONVERTIR-SOLDE
            IF CLI-POSITION = 'DB'
                ADD 1 TO WS-NB-DEBITEURS
                ADD WS-SOLDE-NUM TO WS-MT-DEBITEURS
            ELSE
                ADD 1 TO WS-NB-CREDITEURS
                ADD WS-SOLDE-NUM TO WS-MT-CREDITEURS
            END-IF
        END-EVALUATE
    END-PERFORM

*   Fermeture du browse
    EXEC CICS ENDBR FILE('PCLIENT') END-EXEC.

3000-FIN.
EXIT.

```

Étape 4 : Définitions CICS

Définition du FILE PCLIENT (PATH) :

```

CEDA DEFINE FILE(PCLIENT) GROUP(CLIGROUP)
    DSNAME(ROCHA.CICS.CLIENT.PATH)
    ADD(NO) BROWSE(YES) DELETE(NO) READ(YES) UPDATE(NO)
    LSRPOOLID(1)
    STRINGS(2)
    RECORDFORMAT(F)

CEDA INSTALL FILE(PCLIENT)

```

Paramètre	Valeur	Explication
ADD(NO)	-	Pas d'ajout via le PATH (utiliser FCLIENT)
BROWSE(YES)	-	Permet STARTBR/READNEXT/ENDBR
DELETE(NO)	-	Pas de suppression via le PATH
UPDATE(NO)	-	Pas de mise à jour via le PATH

Note : Le PATH est en lecture seule. Les opérations d'écriture doivent se faire via le fichier de base FCLIENT.

Note environnement TK4- : Comme pour FCLIENT (voir Partie 1, Exercice 4), une manipulation dans le membre CICSTS51 de la bibliothèque ADCD.Z113F.PROCLIB est nécessaire pour que CICS reconnaisse le nouveau FILE PCLIENT. Il faut ajouter une entrée pour PCLIENT pointant vers le dataset ROCHA.CICS.CLIENT.PATH.

Définition de la transaction :

```

CEDA DEFINE MAPSET(CLISTAT) GROUP(CLIGROUP)
CEDA INSTALL MAPSET(CLISTAT)

CEDA DEFINE PROGRAM(PRGSTAT) GROUP(CLIGROUP) LANGUAGE(COBOL)
CEDA INSTALL PROGRAM(PRGSTAT)

CEDA DEFINE TRANSACTION(STAT) GROUP(CLIGROUP) PROGRAM(PRGSTAT)
CEDA INSTALL TRANSACTION(STAT)

```

Visualisation : Utiliser **CEDA DISPLAY GROUP(CLIGROUP)** pour vérifier l'ensemble des ressources du groupe (voir captures ci-dessous).

Procédure de déploiement complète

1. DÉFINITION AIX/PATH (JCL)
 - Soumettre DEFPATH.jcl
 - Crée ROCHA.CICS.CLIENT.AIX et ROCHA.CICS.CLIENT.PATH
2. DÉFINITION FILE CICS
 - CEDA DEFINE FILE(PCLIENT) ... DSN(ROCHA.CICS.CLIENT.PATH)
 - CEDA INSTALL FILE(PCLIENT)
3. ASSEMBLAGE MAP BMS
 - Copier CLISTAT.bms → ROCHA.CICS.SOURCE(CLISTAT)
 - Soumettre ASMSTAT.jcl
4. COMPILATION PROGRAMME
 - Copier PRGSTAT.cbl → ROCHA.CICS.SOURCE(PRGSTAT)
 - Soumettre CMPSTAT.jcl

5. DÉFINITION ET INSTALLATION CICS
 - CEDA DEFINE MAPSET/PROGRAM/TRANSACTION
 - CEDA INSTALL pour chaque ressource
6. TEST
 - STAT → Saisir 01, 02, 03 ou 04

Résultats attendus (avec les données initiales)

Région	Total	Débiteurs	Montant DB	Créditeurs	Montant CR
01 Paris	5	1	80 000	4	871 000
02 Marseille	4	2	77 000	2	395 000
03 Lyon	3	1	12 000	2	598 000
04 Lille	3	2	118 000	1	180 000

Points importants

1. **FILE('PCLIENT')** : Le programme utilise le PATH (accès via AIX) au lieu de FCLIENT.
2. **WS-CLE-AIX PIC X(02)** : La clé de browse est de 2 caractères (code région) au lieu de 6.
3. **DFHRESP(DUPKEY)** : Réponse normale pour un AIX avec NONUNIQUEKEY. Elle indique qu'il existe d'autres enregistrements avec la même clé alternative.
4. **Condition d'arrêt** : **CLI-CODREG NOT = WS-CODE-REGION** - Quand on rencontre un client d'une autre région, on arrête le browse.

Difficultés rencontrées et solutions

Problème	Cause	Solution
Erreur DFH7053I	Ligne COBOL dépassant colonne 72	Raccourcir les messages
NUMVAL not allowed	FUNCTION NUMVAL non supporté dans MOVE	Utiliser REDEFINES
Double affichage	PERFORM sans THRU + GO TO	Ajouter THRU paragraphe- FIN

Règle COBOL mainframe : Si un paragraphe contient un **GO TO** vers un paragraphe de sortie, le **PERFORM** appelant doit inclure **THRU** jusqu'à ce paragraphe (voir Partie 2a, Exercice 7).

Captures d'écran

Vérification des datasets AIX et PATH

Avant de définir les ressources CICS, on vérifie que l'AIX et le PATH ont été correctement créés par le JCL DEFPATH.

```

DSLIS - Data Sets Matching ROCHA.CICS                               Row 1 of 10
Command ==> _____ Scroll ==> CSR

Command - Enter "/" to select action                               Message                               Volume
-----
      ROCHA.CICS.CLIENT                                           *VSAM*
      ROCHA.CICS.CLIENT.AIX                                       *AIX *
      ROCHA.CICS.CLIENT.AIX.DATA                                  FDDBAS
      ROCHA.CICS.CLIENT.AIX.INDEX                                FDDBAS
      ROCHA.CICS.CLIENT.DATA                                      FDDBAS
      ROCHA.CICS.CLIENT.INDEX                                    FDDBAS
      ROCHA.CICS.CLIENT.PATH                                     *PATH*
      ROCHA.CICS.LINK                                           FDDBAS
      ROCHA.CICS.LOAD                                           FDDBAS
      ROCHA.CICS.SOURCE                                          FDDBAS
***** End of Data Set list *****

```

La liste DSLIST montre les datasets du projet CICS : le fichier de base CLIENT, l'index alternatif CLIENT.AIX et le chemin d'accès CLIENT.PATH.

Création de l'AIX et du PATH avec IDCAMS

```

IDC0508I DATA ALLOCATION STATUS FOR VOLUME FDDBAS IS 0
IDC0509I INDEX ALLOCATION STATUS FOR VOLUME FDDBAS IS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0

```

Le JCL DEFPATH utilise IDCAMS pour créer l'AIX (Alternate Index) sur le champ CODREG avec les paramètres KEYS(2 6) et NONUNIQUEKEY.

```

IDC0652I ROCHA.CICS.CLIENT.AIX SUCCESSFULLY BUILT
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0

```

La commande BLDINDEX construit l'index alternatif à partir des données du fichier de base. Le message "AIX SUCCESSFULLY BUILT" confirme la réussite.

```

      DEFINE PATH ( -                                           00780001
          NAME(ROCHA.CICS.CLIENT.PATH) -                       00790001
          PATHENTRY(ROCHA.CICS.CLIENT.AIX) -                   00800001
      )                                                         00810001
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0

```

La commande DEFINE PATH crée le chemin d'accès ROCHA.CICS.CLIENT.PATH qui permet d'accéder au fichier de base via l'index alternatif.

```

LISTCAT ENTRIES(ROCHA.CICS.CLIENT) ALL                                0090000
0CLUSTER ----- ROCHA.CICS.CLIENT
  IN-CAT --- CATALOG.Z113F.MASTER
  HISTORY
    DATASET-OWNER----- (NULL)      CREATION-----2026.012
    RELEASE-----2      EXPIRATION-----0000.000
    CA-RECLAIM----- (YES)
    EATTR----- (NULL)
    BWO STATUS----- (NULL)      BWO TIMESTAMP----- (NULL)
    BWO----- (NULL)
    PROTECTION-PSWD----- (NULL)      RACF----- (NO)
  ASSOCIATIONS
    DATA-----ROCHA.CICS.CLIENT.DATA
    INDEX-----ROCHA.CICS.CLIENT.INDEX
    AIX-----ROCHA.CICS.CLIENT.AIX
0  DATA ----- ROCHA.CICS.CLIENT.DATA
  IN-CAT --- CATALOG.Z113F.MASTER

```

LISTCAT montre les associations entre le cluster de base, les composants *DATA* et *INDEX*, l'*AIX* et le *PATH*.

Définition du FILE CICS pour le PATH

```

OVERTYPE TO MODIFY                                                    CICS RELEASE = 0680
CEDA DEFine File( PCLIENT )
  File           : PCLIENT
  Group          : CLIGROUP
  DEScription    ==>
VSAM PARAMETERS
  DSName         ==> ROCHA.CICS.CLIENT.PATH
  Password       ==>                                PASSWORD NOT SPECIFIED
  RLsaccess      ==> No                               Yes ! No
  LSRPOOLId      : 1                                1-8 ! None
  LSRPOOLNum     ==> 001                             1-255 ! None
  READInteg      ==> Uncommitted                     Uncommitted ! Consistent ! Repeatable
  DSNSharing     ==> Allreqs                          Allreqs ! Modifyreqs
  STRings        ==> 002                             1-255
  Nsrgroup       ==>
REMOTE ATTRIBUTES
  REMOTESystem   ==>
  REMOTENAME     ==>
+ REMOTE AND CFDATATABLE PARAMETERS

                                                                    SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                                                    TIME: 12.00.28 DATE: 01/19/26

```

La commande *CEDA DEFINE FILE(PCLIENT)* définit le fichier CICS qui pointe vers le *PATH*. Le DSN est *ROCHA.CICS.CLIENT.PATH*.


```

OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEfine File( PCLIENT )
+ LSRP00LNum ==> 001                               1-255 ! None
  READInteg  ==> Uncommitted                       Uncommitted ! Consistent ! Repeatable
  DSNSharing ==> Allreqs                           Allreqs ! Modifyreqs
  STRings    ==> 002                               1-255
  Nsrgroup   ==>
REMOTE ATTRIBUTES
  REMOTESystem ==>
  REMOTENAME ==>
REMOTE AND CFDATATABLE PARAMETERS
  RECORDSize ==> 1-32767
  Keylength  ==> 1-255 (1-16 For CF Datatable)
INITIAL STATUS
  Status      ==> Enabled                           Enabled ! Disabled ! Unenabled
  Opentime    ==> Firstref                          Firstref ! Startup
  DIsposition ==> Share                             Share ! Old
BUFFERS
+ Databuffers ==> 00003                             2-32767

SYSID=S670 APPLID=CICSTS51

```

Les paramètres du FILE : ADD(NO), BROWSE(YES), DELETE(NO), READ(YES), UPDATE(NO). Le PATH est en lecture seule.

```

INSTALL FILE(PCLIENT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY
CEDA Install
  ATomservice ==> _
  Bundle      ==>
  CONnection  ==>
  C0Rbaserver ==>
  DB2Conn     ==>
  DB2Entry    ==>
  DB2Tran     ==>
  DJar        ==>
  D0ctemplate ==>
  Enqmodel    ==>
  File        ==> PCLIENT
  Ipconn      ==>
  J0urnalmodel ==>
  JVmserver   ==>
  LIBrary     ==>
  LSrpool     ==>
+ MAPset      ==>

SYSID=S670 APPLID=CICSTS51
INSTALL SUCCESSFUL      TIME: 12.01.59 DATE: 01/19/26

```

La commande CEDA INSTALL FILE(PCLIENT) active le fichier. Le message "INSTALL SUCCESSFUL" confirme que le PATH est accessible.

```

VIEW FILE(pCLIENT) GROUP(CLIGROUP)
OBJECT CHARACTERISTICS                                CICS RELEASE = 0680
CEDA View File( PCLIENT )
+ Indexbuffers   : 00002                               1-32767
DATATABLE PARAMETERS
TABLE           : No                                   No ! Cics ! User ! CF
Maxnumrecs      : Nolimit                             Nolimit ! 1-99999999
CFDATATABLE PARAMETERS
CFdtpool        :
TABLEName       :
UPDATEModel     : Locking                             Contention ! Locking
LOad            : No                                   No ! Yes
DATA FORMAT
RECORDFormat    : F                                   V ! F
OPERATIONS
Add             : No                                   No ! Yes
Browse          : Yes                                 No ! Yes
DElete         : No                                   No ! Yes
READ           : Yes                                 Yes ! No
+ UPDATE        : No                                   No ! Yes

SYSID=S670 APPLID=CICSTS51

```

CEDA VIEW FILE(PCLIENT) montre les opérations autorisées : Browse et Read uniquement (accès via l'index alternatif).

Définition des ressources CICS pour STAT

```

DEFINE MAPSET(CLISTAT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                CICS RELEASE = 0680
CEDA DEFine MAPset( CLISTAT )
MAPset       : CLISTAT
Group        : CLIGROUP
DEscription  ==> _
REsident     ==> No                                   No ! Yes
USAge        ==> Normal                               Normal ! Transient
USElpacopy   ==> No                                   No ! Yes
Status       ==> Enabled                               Enabled ! Disabled
RSl          : 00                                     0-24 ! Public
DEFINITION SIGNATURE
DEFinetime   : 01/19/26 12:10:47
CHANGTime    : 01/19/26 12:10:47
CHANGEUsrid  : CICSUSER
CHANGEAGent  : CSDApi                                CSDApi ! CSDBatch
CHANGEAGRel  : 0680

SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                                TIME: 12.10.47 DATE: 01/19/26
PF 1 HELP 2 COM 3 END                          6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL

```

La commande CEDA DEFINE MAPSET(CLISTAT) GROUP(CLIGROUP) crée la définition du mapset de statistiques.

```

DEFINE program(prgstat) group(cligroup) language(cobol)
OVERTYPE TO MODIFY                                     CICS RELEASE = 0680
CEDA DEFINE PROGram( PRGSTAT )
  PROGram      : PRGSTAT
  Group        : CLIGROUP
  DEScription  ==> _
  Language     ==> CObol          CObol ! Assembler ! Le370 ! C ! Pli
  REload       ==> No             No ! Yes
  RESident     ==> No             No ! Yes
  USAge        ==> Normal         Normal ! Transient
  USElpacopy   ==> No             No ! Yes
  Status       ==> Enabled        Enabled ! Disabled
  RSl          : 00              0-24 ! Public
  CEdf         ==> Yes            Yes ! No
  DAtalocation ==> Below          Below ! Any
  EXECKey      ==> User           User ! Cics
  COncurrency  ==> Quasirent      Quasirent ! Threadsafe ! Required
  Api          ==> Cicsapi        Cicsapi ! Openapi
  REMOTE ATTRIBUTES
+  DYnamic      ==> No            No ! Yes

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 12.11.45 DATE: 01/19/26

```

La commande `CEDA DEFINE PROGRAM(PRGSTAT) GROUP(CLIGROUP) LANGUAGE(COBOL)` crée la définition du programme de statistiques.

```

DEFINE TRANSACTION(STAT) PROGRAM(PRGSTAT) GROUP(CLIGROUP)
OVERTYPE TO MODIFY                                     CICS RELEASE = 0680
CEDA DEFINE TRANSAction( STAT )
  TRANSAction  : STAT
  Group        : CLIGROUP
  DEScription  ==> _
  PROGram      ==> PRGSTAT
  TWasize      ==> 00000          0-32767
  PROFile      ==> DFHCICST
  PArtitionset ==>
  STATUS       ==> Enabled        Enabled ! Disabled
  PRIMedsize   : 00000          0-65520
  TASKDATAloc  ==> Below          Below ! Any
  TASKDATAKey  ==> User           User ! Cics
  STOrageclear ==> No             No ! Yes
  RUnaway      ==> System         System ! 0 ! 500-2700000
  SHutdown     ==> Disabled       Disabled ! Enabled
  ISolate      ==> Yes            Yes ! No
  Brexit       ==>
+ REMOTE ATTRIBUTES

                                           SYSID=S670 APPLID=CICSTS51
DEFINE SUCCESSFUL                               TIME: 12.12.39 DATE: 01/19/26

```

La commande `CEDA DEFINE TRANSACTION(STAT) GROUP(CLIGROUP) PROGRAM(PRGSTAT)` associe le code "STAT" au programme PRGSTAT.

```

DISPLAY GROUP(CLIGROUP)
ENTER COMMANDS

```

NAME	TYPE	GROUP	LAST CHANGE
FCLIENT	FILE	CLIGROUP	01/13/26 10:54:32
PCLIENT	FILE	CLIGROUP	01/19/26 12:08:04
CLIAFF	MAPSET	CLIGROUP	01/12/26 17:59:42
CLIAJT	MAPSET	CLIGROUP	01/13/26 14:31:46
CLIDEL	MAPSET	CLIGROUP	01/16/26 11:12:15
CLILIST	MAPSET	CLIGROUP	01/16/26 15:34:33
CLIMAJ	MAPSET	CLIGROUP	01/15/26 11:30:18
CLISTAT	MAPSET	CLIGROUP *	INSTALL SUCCESSFUL
CLISUP	MAPSET	CLIGROUP	01/15/26 15:51:20
PRGAJT	PROGRAM	CLIGROUP	01/13/26 14:33:36
PRGCLIA	PROGRAM	CLIGROUP	01/12/26 17:06:28
PRGDELG	PROGRAM	CLIGROUP	01/16/26 11:56:18
PRGLGEN	PROGRAM	CLIGROUP	01/16/26 15:36:04
PRGMAJ	PROGRAM	CLIGROUP	01/15/26 12:16:13
PRGSTAT	PROGRAM	CLIGROUP *	INSTALL SUCCESSFUL
PRGSUP	PROGRAM	CLIGROUP —	01/15/26 17:11:17
+ AFFI	TRANSACTION	CLIGROUP	01/12/26 17:09:12

SYSID=S670 APPLID=CICSTS51

CEDA DISPLAY GROUP(CLIGROUP) montre les ressources CLISTAT installées.

AJOU	TRANSACTION	CLIGROUP	01/13/26 14:34:55
DELG	TRANSACTION	CLIGROUP	01/16/26 12:00:01
LGEN	TRANSACTION	CLIGROUP	01/16/26 15:37:08
MAJO	TRANSACTION	CLIGROUP	01/15/26 12:30:37
STAT	TRANSACTION	CLIGROUP *	INSTALL SUCCESSFUL
SUPP	TRANSACTION	CLIGROUP —	01/15/26 17:18:45

Suite de la liste montrant la transaction STAT définie et installée.

Test fonctionnel - Statistiques par région

```
*** STATISTIQUES PAR REGION ***
-----
CODE REGION      :  01  (01=PARIS, 02=MARSEILLE, 03=LYON, 04=LIL
REGION           :  PARIS
-----

NOMBRE TOTAL DE CLIENTS      :  00010
CLIENTS DEBITEURS (DB)      :  00005
SOMME DES SOLDES DEBITEURS   :  30,000,111,997

CLIENTS CREDITEURS (CR)     :  00005
SOMME DES SOLDES CREDITEURS  :  1,000,565,000
-----

MESSAGE :  STATISTIQUES CALCULEES AVEC SUCCES

ENTER=CALCULER  PF3=QUITTER  CLEAR=REINIT.
```

Transaction STAT avec code région 01 (PARIS). L'écran affiche : 10 clients total, 5 débiteurs, 5 créditeurs, avec les sommes des soldes pour chaque catégorie.

```
*** STATISTIQUES PAR REGION ***
-----
CODE REGION      :  02  (01=PARIS, 02=MARSEILLE, 03=LYON, 04=LIL
REGION           :  MARSEILLE
-----

NOMBRE TOTAL DE CLIENTS      :  00004
CLIENTS DEBITEURS (DB)      :  00002
SOMME DES SOLDES DEBITEURS   :  112,000

CLIENTS CREDITEURS (CR)     :  00002
SOMME DES SOLDES CREDITEURS  :  528,000
-----

MESSAGE :  STATISTIQUES CALCULEES AVEC SUCCES

ENTER=CALCULER  PF3=QUITTER  CLEAR=REINIT.
```

Transaction STAT avec code région 02 (MARSEILLE). Résultats : 4 clients, 2 débiteurs, 2 créditeurs.

```
*** STATISTIQUES PAR REGION ***
-----
CODE REGION      :  03  (01=PARIS, 02=MARSEILLE, 03=LYON, 04=LIL
REGION          :  LYON
-----

NOMBRE TOTAL DE CLIENTS      :  00003
CLIENTS DEBITEURS (DB)      :  00002
SOMME DES SOLDES DEBITEURS   :                101,000

CLIENTS CREDITEURS (CR)     :  00001
SOMME DES SOLDES CREDITEURS  :                320,000
-----

MESSAGE :  STATISTIQUES CALCULEES AVEC SUCCES

ENTER=CALCULER  PF3=QUITTER  CLEAR=REINIT.
```

Transaction STAT avec code région 03 (LYON). Résultats : 3 clients, 2 débiteurs, 1 créateur.

```
*** STATISTIQUES PAR REGION ***
-----
CODE REGION      :  04  (01=PARIS, 02=MARSEILLE, 03=LYON, 04=LIL
REGION          :  LILLE
-----

NOMBRE TOTAL DE CLIENTS      :  00003
CLIENTS DEBITEURS (DB)      :  00001
SOMME DES SOLDES DEBITEURS   :                95,000

CLIENTS CREDITEURS (CR)     :  00002
SOMME DES SOLDES CREDITEURS  :               336,000
-----

MESSAGE :  STATISTIQUES CALCULEES AVEC SUCCES

ENTER=CALCULER  PF3=QUITTER  CLEAR=REINIT.
```

Transaction STAT avec code région 04 (LILLE). Résultats : 3 clients, 1 débiteur, 2 créateurs.

Test des cas d'erreur

```
*** STATISTIQUES PAR REGION ***
-----
CODE REGION      :   05   (01=PARIS, 02=MARSEILLE, 03=LYON, 04=LIL
REGION          :
-----

NOMBRE TOTAL DE CLIENTS      :
CLIENTS DEBITEURS (DB)      :
SOMME DES SOLDES DEBITEURS   :

CLIENTS CREDITEURS (CR)     :
SOMME DES SOLDES CREDITEURS  :
-----

MESSAGE : CODE REGION INVALIDE - SAISIR 01, 02, 03 OU 04

ENTER=CALCULER  PF3=QUITTER  CLEAR=REINIT.
```

Transaction STAT avec code région 05. Le message "CODE REGION INVALIDE" s'affiche car seules les régions 01 à 04 sont autorisées.

```
*** STATISTIQUES PAR REGION ***
-----
CODE REGION      :   05   (01=PARIS, 02=MARSEILLE, 03=LYON, 04=LIL
REGION          :
-----

NOMBRE TOTAL DE CLIENTS      :   00000
CLIENTS DEBITEURS (DB)      :   00000
SOMME DES SOLDES DEBITEURS   :           0

CLIENTS CREDITEURS (CR)     :   00000
SOMME DES SOLDES CREDITEURS  :           0
-----

MESSAGE : AUCUN CLIENT DANS CETTE REGION

ENTER=CALCULER  PF3=QUITTER  CLEAR=REINIT.
```

Après correction, si une région valide ne contient aucun client, le message "AUCUN CLIENT DANS CETTE REGION" s'affiche avec des statistiques à zéro.

Récapitulatif des ressources CLIGROUP après Partie 3

Type	Nom	Description	Défini dans
FILE	FCLIENT	Fichier VSAM clients (base)	Exercice 1
FILE	PCLIENT	PATH vers AIX sur CODREG	Exercice 19
MAPSET	CLIAFF	Écran affichage	Exercice 4
MAPSET	CLIAJT	Écran ajout	Exercice 8
MAPSET	CLIMAJ	Écran mise à jour	Exercice 9
MAPSET	CLISUP	Écran suppression	Exercice 12
MAPSET	CLIDEL	Écran suppression générique	Exercice 17
MAPSET	CLILIST	Écran liste paginée	Exercice 18
MAPSET	CLISTAT	Écran statistiques	Exercice 19
PROGRAM	PRGCLIA	Programme affichage	Exercice 4
PROGRAM	PRGAJT	Programme ajout	Exercice 8
PROGRAM	PRGMAJ	Programme mise à jour	Exercice 10
PROGRAM	PRGSUP	Programme suppression	Exercice 13
PROGRAM	PRGDELG	Programme suppression générique	Exercice 17
PROGRAM	PRGLGEN	Programme liste paginée	Exercice 18
PROGRAM	PRGSTAT	Programme statistiques	Exercice 19
TRANSACTION	AFFI	Transaction affichage	Exercice 4
TRANSACTION	AJOU	Transaction ajout	Exercice 8
TRANSACTION	MAJO	Transaction mise à jour	Exercice 11
TRANSACTION	SUPP	Transaction suppression	Exercice 14
TRANSACTION	DELG	Transaction suppression générique	Exercice 17
TRANSACTION	LGEN	Transaction liste paginée	Exercice 18
TRANSACTION	STAT	Transaction statistiques	Exercice 19

Conclusion et Annexes

Bilan du projet

Synthèse chiffrée

Catégorie	Quantité	Détails
Programmes COBOL-CICS	7	PRGCLIA, PRGAJT, PRGMAJ, PRGSUP, PRGDELG, PRGLGEN, PRGSTAT
MAPs BMS	7	CLIAFF, CLIAJT, CLIMAJ, CLISUP, CLIDEL, CLILIST, CLISTAT
Transactions CICS	7	AFFI, AJOU, MAJO, SUPP, DELG, LGEN, STAT
Fichiers VSAM	2	FCLIENT (KSDS), PCLIENT (PATH via AIX)
JCL	17	Définition VSAM, assemblage BMS, compilation COBOL
Exercices réalisés	19	Répartis en 4 parties thématiques
Captures d'écran	160+	Documentation complète de chaque étape

Commandes CICS maîtrisées :

Opération	Commandes
Lecture	READ, READ UPDATE
Écriture	WRITE, REWRITE, DELETE
Navigation	STARTBR, READNEXT, ENDBR
Écrans	SEND MAP, RECEIVE MAP
Contrôle	RETURN, RETURN TRANSID

Difficultés rencontrées et solutions

Définition VSAM et chargement (Partie 1)

Problème	Cause	Solution
Erreur VSAM 108 au chargement	Longueur incorrecte des enregistrements	Le DD * du JCL lit en LRECL=80 par défaut. Définir RECORDSIZE(80 80) et utiliser un FILLER de 16 octets
Volume non spécifié (TK4-)	Paramètre manquant	Ajouter VOLUMES(FDDBAS) dans la définition du cluster VSAM

Problème	Cause	Solution
Fichier VSAM vide après REPRO	LRECL incompatible	Passer à 80 octets pour tous les enregistrements

Programmation COBOL-CICS (Parties 2 et 3)

Problème	Cause	Solution	Programme(s)
Écrasement données saisies	MODE=INOUT partage zones I/O (suffixes I et O)	Sauvegarder dans WS-SAISIE immédiatement après RECEIVE MAP, avant tout MOVE LOW-VALUES	PRGAJT
PERFORM sans THRU (GO TO bug)	GO TO vers paragraphe-FIN sort de la plage du PERFORM	Utiliser PERFORM ... THRU paragraphe-FIN pour inclure le paragraphe de sortie	PRGAJT, PRGLGEN, PRGSTAT
Message erreur non visible	SEND MAP sans ERASE ne rafraîchit pas l'écran	Ajouter ERASE au SEND MAP d'erreur	PRGAJT
LINKAGE SECTION manquante	DFHCOMMAREA non déclarée	Ajouter LINKAGE SECTION avec DFHCOMMAREA pour accéder aux données du RETURN précédent	PRGMAJ
Données effacées après MAJ partielle	Champs non modifiés ont longueur = 0	Fusionner avec les données actuelles : si longueur > 0, utiliser la saisie ; sinon, conserver l'existant	PRGMAJ
Deadlock DELETE pendant browse	DELETE demande verrou exclusif pendant STARTBR actif	Collecter les clés en table (max 100), ENDBR, puis DELETE en boucle	PRGDELG
COMMAREA non réinitialisée	Contexte précédent conservé si aucun résultat	Réinitialiser WS-COMMAREA quand aucun client trouvé	PRGLGEN
NUMVAL non supporté	FUNCTION NUMVAL incompatible avec MOVE sur IBM Enterprise COBOL	Utiliser REDEFINES pour réinterpréter la zone alphanumérique comme numérique	PRGSTAT
Lignes COBOL > 72 colonnes	Messages trop longs dans le source	Raccourcir les messages ou les découper sur plusieurs lignes	PRGSTAT

Administration CICS

Problème	Cause	Solution
CEDA INSTALL GROUP échoue	Fichier FCLIENT déjà ouvert	Installer uniquement la ressource spécifique : CEDA INSTALL TRANSACTION(xxx)
CEMT INQ MAPSET inexistant	Commande non disponible pour les mapsets	Utiliser CEDA VIEW MAPSET(xxx) à la place
Justification à droite des clés	Attribut NUM sur champ préfixe	Utiliser PIC X sans NUM pour les champs de clé partielle

Assemblage BMS et JCL (Partie 3)

Problème	Cause	Solution
Erreurs assemblage BMS (CLILIST)	Structure MAP trop complexe	Simplifier la MAP en réduisant le nombre de champs ou en utilisant des noms plus courts
Format JCL incorrect	Paramètres mal positionnés dans ASMLIST/CMPLGEN	Corriger l'alignement et la syntaxe des cartes JCL

Compétences mises en œuvre

VSAM et CICS :

- Définition de fichiers VSAM KSDS (IDCAMS)
- Création d'AIX (Alternate Index) et PATH pour accès par clé alternative
- Intégration fichiers dans CICS (FCT - File Control Table)
- Commandes CICS : READ, WRITE, REWRITE, DELETE
- Navigation VSAM : STARTBR, READNEXT, ENDBR

BMS et Écrans :

- Conception d'écrans BMS (Basic Mapping Support)
- Gestion des attributs (ASKIP, UNPROT, BRT)
- Commandes SEND MAP, RECEIVE MAP

Programmation COBOL-CICS :

- Mode pseudo-conversationnel (RETURN TRANSID, COMMAREA)
- LINKAGE SECTION pour DFHCOMMAREA
- Validation et contrôle des données saisies
- Gestion des erreurs (RESP, DFHRESP)

Administration :

- Définition de transactions (CEDA DEFINE)
- Installation de ressources (CEDA INSTALL)
- Débogage avec CEDF

Conclusion

Ce projet m'a permis de mettre en pratique l'ensemble des compétences acquises durant la formation POEI Mainframe COBOL pour le volet CICS. À travers les différentes parties du projet, j'ai pu :

- **Maîtriser VSAM sous CICS** : Définition de fichiers KSDS, création d'index alternatifs (AIX) avec PATH pour l'accès par clé secondaire, intégration dans la FCT (File Control Table), et gestion des opérations de lecture, écriture, mise à jour et suppression.
- **Développer des écrans BMS** : Conception de MAPs avec gestion des attributs (couleurs, protection), zones de saisie et d'affichage, messages d'erreur.
- **Programmer en COBOL-CICS** : Utilisation des commandes CICS (SEND/RECEIVE MAP, READ, WRITE, REWRITE, DELETE), gestion pseudo-conversationnelle avec RETURN TRANSID, et navigation VSAM avec STARTBR/READNEXT/ENDBR.
- **Administrer les transactions** : Définition via CEDA, installation de groupes, tests avec CEDF.

Le projet couvre un cas concret de gestion clientèle dans le secteur financier, avec **7 programmes COBOL-CICS**, **7 MAPs BMS**, **7 transactions** et **2 fichiers VSAM** (FCLIENT pour l'accès direct, PCLIENT via AIX/PATH pour les statistiques par région). Les principales difficultés rencontrées (gestion des attributs BMS, validation des données, navigation VSAM, fusion des modifications, deadlock lors de suppressions multiples, optimisation des accès via AIX) m'ont permis de développer une approche méthodique de résolution de problèmes.

Cette expérience constitue une base solide pour aborder des projets mainframe transactionnels en entreprise.

Annexes

A. Vue d'ensemble du projet

Ces captures montrent l'ensemble des ressources créées au cours du projet.

Ressources CICS - Groupe CLIGROUP

Le groupe CLIGROUP contient toutes les ressources définies pour l'application de gestion clientèle.

```
DISPLAY GROUP(CLIGROUP)
ENTER COMMANDS
```

NAME	TYPE	GROUP	LAST CHANGE
FCLIENT	FILE	CLIGROUP	01/13/26 10:54:32
PCLIENT	FILE	CLIGROUP	01/19/26 12:08:04
CLIAFF	MAPSET	CLIGROUP	01/12/26 17:59:42
CLIAJT	MAPSET	CLIGROUP	01/13/26 14:31:46
CLIDEL	MAPSET	CLIGROUP	01/16/26 11:12:15
CLILIST	MAPSET	CLIGROUP	01/16/26 15:34:33
CLIMAJ	MAPSET	CLIGROUP	01/15/26 11:30:18
CLISTAT	MAPSET	CLIGROUP	01/19/26 12:10:47
CLISUP	MAPSET	CLIGROUP	01/15/26 15:51:20
PRGAJT	PROGRAM	CLIGROUP	01/13/26 14:33:36
PRGCLIA	PROGRAM	CLIGROUP	01/12/26 17:06:28
PRGDELG	PROGRAM	CLIGROUP	01/16/26 11:56:18
PRGLGEN	PROGRAM	CLIGROUP	01/16/26 15:36:04
PRGMAJ	PROGRAM	CLIGROUP	01/15/26 12:16:13
PRGSTAT	PROGRAM	CLIGROUP	01/19/26 12:11:45
PRGSUP	PROGRAM	CLIGROUP	01/15/26 17:11:17
+ AFFI	TRANSACTION	CLIGROUP	01/12/26 17:09:12

Vue des ressources du groupe CLIGROUP : 2 fichiers (FCLIENT, PCLIENT), 7 mapsets BMS (CLIAFF à CLISTAT), et les premiers programmes COBOL.

```
DISPLAY GROUP(CLIGROUP)
ENTER COMMANDS
```

NAME	TYPE	GROUP	LAST CHANGE
+ CLISUP	MAPSET	CLIGROUP	01/15/26 15:51:20
PRGAJT	PROGRAM	CLIGROUP	01/13/26 14:33:36
PRGCLIA	PROGRAM	CLIGROUP	01/12/26 17:06:28
PRGDELG	PROGRAM	CLIGROUP	01/16/26 11:56:18
PRGLGEN	PROGRAM	CLIGROUP	01/16/26 15:36:04
PRGMAJ	PROGRAM	CLIGROUP	01/15/26 12:16:13
PRGSTAT	PROGRAM	CLIGROUP	01/19/26 12:11:45
PRGSUP	PROGRAM	CLIGROUP	01/15/26 17:11:17
AFFI	TRANSACTION	CLIGROUP	01/12/26 17:09:12
AJOU	TRANSACTION	CLIGROUP	01/13/26 14:34:55
DELG	TRANSACTION	CLIGROUP	01/16/26 12:00:01
LGEN	TRANSACTION	CLIGROUP	01/16/26 15:37:08
MAJO	TRANSACTION	CLIGROUP	01/15/26 12:30:37
STAT	TRANSACTION	CLIGROUP	01/19/26 12:12:39
SUPP	TRANSACTION	CLIGROUP	01/15/26 17:18:45

Suite des ressources : les 7 programmes (PRGAJT à PRGSUP) et les 7 transactions (AFFI, AJOU, DELG, LGEN, MAJO, STAT, SUPP).

Datasets sur z/OS

L'ensemble des fichiers du projet sont organisés dans l'espace de noms ROCHA.CICS.

```

DSLIS - Data Sets Matching ROCHA.CICS                               Row 1 of 10
Command ==> _____ Scroll ==> CSR

Command - Enter "/" to select action                               Message                               Volume
-----
      ROCHA.CICS.CLIENT                                           *VSAM*
      ROCHA.CICS.CLIENT.AIX                                       *AIX *
      ROCHA.CICS.CLIENT.AIX.DATA                                  FDDBAS
      ROCHA.CICS.CLIENT.AIX.INDEX                                FDDBAS
      ROCHA.CICS.CLIENT.DATA                                      FDDBAS
      ROCHA.CICS.CLIENT.INDEX                                    FDDBAS
      ROCHA.CICS.CLIENT.PATH                                     *PATH*
      ROCHA.CICS.LINK                                             FDDBAS
      ROCHA.CICS.LOAD                                             FDDBAS
      ROCHA.CICS.SOURCE                                          FDDBAS
***** End of Data Set list *****

```

Liste des 10 datasets du projet : CLIENT (VSAM KSDS), AIX (index alternatif), PATH, les composants VSAM (DATA, INDEX), et les bibliothèques (LINK, LOAD, SOURCE).

Bibliothèque des sources - ROCHA.CICS.SOURCE

Cette bibliothèque contient tous les codes sources : BMS, COPY, JCL et programmes COBOL.

```

EDIT                               ROCHA.CICS.SOURCE                               Row 00001 of 00030
Command ==> _____ Scroll ==> CSR

      Name      Prompt      Size  Created      Changed      ID
-----
      ASMAJT      27  2026/01/13  2026/01/13  12:44:43  IBMUSER
      ASMCLAF      27  2026/01/12  2026/01/12  14:40:19  IBMUSER
      ASMDEL      20  2026/01/16  2026/01/16  11:10:55  IBMUSER
      ASMLIST      24  2026/01/16  2026/01/16  14:57:51  IBMUSER
      ASMMAJ      32  2026/01/15  2026/01/15  11:25:10  IBMUSER
      ASMSTAT      27  2026/01/19  2026/01/19  10:59:08  IBMUSER
      ASMSUP      31  2026/01/15  2026/01/15  15:48:49  IBMUSER
      CLIAFF      96  2026/01/12  2026/01/12  14:54:21  IBMUSER
      CLIAJT     100  2026/01/13  2026/01/13  13:01:55  IBMUSER
      CLIDEL      69  2026/01/16  2026/01/16  12:13:41  IBMUSER
      CLILIST     140  2026/01/16  2026/01/16  15:12:27  IBMUSER
      CLIMAJ     115  2026/01/15  2026/01/15  11:23:14  IBMUSER
      CLISTAT      96  2026/01/19  2026/01/19  10:57:40  IBMUSER
      CLISUP     114  2026/01/15  2026/01/16  10:55:37  IBMUSER
      CMPAJT      33  2026/01/13  2026/01/13  13:19:26  IBMUSER
      CMPCLAF      33  2026/01/12  2026/01/12  15:55:32  IBMUSER

```

Membres sources : ASM (JCL d'assemblage BMS), CLI* (sources BMS des mapsets), CMP* (JCL de compilation), ainsi que les tailles et dates de modification.*

EDIT	ROCHA.CICS.SOURCE						Row 00017 of 00030
Command ==>							Scroll ==> CSR
	Name	Prompt	Size	Created	Changed		ID
	CMPDELG		38	2026/01/16	2026/01/16 11:29:21		IBMUSER
	CMPLGEN		38	2026/01/15	2026/01/16 15:33:49		IBMUSER
	CMPMAJ		38	2026/01/15	2026/01/15 12:13:07		IBMUSER
	CMPSTAT		43	2026/01/19	2026/01/19 11:16:30		IBMUSER
	DEFPATH		94	2026/01/19	2026/01/19 10:40:32		IBMUSER
	DEFVSAM		57	2025/11/25	2026/01/12 12:37:38		IBMUSER
	LOADVSAM		65	2026/01/12	2026/01/12 12:42:31		IBMUSER
	PRGAJT		395	2026/01/13	2026/01/15 10:15:05		IBMUSER
	PRGCLIA		282	2026/01/12	2026/01/13 10:04:08		IBMUSER
	PRGDELG		586	2026/01/16	2026/01/16 13:03:04		IBMUSER
	PRGLGEN		643	2026/01/16	2026/01/16 17:40:57		IBMUSER
	PRGMAJ		618	2026/01/15	2026/01/15 14:47:39		IBMUSER
	PRGSTAT		406	2026/01/19	2026/01/19 12:25:37		IBMUSER
	PRGSUP		454	2026/01/15	2026/01/15 17:07:35		IBMUSER
	End						

Suite des membres : CMP (compilations), DEFPATH (définition AIX/PATH), DEFVSAM (définition cluster), LOADVSAM (chargement données), et les 7 programmes PRG* (PRGAJT, PRGCLIA, PRGDELG, PRGLGEN, PRGMAJ, PRGSTAT, PRGSUP).*

Bibliothèque des modules - ROCHA.CICS.LOAD

Cette bibliothèque contient les programmes compilés (load modules) prêts à être exécutés.

EDIT	ROCHA.CICS.LOAD						Row 00001 of 00014
Command ==>							Scroll ==> CSR
	Name	Prompt	Alias-of	Size	TTR	AC	AM RM
	CLIAFF			00000690	000021	00	31 24
	CLIAJT			00000608	000112	00	31 24
	CLIDEL			00000330	000E1C	00	31 24
	CLILIST			00000E70	000F1C	00	31 24
	CLIMAJ			00000628	000C02	00	31 24
	CLISTAT			00000500	001115	00	31 24
	CLISUP			000006F0	000D18	00	31 24
	PRGAJT			00002070	000B1B	00	31 ANY
	PRGCLIA			00001C58	000103	00	31 ANY
	PRGDELG			00002978	000F06	00	31 ANY
	PRGLGEN			00002CC8	001106	00	31 ANY
	PRGMAJ			000027F8	000D09	00	31 ANY
	PRGSTAT			00002108	001206	00	31 ANY
	PRGSUP			000025F0	000D1F	00	31 ANY
	End						

14 modules load : 7 mapsets BMS compilés (CLI) et 7 programmes COBOL-CICS compilés (PRG*). On note la taille de chaque module et le mode d'adressage (AMODE 24/31).*

Bibliothèque des linkedits BMS - ROCHA.CICS.LINK

Cette bibliothèque contient les copybooks générés lors de l'assemblage BMS.

```

EDIT                                ROCHA.CICS.LINK                                Row 00001 of 00007
Command ==> _                        Scroll ==> CSR
      Name      Prompt      Size  Created      Changed      ID
      CLIAFF
      CLIAJT
      CLIDEL
      CLILIST
      CLIMAJ
      CLISTAT
      CLISUP
      **End**

```

7 copybooks BMS (CLI) générés par l'assembleur. Ces copybooks sont inclus dans les programmes COBOL via la COPY statement pour décrire la structure des écrans.*

B. Référence des commandes CICS

Opérations sur enregistrements

Commande	Usage	Prérequis
READ	Lecture directe par clé	Aucun
READ UPDATE	Lecture avec verrouillage	Pour REWRITE
WRITE	Ajout nouvel enregistrement	Le client ne doit PAS exister
REWRITE	Mise à jour enregistrement	READ UPDATE obligatoire dans même UOW
DELETE	Suppression enregistrement	Aucun

Navigation VSAM (Browse)

Commande	Usage
STARTBR	Positionner le curseur sur une clé (partielle) avec GTEQ
READNEXT	Lire l'enregistrement suivant
ENDBR	Terminer le parcours et libérer les ressources

Écrans BMS

Commande	Usage
SEND MAP	Afficher un écran à l'utilisateur
RECEIVE MAP	Recevoir les données saisies
RETURN TRANSID	Retour pseudo-conversationnel
RETURN	Fin de transaction

C. Liste des programmes COBOL-CICS

Programme	Transaction	Commandes	Fichier	Description
PRGCLIA	AFFI	READ	FCLIENT	Affichage d'un client
PRGAJT	AJOU	WRITE	FCLIENT	Ajout d'un nouveau client
PRGMAJ	MAJO	READ UPDATE, REWRITE	FCLIENT	Mise à jour d'un client
PRGSUP	SUPP	READ, DELETE	FCLIENT	Suppression d'un client (avec affichage)
PRGDELG	DELG	STARTBR, READNEXT, DELETE	FCLIENT	Suppression générique par préfixe
PRGLGEN	LGEN	STARTBR, READNEXT	FCLIENT	Liste générique paginée (10 clients/page)
PRGSTAT	STAT	STARTBR, READNEXT	PCLIENT	Statistiques par région via AIX/PATH

D. Liste des MAPs BMS

Mapset	Map	Programme	Description
CLIAFF	MAPAFF	PRGCLIA	Écran d'affichage client
CLIAJT	MAPAJT	PRGAJT	Écran d'ajout client
CLIMAJ	MAPMAJ	PRGMAJ	Écran de mise à jour
CLISUP	MAPSUP	PRGSUP	Écran de suppression
CLIDEL	MAPDEL	PRGDELG	Écran de suppression générique
CLILIST	MAPLGEN	PRGLGEN	Écran de liste générique paginée
CLISTAT	MAPSTAT	PRGSTAT	Écran de statistiques par région

E. Liste des transactions CICS

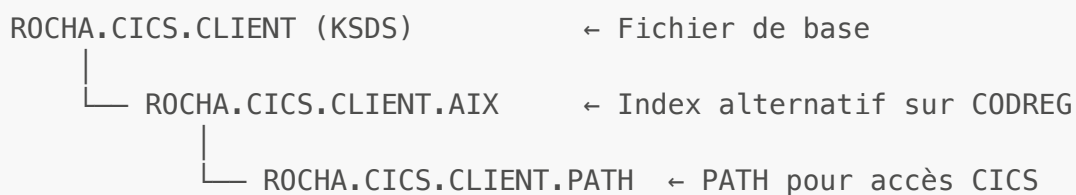
Code	Programme	Description
AFFI	PRGCLIA	Affichage client
AJOU	PRGAJT	Ajout client
MAJO	PRGMAJ	Mise à jour client
SUPP	PRGSUP	Suppression client
DELG	PRGDELG	Suppression générique

Code	Programme	Description
LGEN	PRGLGEN	Liste générique paginée
STAT	PRGSTAT	Statistiques par région

F. Liste des fichiers VSAM

Nom CICS	Dataset	Type	Description
FCLIENT	ROCHA.CICS.CLIENT	KSDS	Fichier de base (clé primaire : NUMCPT)
PCLIENT	ROCHA.CICS.CLIENT.PATH	PATH	Accès via AIX sur CODREG

Architecture VSAM :



G. Liste des JCL

JCL	Description	Exercice
DEFVSAM.jcl	Définition du cluster VSAM KSDS	Ex 1
LOADVSAM.jcl	Chargement des données initiales	Ex 1
ASMCLIA.jcl	Assemblage MAP CLIAFF	Ex 3
CMPCLIA.jcl	Compilation PRGCLIA	Ex 5
ASMAJT.jcl	Assemblage MAP CLIAJT	Ex 7
CMPAJT.jcl	Compilation PRGAJT	Ex 8
ASMMAJ.jcl	Assemblage MAP CLIMAJ	Ex 10
CMPTMAJ.jcl	Compilation PRGMAJ	Ex 11
ASMSUP.jcl	Assemblage MAP CLISUP	Ex 13
CMPSUP.jcl	Compilation PRGSUP	Ex 14
ASMDEL.jcl	Assemblage MAP CLIDEL	Ex 17
CMDELG.jcl	Compilation PRGDELG	Ex 17
ASMLIST.jcl	Assemblage MAP CLILIST	Ex 18
CMPLGEN.jcl	Compilation PRGLGEN	Ex 18

JCL	Description	Exercice
DEFPATH.jcl	Définition AIX et PATH sur CODREG	Ex 19
ASMSTAT.jcl	Assemblage MAP CLISTAT	Ex 19
CMPSTAT.jcl	Compilation PRGSTAT	Ex 19

H. Structure du fichier CLIENT (80 octets)

Position	Champ	Type	Longueur	Description
01-06	NUMCPT	NUM	6	Numéro compte (clé)
07-08	CODREG	NUM	2	Code région (01-04)
09-10	NATCPT	NUM	2	Nature compte
11-20	NOM	ALPHA	10	Nom client
21-30	PRENOM	ALPHA	10	Prénom client
31-38	DATNAISS	NUM	8	Date naissance (AAAAMMJJ)
39	SEXE	ALPHA	1	Sexe (M/F)
40-41	ACTPRO	NUM	2	Activité professionnelle
42	SITSO	ALPHA	1	Situation sociale (C/M/D/V)
43-52	ADRESSE	ALPHA	10	Adresse
53-62	SOLDE	NUM	10	Solde
63-64	POSITION	ALPHA	2	Position (DB/CR)
65-80	FILLER	-	16	Réserve

I. Messages d'erreur standards

Message	Contexte
ENREGISTREMENT EN DOUBLE	Ajout d'un client existant
ZONE NUMERIQUE, RESAISIR CE CHAMP	Champ numérique invalide
REGION INEXISTANTE, SAISIR CODE REGION	Code région invalide
CLIENT INEXISTANT	Recherche sans résultat
SUPPRESSION EFFECTUEE	Confirmation suppression
MISE A JOUR EFFECTUEE	Confirmation mise à jour

